

考虑预测误差与日内调整的微网购电优化策略

摘 要

针对研究背景中负荷、光伏与电价逐步不确定、储能跨时段耦合以及购电合同受限等难点，本文以线性规划（LP）和模型预测控制（MPC）为核心，融合历史预测、经验分位风险修正、滚动优化与闭环仿真，建立“日前计划—日内执行—实际结算—稳健性检验”的微网购电优化框架。

针对问题一，基于典型日全信息建立以购电费用最小为目标的线性规划（LP）模型，综合考虑功率平衡、储能荷电状态递推、充放电功率上限和日末状态闭环约束。调用 HiGHS 线性规划求解器得到典型日最优购电量为 59482.70 kWh，购电费用为 35126.95 元，且储能始末均为 6000 kWh。

针对问题二，引入全年连续运行、每日 0:00 签订合同且日内不可调整、缺口按 5 倍电价应急购电等约束，采用 F1/F2 历史预测并以净负荷残差经验分位进行风险修正。基于报童模型得到理论分位 0.8，主方案取 $W = 35, \alpha = 0.805, H = 49$ ，滚动 LP 完成 334 天闭环仿真；年度现金总费用为 14786101.35 元，应急购电量为 245483.29 kWh。

针对问题三，针对全年 334 天、144 个 10 分钟时段及多个日内信息更新节点的规模特征，设计“日前合同外层优化 + 日内滚动 LP 内层执行”的分层框架，在 0:00、6:00、12:00 和 18:00 依据新光伏预报与实际储能状态调整尚未执行合同。主方案采用 $W = 35, \alpha = 0.7725, H = 6$ ，总费用为 14664345.40 元，相对不使用日内新预报的方案减少 566030.72 元。

针对问题四，引入实际电价随机波动及其历史可得性约束，沿用 F1 负荷预测与官方光伏预报，采用近 30 日价格中位数预测，并通过固定合同与可调合同的闭环仿真比较扩展性。结果表明，固定合同和可调合同费用分别为 17378687.69 元和 15465962.60 元；允许日内调整后，应急购电量下降 63.14%，说明合同灵活性能显著削弱价格与源荷预测误差的叠加风险。

针对研究背景，本文模型将物理约束、信息边界与费用结算统一到可审计的时序框架中，兼顾可行性、经济性和风险控制；其 LP、MPC 与经验分位方法可推广至含储能的园区微网、综合能源系统及其他滚动购电决策场景。

关键词：微网购电；线性规划；风险修正；滚动优化；合同调整；波动电价

一 问题重述与分析

1.1 问题背景与任务拆解

微网系统包含光伏电源、用电负荷、储能装置以及对外购电通道。光伏出力与用电负荷存在明显日内波动；储能能够跨时段转移电能，但运行受容量、充放电功率及充放电损耗约束。

购电合同制定存在两难权衡：若日前合同购电量偏低，功率缺口需要高价应急购电；若合同电量过多，富余电量无法全额退款。因此，本题并非简单峰谷套利，而是受预测误差、购电合同规则、储能状态、预报更新机制共同约束的不确定性时序决策问题。建模严格遵守非前视约束：决策仅能使用当前时刻之前可获取的信息，不得引入未来负荷、光伏、电价的真实数据。

全文建模与求解的技术流程如图1所示。

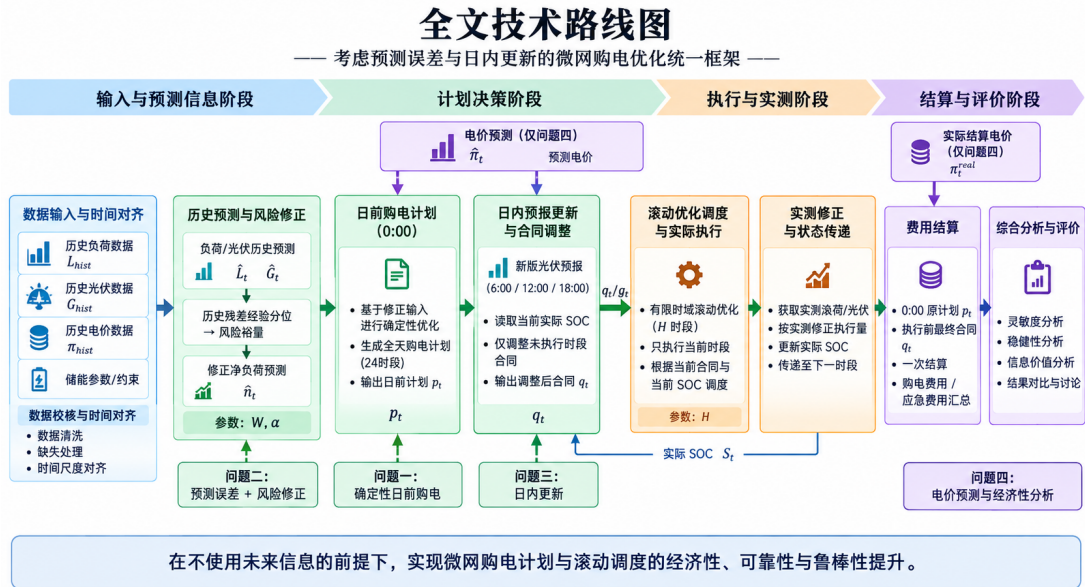


图 1: 全文建模与求解技术流程

全年数据的总体结构与季节性特征如图2所示，为后续按日连续仿真提供数据背景。

四道子问题逐层递进，各司其职：

1. 问题一：给定典型日源荷与固定电价，约束储能日始末电量一致，用于确定系统物理边界，构建确定性调度基准。
2. 问题二：拓展至全年时序数据，仅允许每日零点制定购电计划，不支持日内调账；需要完成源荷预测，预留不确定性余量，保证储能状态跨日连续传递。

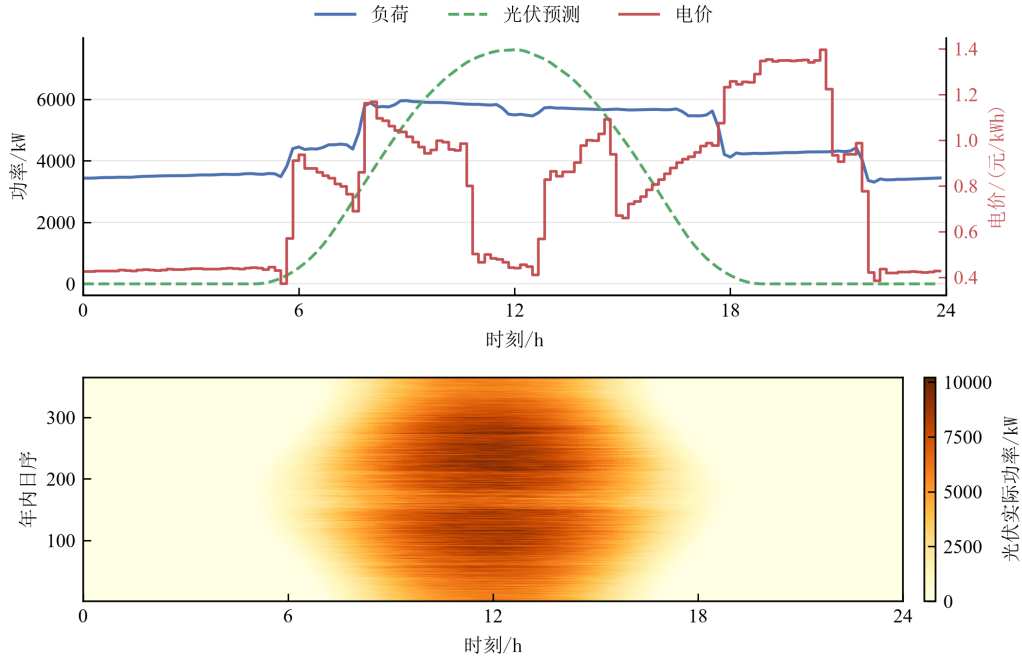


图 2: 全年负荷、光伏与电价数据概览及光伏季节性。

3. 问题三：每日 0、6、12、18 时更新光伏预报，允许调整尚未执行的购电合同；建模需区分“决策时刻已知信息”与事后真值，按赛题规则核算调账成本。
4. 问题四：电价为时变序列，分别构建无调账、支持调账两套策略；电价仅依靠历史数据预测，结算阶段才采用真实电价。

问题内部存在三组相互制衡的目标：常规购电成本与应急高价购电成本；供电可靠性与光伏消纳能力；短期调度开销与储能长期调度价值。本文以总购电费用为核心优化目标，同时统计应急购电量、闲置合同电量、弃光量与储能期末电量。

为保证多方案横向对比有效，统一设置仿真基准：储能从 1 月 1 日同一初始状态启动，1 月作为模型预热期；正式统计周期统一选取 2 月 1 日至 12 月 31 日。问题二、三之间存在预报信息源差异，本文增设对照实验，区分预报更新频次与数据源带来的收益差异。

1.2 评价标准与难点分析

本文采用多维度指标评价方案效果：经济指标包含总购电费用、应急购电量、闲置合同电量、弃光量及储能终态电量；预测效果采用 MAE、RMSE、MAPE 与区间覆盖率衡量。由于源荷属于时序相关数据，不采用独立同分布假设下的显著性检验。

综合分析，本课题核心难点在于：在非前视信息限制下，兼顾预测不确定性与储能跨时段耦合约束，在多冲突目标之间寻找可行调度策略。

二 模型假设、数据口径与基础物理模型

2.1 模型基本假设

为构建贴合赛题计费规则、物理可行且可复现的微网时序优化模型，本文在不改变题目核心约束的前提下设置如下建模假设：

1. **电网交易规则**：微网仅支持向上级电网购电，无反向售电能力。富余光伏可弃光且无惩罚；已签约的日前合同电量若未完全消纳，作为闲置弃购电量按原价计费，电网不予以补偿抵扣。
2. **储能参数约束**：储能额定容量为 12000 kWh，有效运行区间为 [1200, 10800] kWh。单时段最大充、放电功率均为 5000 kW，充放电效率取 $\eta_c = \eta_d = 0.9$ 。后续通过效率参数替换完成敏感性分析。
3. **调度行为约束**：同一调度时段不存在物理意义上的同时充放电行为。求解过程中出现的极小数值共生解将通过次级目标筛选剔除。
4. **时序仿真规则**：问题一采用典型日闭环调度，约束日初与日末 SOC 相等；问题二至四采用全年连续仿真，取消每日电量闭环约束。引入终端价值罚项抑制有限时域优化短视性，仅用于优化引导，不参与费用统计。
5. **非前视信息约束（核心）**：全文严格遵循非前视决策准则，所有预测、购电计划制定、合同调账仅可使用决策时刻之前的已知信息，禁止使用未来真实源荷、电价数据。
6. **成本核算边界**：题目未给出设备损耗、潮流约束及老化成本，模型仅统计合同购电费用与应急购电费用，储能吞吐作为次级优化指标以提升调度平滑性。

核心交易规则、非前视信息集为刚性边界假设；储能参数、数值处理规则为可检验建模设定，可通过敏感性分析验证稳健性。

2.2 统一时间口径与量纲转换

本文采用题目标准 10 分钟调度时序，单时段时长 $\Delta t = \frac{1}{6}$ h，通过 $E = P \cdot \Delta t$ 统一将功率量纲转换为电量，保证储能递推、能量平衡与费用计算量纲一致。

数据时刻标签为时段结束时刻，第 t 行数据对应区间 $[(t-1) \times 10, t \times 10)$ min，实现全日时序无缝衔接。针对官方输出模板存在的时序偏移问题，本文仿真计算、SOC 更新均基于原始时序逻辑执行，并设置对照实验完成稳健性检验。

2.3 基础功率平衡物理模型

结合微网源储荷耦合特性，构建适用于全场景的时序优化模型。定义 L_t 为时段负荷电量， V_t 为可用光伏电量， g_t 为生效合同购电量， w_t 为闲置弃购电量， v_t 为实际光伏消纳量， x_t, y_t 为储能充、放电电量， u_t 为应急购电量， S_t 为时段初始储能电量。

时段能量平衡方程：

$$g_t - w_t + v_t + y_t + u_t = L_t + x_t$$

储能荷电状态递推关系：

$$S_{t+1} = S_t + \eta_c x_t - \frac{y_t}{\eta_d}, \quad \eta_c = \eta_d = 0.9$$

系统约束条件：

$$\begin{cases} 0 \leq v_t \leq V_t, & 0 \leq w_t \leq g_t, & u_t \geq 0 \\ 0 \leq x_t, y_t \leq 833.33 \\ 1200 \leq S_t \leq 10800 \end{cases}$$

全年仿真采用连续 SOC 跨日传递机制，以前一日末电量作为次日初值。本文以 1 月为模型预热期，2—12 月为正式统计周期，规避每日重置 SOC 导致的仿真失真。

2.4 模型符号说明

本文引入闲置弃购变量 w_t ，严格区分合同采购电量与实际入网电量，精准匹配“按量计费、富余不退”的题目规则，保证能量平衡与经济核算逻辑自洽。

信息集定义为

$I_{d,t} = \{\text{时隙 } t \text{ 前已完成的源荷与电价观测, } S_{d,t}, \text{ 截至 } t \text{ 已发布的预测、合同与模型参数}\}.$

因此， $I_{d,t}$ 不包含第 d 日尚未发生时隙的真实负荷、光伏或电价。所有计划与调度变量必须是 $I_{d,t}$ 的函数；真实值仅在该时隙决策完成后进入执行修正与结算。

三 问题一：确定性典型日购电优化模型

3.1 问题分析与模型构建：确定性典型日基准调度场景

问题一提供完整典型日的负荷、光伏出力与分时电价数据，全部源荷电价信息在调度起始时刻已知。该场景要求储能日内电量闭环，属于确定性线性规划基准算例，用于校验本文微网基础模型的正确性。

表 1: 模型符号说明

符号	单位	含义说明
决策变量		
g_t, p_t, q_t	kWh	生效合同、0:00 日前合同、调账后合同购电量
v_t, x_t, y_t, u_t, w_t	kWh	光伏消纳、储能充电、储能放电、应急购电、闲置弃购量
δ_t^+, δ_t^-	kWh	合同上调量、下调量
参数与状态量		
L_t, V_t, S_t	kWh	负荷电量、可用光伏电量上限、时段初始储能电量
$S_{\min}, S_{\max}, B_{\max}$	kWh	储能电量下限、上限与单时段充放电功率上限
c_t	元/kWh	分时计划购电单价；应急电单价为 $5c_t$
η_c, η_d	—	储能充电、放电效率
$\hat{L}_{d,t}, \hat{V}_{d,t}$	kWh	负荷、光伏点预测值
$\tilde{L}_{d,t}, \tilde{V}_{d,t}$	kWh	叠加风险裕度后的保守预测值
$\hat{N}_{d,t}, \tilde{N}_{d,t}$	kWh	中心净负荷、安全净负荷预测值
$e_{d,t}^L, e_{d,t}^V, e_{d,t}^N$	kWh	负荷、光伏、净负荷预测残差
$Q_\alpha, J(p), A_t$	kWh; 元; kWh	经验分位数、单时段合同费用函数、预测盈余
$W, \alpha, H, \lambda, \varepsilon$	—; —; 时 段; 元/kWh; 元/kWh	残差窗口、风险分位、MPC 时域、终端价值系数、 辅助微小罚项
集合与信息		
d, t, j, h	—	日、10 分钟时段、历史日、预测提前期索引
$\mathcal{H}_t$	—	时隙 t 起始的滚动优化时域
$I_{d,t}$	—	第 d 日第 t 时隙决策时可获得的信息集
$\mathcal{F}_{d,h}$	—	第 d 日提前期 h 的历史可用预测信息集

复用第三章的通用微网物理模型，针对本问题做场景限定：关闭应急购电 $u_t = 0$ ，闲置弃购量 $w_t = 0$ ，施加储能日闭环约束 $S_1 = S_{145} = 6000$ kWh (S_t 为 t 时段末储能荷电量)。模型功率平衡与储能状态递推约束如下：

$$g_t + v_t + y_t = L_t + x_t, \quad (1)$$

$$S_{t+1} = S_t + 0.9x_t - \frac{y_t}{0.9}, \quad (2)$$

$$0 \leq v_t \leq V_t, \quad 0 \leq x_t, y_t \leq 833.33, \quad (3)$$

$$1200 \leq S_t \leq 10800. \quad (4)$$

式中： g_t 为 t 时段电网购电量； v_t 为本时段光伏出力； y_t 为储能放电功率； L_t 为负荷功率； x_t 为储能充电功率； V_t 为光伏最大可用出力。

模型主目标为最小化全天购电费用：

$$\min \sum_{t=1}^{144} c_t g_t. \quad (5)$$

其中 c_t 代表 t 时段购电单价。

仅以费用作为目标时，最优解集合内可能出现无物理意义的微小同时充放电数值解。为此采用两阶段优化策略：第一步求解得到最优理论费用 f^* ；第二步在总购电费用不超过 $f^* + \varepsilon$ ($\varepsilon = 10^{-4}$ 元) 的约束下，以最小化储能吞吐总量作为次级目标：

$$\min \sum_{t=1}^{144} (x_t + y_t) \quad \text{s.t.} \quad \sum_{t=1}^{144} c_t g_t \leq f^* + \varepsilon. \quad (6)$$

两级优化问题均为线性规划，可直接调用求解器得到全局最优解。

命题 2 (连续 LP 松弛的可行性) 在电价非负、微网不可售电、无充放电补贴且充放电效率不超过 1 的条件下，费用最优解集合中必存在一个不同时充、放电的解。因此，无须在基础模型中引入充放电 0-1 互斥变量。

证明 若某费用最优解在时段 t 满足 $x_t > 0, y_t > 0$ ，取足够小的 $\epsilon > 0$ ，同步减少充电量与放电量，使 SOC 变化量保持不变：令 $x'_t = x_t - \epsilon/\eta_c$ ， $y'_t = y_t - \epsilon\eta_d$ 。则 $\eta_c x'_t - y'_t/\eta_d = \eta_c x_t - y_t/\eta_d$ ，SOC 递推不变；同时，因 $\eta_c \eta_d \leq 1$ ，该调整不会增加满足功率平衡所需的外购电量，也不会增加费用。重复该操作直至 $x'_t = 0$ 或 $y'_t = 0$ ，即可得到同费用且无同时充放电的可行解。对所有时段逐一处理即得结论。第二阶段最小化 $\sum_t (x_t + y_t)$ 仅用于在数值上选取该类解。证毕。本文采用两阶段线性规划方法进行求解。

3.2 仿真求解与结果验证：基准模型校验

求解得到典型日全天计划购电量 **59482.70 kWh**，总购电费用 **35126.95 元**。按题目表 1、表 2 格式整理的指定时段购电量与分段储能结果见附录 A；完整全天调度时序曲线见图3。

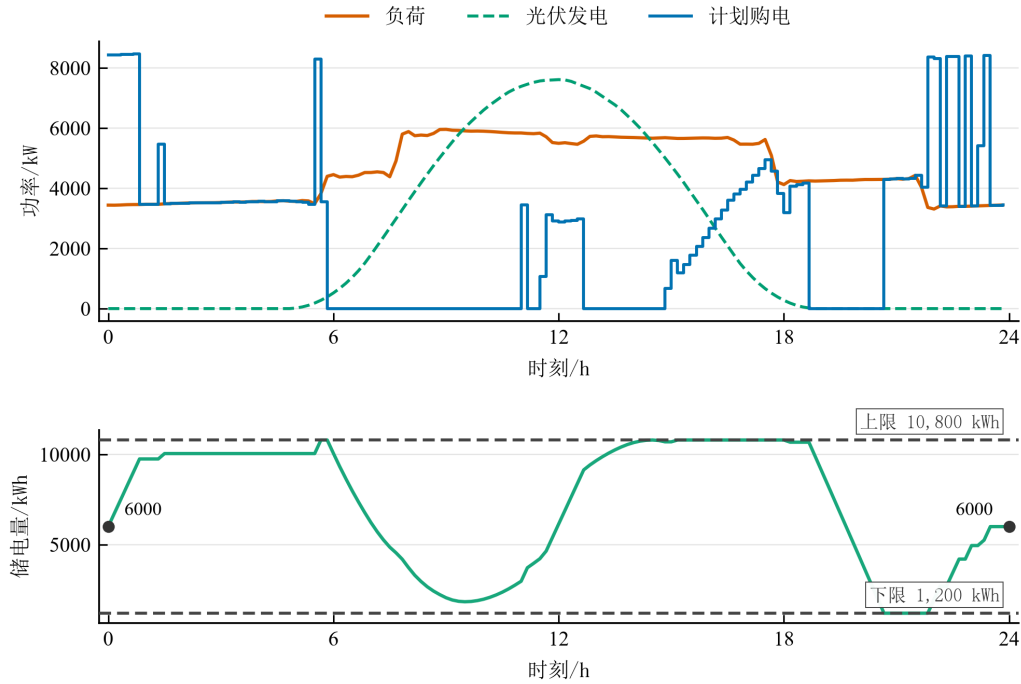


图 3: 典型日微网购电、储能充放电与荷电状态时序曲线

调度结果体现储能低电价充电、高电价放电的套利特性：凌晨低电价时段购电并给储能充电；光伏大发时段依靠光伏出力供给负荷，部分时段购电量降至 0；晚高峰电价抬升时，储能集中放电，降低高价时段购电需求。全天总充电量 20740.66 kWh，总放电量 16799.94 kWh，光伏全部就地消纳，无弃光。

逐时段功率平衡与 SOC 递推残差处于 10^{-13} 量级，属于求解器浮点计算误差范围；储能荷电量全程保持在安全区间内，同时满足日初-日末 SOC 闭环约束。该结果验证本文基础微网模型的正确性，为后续问题二、三、四的不确定性调度模型提供可靠基准。

四 问题二：基于日初预测的年度日前购电优化

4.1 问题分析与模型构建：源荷预测不确定性下的日前购电调度

问题一在典型日全信息条件下给出确定性最优调度，为全文物理模型提供基准；问题二将该模型推广到全年连续运行，并增加一条关键交易约束：微网只允许在每日 0:00

依据当时可得的预测签订全天购电合同 p_t ，日内不得修改合同，实测缺口只能按 5 倍电价应急购电。因此本问题的难点不再是峰谷套利，而是在只掌握历史信息条件下确定合同电量：合同购电量偏低会产生高价应急购电，偏高则已付费用无法退回。

全部决策严格服从因果信息边界。设第 d 日第 t 个 10 分钟时隙的可用信息集为

$$I_{d,t} = \{\text{时隙 } t \text{ 之前已完成的源荷观测, } S_{d,t}, \text{ 当前生效合同, 截至 } t \text{ 已发布的预测}\}, \quad (7)$$

则购电计划与储能动作均为 $I_{d,t}$ 的函数。调度日尚未发生的真实负荷与光伏出力不参与任何优化计算，真实值只在该时隙动作确定之后用于可行性裁剪、功率平衡校验与费用结算。

储能不再施加日闭环约束，荷电状态按跨日连续方式传递。仿真以 2025 年 1 月为预热期完成初始状态过渡，正式统计周期取 2 月 1 日至 12 月 31 日共 334 天。

本文对问题二的处理分为五个环节：先比较两组只使用历史数据的点预测方案并确定预测器；再对同一预测版本下的净负荷残差作逐槽经验分位修正，得到安全轨迹；随后以安全轨迹、真实日初储电量与分时电价求解 0:00 日前合同线性规划；日内以预测驱动的滚动时域线性规划（MPC 执行器）执行储能动作；最后代入真实源荷完成裁剪、平衡与结算。需要说明的是，本问题的风险建模不构造场景树，也不依赖任何场景生成步骤，而是直接对同一预测版本下的逐槽净负荷残差作经验分位修正，风险信息以单条安全轨迹的形式进入合同优化。技术流程如图4所示。

4.2 基于历史数据的候选预测模型与误差评价

所有预测只使用目标日之前已经完成的历史观测，记目标日为 d 、时段为 t 。F1 利用负荷的周周期与光伏的近期平均水平：

$$\hat{L}_{d,t}^{F1} = L_{d-7,t}, \quad (8)$$

$$\hat{V}_{d,t}^{F1} = \frac{1}{7} \sum_{k=1}^7 V_{d-k,t}. \quad (9)$$

F2 只把负荷部分改为三个滞后周的中位数，以抑制单周异常，光伏预测与 F1 完全相同：

$$\hat{L}_{d,t}^{F2} = \text{median}(L_{d-7,t}, L_{d-14,t}, L_{d-21,t}). \quad (10)$$

另设基于 90 日历史加权平均的模拟日模型 F0 作为历史基线，用于说明既有方案的误差水平。历史样本不足时按实际可用数据构造预测。

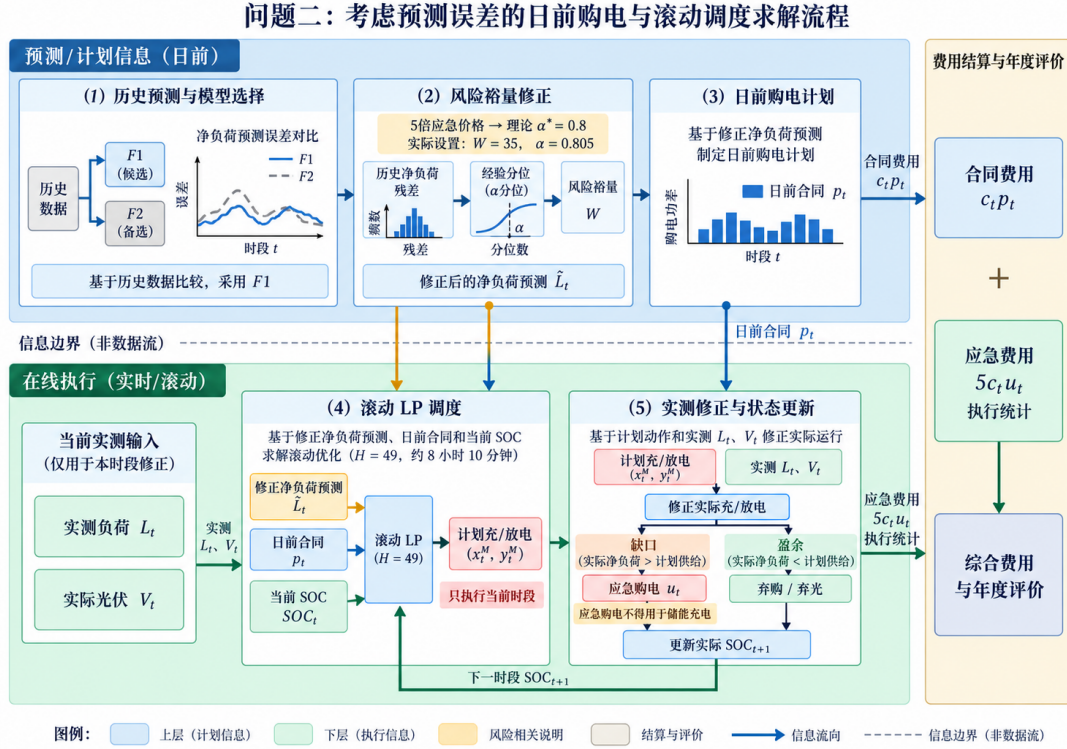


图 4: 问题二预测驱动日前购电与 MPC 执行流程

点预测精度用平均绝对误差与均方根误差衡量：

$$MAE = \frac{1}{n} \sum_{i=1}^n |z_i - \hat{z}_i|, \quad RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (z_i - \hat{z}_i)^2}. \quad (11)$$

三个候选方案在 2025 年 2 月 1 日至 12 月 31 日上的误差如表2所示。

表 2: 候选预测方案的误差对比

预测器	负荷 MAE	光伏 MAE	净负荷 MAE	净负荷 RMSE
F0	722.57	335.92	804.08	1074.83
F1	176.80	153.40	273.35	389.97
F2	187.90	153.40	284.35	403.25

单位：kW；F0 为历史基线。

由表2可得，F1 的净负荷 MAE 为 273.35 kW，比 F2 低 11.00 kW；净负荷 RMSE 为 389.97 kW，比 F2 低 13.28 kW。F1 与 F2 的唯一差别是负荷预测的构造方式，光伏部分完全一致，因此该差值可以单独归因于负荷预测方法。F0 的净负荷 MAE 为 804.08 kW，接近 F1 的三倍，说明长窗口加权平滑会同时削弱负荷的星期效应与光伏的天气效

应，故仅保留为历史基线。基于以上点预测误差，本文以 F1 作为主预测器，并在第4.5节用统一执行与结算口径下的闭环现金费用复核该选择。F1 与 F2 净负荷预测值与实际值的散点对比如图5所示，散点越接近对角参考线表示预测越准确。

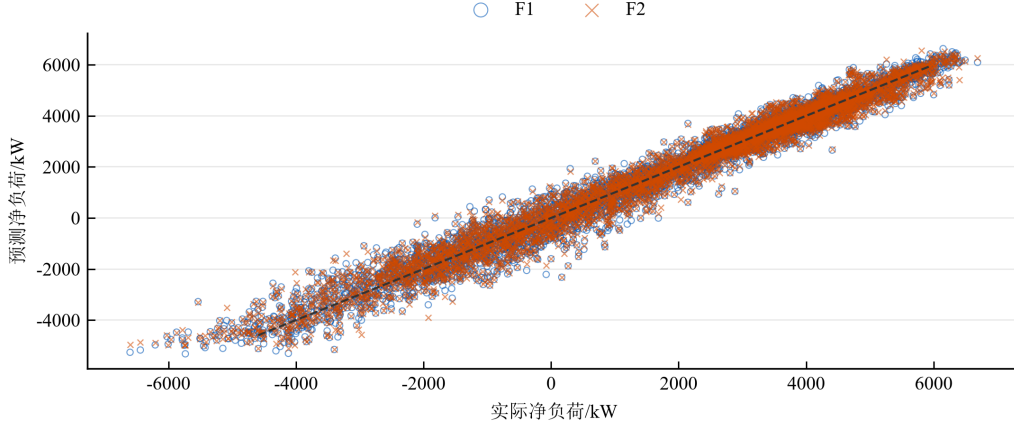


图 5: F1 与 F2 基础预测的净负荷预测值与实际值对比

4.3 净负荷残差的经验分位风险修正

记中心净负荷预测为 $\hat{N}_{d,t} = \hat{L}_{d,t} - \hat{V}_{d,t}$ ，同一预测版本下已完成日 $j < d$ 的逐槽残差定义为

$$e_{j,t}^N = N_{j,t} - \hat{N}_{j,t}, \quad j < d. \quad (12)$$

使用目标日之前最近 W 个已完成日的同槽残差经验分位构造安全净负荷：

$$\tilde{N}_{d,t} = \hat{N}_{d,t} + Q_{\alpha} \{e_{j,t}^N : d - W \leq j < d\}. \quad (13)$$

式(13)直接对残差的经验分布取分位，不要求残差服从正态分布，也不需要额外的场景生成步骤； W 控制所使用历史误差的时间范围， α 控制风险裕度的位置。加入分位裕度后净负荷可能出现负值，为保持重构后的负荷与光伏非负，按

$$\tilde{V}_{d,t} = \max(\hat{V}_{d,t}, -\tilde{N}_{d,t}, 0), \quad \tilde{L}_{d,t} = \tilde{V}_{d,t} + \tilde{N}_{d,t} \quad (14)$$

将其重新表示为非负的负荷预测与光伏预测。该处理只改变净负荷的分解方式，不改变 $\tilde{N}_{d,t}$ 本身，因而不影响供需缺口的计算。

分位水平 α 由计划电价与应急购电价格的比值确定。考虑单个时段，计划购电价格为 $c > 0$ 、计划购电量为 p ，该时段净负荷 N 为随机变量，剩余缺口按 $5c$ 应急购电，则该时段的期望费用为

$$J(p) = cp + 5c \mathbb{E}[(N - p)^+], \quad (N - p)^+ = \max(N - p, 0). \quad (15)$$

命题 2 (5 倍应急价格对应的报童临界分位) 在合同单位成本为 $c > 0$ 、缺电应急购电单价为 $5c$ 、富余合同不退款的条件下，使式(15)最小的计划购电量为 N 的 0.8 分位，即 $p^* = Q_{0.8}(N)$ 。

证明 记 N 的分布函数为 $F(\cdot)$ 。对式(15)关于 p 求导，

$$J'(p) = c - 5c \mathbb{P}(N > p) = c[1 - 5(1 - F(p))].$$

令 $J'(p) = 0$ 得 $F(p) = 1 - c/(5c) = 0.8$ ，即 $p^* = Q_{0.8}(N)$ 。再对 $J'(p)$ 求导得 $J''(p) = 5c f(p) \geq 0$ ，故 $J(p)$ 为凸函数，该驻点即全局最小值点； N 取离散分布时按次梯度理解，结论不变。证毕。

命题 2 的边际含义是：多购 1 kWh 计划电需支付 c ，少购 1 kWh 计划电则以 $1 - \alpha$ 的概率支付 $5c$ ，两种边际代价相等时分位水平为 $1 - 1/5 = 0.8$ 。实际计算中净负荷的分布未知，本文用最近 $W = 35$ 个已完成日的同槽历史残差经验分位数实现该分位水平；受有限样本影响，分位取值与理论值存在小幅差别，主配置取 $\alpha = 0.805$ 。需要明确该分位的适用边界：命题 2 针对单个时段的边际分布，0.8 是单槽风险基准，并不表示整日供电可靠度为 80%，也不保证计入储能跨时段耦合后合同量随分位单调变化，因此 W 与 α 的最终取值由第4.5节的参数敏感性确定。以夏至日为例，分位风险修正前后的净负荷预测与实际净负荷对比如图6所示。

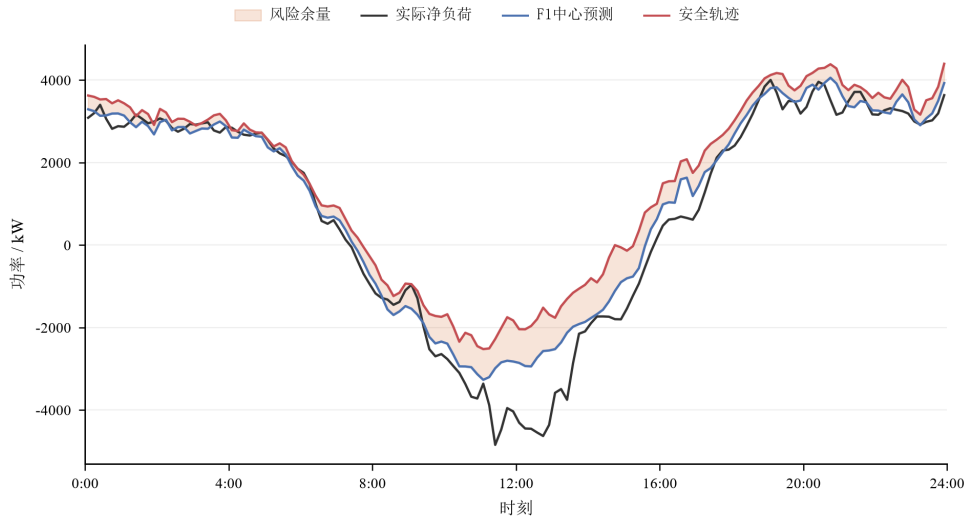


图 6: 分位风险修正预测、风险裕量与实际净负荷（以 2025-06-21 为例）

4.4 日前合同优化与 MPC 滚动执行规则

每天 0:00 以真实日初储电量为初值、以安全轨迹为输入求解日前合同：

$$\min \sum_{t=1}^{144} c_t p_t - \lambda (S_{145}^P - S_{\min}), \quad (16)$$

约束沿用前文通用微网物理模型，并令 $g_t = p_t$ 。式(16)第一项是当天必须支付的合同费用；第二项赋予有限时域末储电量机会价值，用于抑制时域截断引起的过度放空， λ 仅用于求解引导，不计入实际现金费用。问题二的实际现金费用为

$$C_2 = \sum_{t=1}^{144} (c_t p_t + 5c_t u_t). \quad (17)$$

计划购电量无论最终是否全部消纳都要按合同计费，弃购成本已包含在 $c_t p_t$ 中，不能再次计费； $5c_t u_t$ 只在实际供电缺口发生时产生。此处需要说明一个建模边界：题目未设并网功率上限，若在单条确定性安全轨迹上再加入可自由替代的应急变量，模型将倾向于用 5 倍高价电填补全部缺口，该写法不能称为风险联合优化；因此本文让风险通过安全轨迹进入合同决策，再由真实闭环结算评价 5 倍费用的影响。

合同签订后日内不可调整，储能动作由预测驱动的滚动时域线性规划（MPC 执行器，下文简称滚动 LP）给出。在时隙 t ，滚动 LP 以 $\mathcal{H}_t = \{t, \dots, \min(t + H - 1, 144)\}$ 为范围，取 $H = 49$ （约 8 小时 10 分钟），在 $I_{d,t}$ 给定的信息下联合安排充放电、光伏利用、弃购、应急购电与储电量：

$$\min \sum_{\tau \in \mathcal{H}_t} 5\hat{c}_\tau u_\tau + \varepsilon \sum_{\tau \in \mathcal{H}_t} (x_\tau + y_\tau) - \lambda S_{\text{end}}, \quad (18)$$

约束为通用能量平衡、储能递推与设备运行边界。合同费用在日内已成为沉没成本，滚动目标只需最小化预测的应急购电费用； ε 为极小罚项，用于在多重最优解中抑制储能同时充放电的数值伪解。每次求解只执行当前时隙动作，下一时隙用更新后的真实储电量重新计算。

设滚动 LP 给出的当前时隙动作记为 (x_t^M, y_t^M) 。本时隙真实负荷与光伏到达后，只允许在已决定动作的基础上缩减：

$$x_t^E = \min \{x_t^M, (g_t + V_t - L_t)^+\}, \quad (19)$$

$$y_t^E = \min \{y_t^M, (L_t - g_t - V_t + x_t^E)^+\}, \quad (20)$$

$$b_t = L_t + x_t^E - y_t^E - g_t - V_t. \quad (21)$$

若 $b_t \geq 0$ ，则 $u_t = b_t$ 、 $w_t = 0$ ；若 $b_t < 0$ ，则 $u_t = 0$ ，并将 $-b_t$ 依次解释为未利用合同电量与弃光电量。式(19)至式(21)保证应急电只补实际缺口、不用于主动充电，同时禁

止执行层依据本时隙实测缺口临时增加放电，使两种执行器在同一信息集下可比。时隙末真实储电量由储能递推关系传入下一时隙。

作为同信息集对照，本文另设预测驱动贪心执行器。令预测盈余

$$A_t = g_t + \tilde{V}_{d,t} - \tilde{L}_{d,t},$$

若 $A_t \geq 0$ ，则盈余用于充电：

$$x_t = \min \left\{ A_t, B_{\max}, \frac{S_{\max} - S_t^E}{\eta_c} \right\}, \quad y_t = u_t = 0;$$

若 $A_t < 0$ ，则先放电、缺口再应急：

$$y_t = \min \left\{ -A_t, B_{\max}, (S_t^E - S_{\min}) \eta_d \right\}, \quad u_t = -A_t - y_t, \quad x_t = w_t = 0.$$

贪心只依据当前一个时隙的安全轨迹决定动作，同样在动作确定之后才读取实测值并调用式(19)至式(21)的统一裁剪器。两种执行器共享同一合同、预测方法、风险裕度、储能约束与结算规则，唯一差别是滚动 LP 额外利用了未来 H 个时隙的价格与安全轨迹信息。贪心的作用在于提供唯一、可行、可审计的基线，不承诺跨时段执行最优性；年度正式结果以滚动 LP 为主执行器。

4.5 年度仿真结果与对比分析

334 天仿真结束后，年度关键指标如表3所示。

由表3可得，全年计划购电量 22382650.65 kWh，现金总费用 14786101.35 元，其中计划购电费用 13665855.74 元、应急购电费用 1120245.61 元。应急购电量仅占合同电量的 1.10%，但按 5 倍价格结算，对总费用的贡献达到 7.58%，说明风险修正的主要作用是把预测误差的尾部损失从高价应急购电转移到可控的合同冗余上。全年弃购 1596110.94 kWh、弃光 1741589.34 kWh，二者同时出现说明安全轨迹在部分时隙偏保守而另一些时隙光伏出力高于预期：前者表现为已付费但未消纳的合同电量，后者表现为可用而未利用的光伏电量。储能全年充电 7112133.95 kWh、放电 5760889.78 kWh、期末储电量 10731.91 kWh，全程满足 $1200 \leq S_t \leq 10800$ kWh。日费用的 95% 分位 VaR 为 66075.35 元、CVaR 为 70048.64 元，最坏单日为 2025-06-01 的 88726.19 元；按月统计，12 月费用最高 (1788158.05 元)，6 月应急购电量最高 (45846.13 kWh)，4 月费用最低 (1050872.88 元)，说明年度成本同时受负荷水平、光伏季节性、合同冗余与缺口分布的影响。

为检验主配置的稳健性，对预测器、残差窗口、分位水平与滚动时域逐一重跑全年闭环仿真，结果如表4所示。

表 3: 问题二年度运行汇总结果 (2-12 月)

指标	数值
计划购电量	22382650.65 kWh
计划购电费用	13665855.74 元
应急购电量	245483.29 kWh
应急购电费用	1120245.61 元
应急购电发生槽数	6822
闲置合同电量	1596110.94 kWh
弃光电量	1741589.34 kWh
储能充电量	7112133.95 kWh
储能放电量	5760889.78 kWh
期末储电量	10731.91 kWh
年度现金总费用	14786101.35 元

表 4: 问题二参数与执行器敏感性结果 (2-12 月)

方案	年度现金费用/元	应急购电量/kWh	相对主配置/元
主配置 F1-W35- α 0.805-H49	14786101.35	245483.29	0.00
分位 $\alpha = 0.800$	14820043.34	289390.70	+33941.99
分位 $\alpha = 0.810$	14786818.42	245477.85	+717.07
窗口 $W = 30$	14837364.77	247334.59	+51263.42
窗口 $W = 45$	14815783.49	260994.56	+29682.14
时域 $H = 48$	14786354.19	245523.92	+252.84
时域 $H = 50$	14786319.50	245518.43	+218.15
预测器 F2	14788842.58	249032.38	+2741.23
预测器 F0	17345539.63	403927.41	+2559438.28

由表4可得, 分位水平取 0.800 至 0.810、残差窗口取 30 至 45 日、滚动时域取 48 至 50 个时隙时, 年度现金费用相对主配置的变化均不超过 0.35%, 其中时域长度在 48 至 50 之间变化时费用差异不足 300 元, 表明主结论对参数设置并不敏感。分位由 0.805 降到理论值 0.800 时, 应急购电量由 245483.29 kWh 升至 289390.70 kWh、费用上升 33941.99 元, 说明理论分位在有限样本条件下略偏激进; 分位提高到 0.810 时费用基本不变, 说明继续增厚裕度的边际收益已趋近于零。预测器的选择影响远大于参数微调: 改用 F2 使费用上升 2741.23 元, 改用 F0 则上升 2559438.28 元 (约 17.3%), 与表2的点预测误差排序一致, 说明预测质量而非风险参数是本问题的主要误差来源。

为评价风险修正策略的相对水平，另设两组参照方案开展全周期对照：理想信息参照假设调度日 0:00 完全已知真实源荷，仅作为事后不可实施的理论下界；无储能基线在相同预测与结算口径下关闭储能。结果如表5所示。

表 5: 不同参照方案年度运行指标对比（2-12 月）

方案	年度现金费用/元	应急购电费用/元	应急购电量/kWh
主配置 F1-W35- α 0.805-H49	14786101.35	1120245.61	245483.29
理想信息参照（事后下界，不可实施）	12291024.77	0.00	0.00
无储能基线	16407319.63	0.00	0.00

理想信息参照与主配置的差额 2495076.58 元，度量了全年源荷预测不确定性带来的总经济代价。无储能基线为规避缺口必须按更保守的口径采购合同电量，全年不产生应急购电，但现金费用高出主配置 1621218.28 元（约 11.0%）。该对比说明储能的削峰填谷收益只有在合理预测与风险裕度配合下才能实现：储能本身并不能消除预测误差，它把误差从高价应急购电转移到合同冗余与储能吞吐两侧，而这一转移是否盈利取决于分位裕度设置是否合理。

四个代表日的运行结果如表6与图7所示。

表 6: 问题二四个代表日（春分、夏至、秋分、冬至）运行结果

日期	日购电费用/元	计划购电量/kWh	应急购电量/kWh	日初-日末储电量/kWh
2025-03-20	45158.05	70894.49	542.74	10800.00-10795.24
2025-06-21	24282.59	41689.27	9.36	10800.00-10800.00
2025-09-23	47616.38	72083.06	600.29	10760.83-10793.77
2025-12-21	66632.03	101642.57	234.75	10756.53-10800.00

代表日结果与光伏季节性一致：12 月 21 日负荷高、光伏弱，日费用最高（66632.03 元）；6 月 21 日光伏充足，日费用最低（24282.59 元）；3 月 20 日与 9 月 23 日的应急购电分别集中在凌晨与清晨的连续缺口时段，说明应急购电风险与日内源荷时序密切相关，而非由全天平均工况决定。上述代表日按题目要求整理的指定时段购电量、4 小时充放电与储电量、应急购电区间见附录。

最后需要说明风险裕度的工程折中关系：裕度设置过高会抬高合同采购成本并增加弃购，裕度设置过低则应急购电风险上升。仿真中可以观察到同一日既有弃购 $w_t > 0$ 、又有应急购电 $u_t > 0$ 的情形，这是日内源荷条件逐时隙变化的结果：部分时隙合同电量富余，另一些时隙的供给缺口无法由合同与储能覆盖，两类现象在同一日内可以并存。主配置在计入该折中关系后，于表4的参数网格内取得最低现金费用，年度结果稳定可

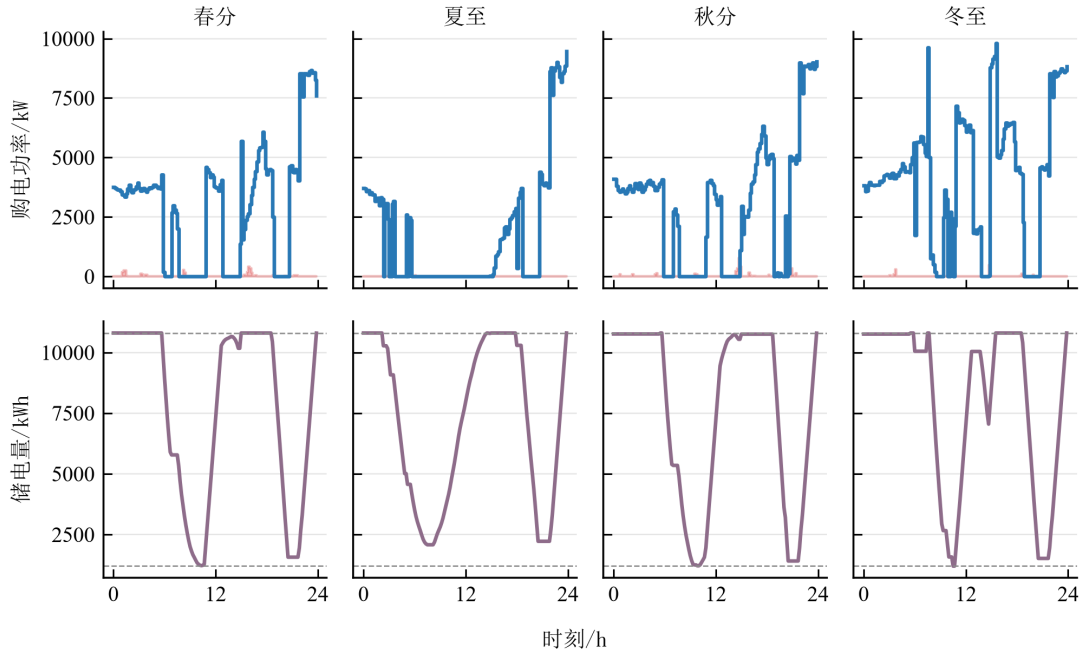


图 7: 春分、夏至、秋分、冬至四个代表日的购电功率与储电量

行。

五 问题三：考虑日内购电合同修正的微网滚动优化策略

5.1 问题分析：日内合同修正的经济价值

问题二的结果表明，在仅有日前固定合同、不允许日内调整的约束下，MPC 的跨时段优化增益有限，其收益大部分被源荷预测不确定性抵消。在此基础上，问题三引入日内购电合同修正机制。微网在日前市场依据光伏功率预测签订购电合同，得到计划购电量 p_t ；受光伏发电随机性与预测误差影响，日内光伏实际出力与日前预测存在显著偏差。若不允许对日前购电合同做任何修改，功率偏差只能依赖储能充放电平抑，储能触及 SOC 上下限后剩余偏差只能通过高价应急购电、闲置合同惩罚与弃光三类兜底手段承受，显著推高运行成本。

问题三要求在日内运行阶段引入合同修正机制：利用更新后的光伏预报，在合理约束内对尚未执行时段的购电合同适度修正，得到实际执行的最终购电合同 q_t ，以此削弱预测误差的不利影响。题目在每日 0:00、6:00、12:00、18:00 四个时点发布光伏预报，每个时段的最终购电量只相对 0:00 原计划结算一次，中间更新仅改变尚未执行时段的购电量。

为定量评估日内调账机制的经济价值，设置两组对照方案：主策略开启全部三个日

内预报时点的合同调整；基线方案 $S = \emptyset$ 不使用任何日内新预报，仍保留相同的滚动调度方法与结算口径。两组方案采用完全一致的负荷、光伏时序、储能参数、价格体系与预测输入，仿真区间为 2025-02-01 至 2025-12-31，共计 334 天。

5.2 决策时序与信息边界

一天内共有四个预报版本，新预测只能在规定发布时间获得。0:00 版本用于形成初始计划购电量 p_t ；6:00、12:00 和 18:00 时，新版本只影响尚未执行的时段，已执行时段不再调整。每次预报更新先读取上一时段结束后的实际储能 SOC，再调整剩余时段的购电量，最后进入下一时段的滚动调度。

模型严格遵循全文统一的信息边界约束。设第 d 日 t 时刻决策可用的信息集为

$$I_{d,t} = \{\text{已完成历史观测}, S_{d,t}, \text{当前生效合同}, \text{截至}t\text{已发布预测}\},$$

所有购电、储能、合同调整决策仅为 $I_{d,t}$ 的函数；当日未来真实负荷、光伏不参与决策计算。本时段真实源荷数据仅在决策完成之后，用于功率裁剪、执行修正与费用结算，杜绝未来信息泄露。

5.3 官方预报校正与分层风险

对目标日 d 、发布时刻 r 和提前期 h ，使用历史同版本误差对官方预报校正：

$$\hat{V}_{d,r,h}^{\text{cal}} = \max \left(0, \hat{V}_{d,r,h}^{\text{raw}} + \text{mean}_{j < d} e_{j,r,h}^V \right), \quad (22)$$

式中 $e_{j,r,h}^V$ 为历史第 j 日同发布时刻、同提前期的光伏预报残差。校正后插值到 10 分钟网格。不同提前期的预报误差存在差异，预测时域越长通常不确定性越大。若对每个提前期单独估计分位，会因样本不足引入较大统计噪声。本文按提前期将误差分为六层：0-1、1-2、2-3、3-4、4-5、5-6 小时，提前期超过 6 小时的远期时段统一归入第六层；0:00 版本覆盖的 6-24 小时部分及后续版本中仍超过 6 小时的远期时段均按第六层处理。令 s_h 为提前期 h 对应残差的经验均方根尺度，标准化残差 $z_{j,r,h} = e_{j,r,h}^V / s_h$ ，则风险裕量写为

$$R_{d,r,h} = s_h Q_\alpha \{z_{j,r,h} : d - W \leq j < d\}, \quad W = 35, \alpha = 0.77, \quad (23)$$

式中 Q_α 为经验分位函数。该方法保留“越远期误差通常越大”的结构，同时避免小样本下逐层估计的不稳定。校正后 1 小时、6 小时的误差 MAE 分别约为 19.38 kW 和 92.71 kW，其增长趋势为分层风险设置提供了直接数据依据。

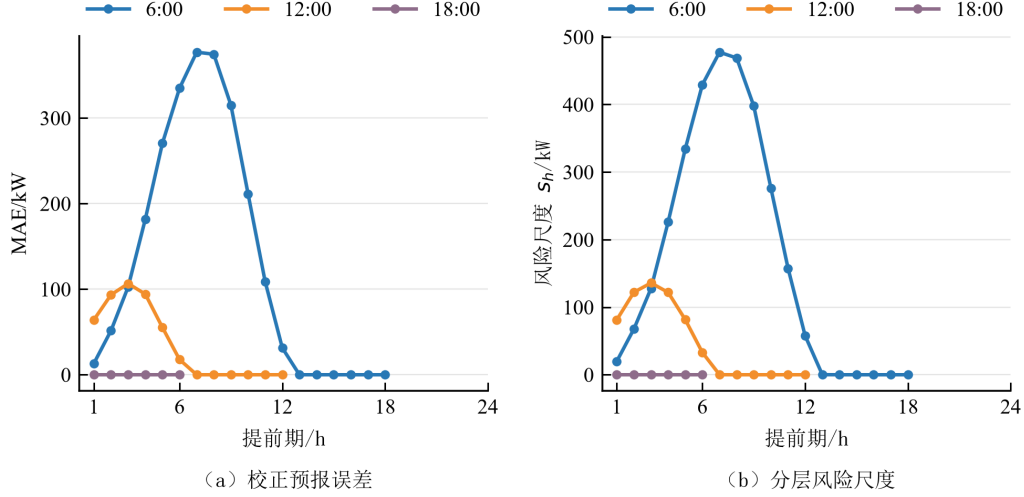


图 8: 问题三不同预报提前期的校正质量。

5.4 优化模型构建: MPC 滚动求解与合同调整规则

5.4.1 目标函数与结算规则

在 0:00 形成初始计划购电量 p_t , 6:00、12:00、18:00 保持已执行时段不变, 以当时实际 SOC 重新计算剩余时段购电量。令

$$q_t = p_t + \delta_t^+ - \delta_t^-, \quad \delta_t^+, \delta_t^- \geq 0, \quad (24)$$

式中 δ_t^+, δ_t^- 分别为上调、下调量。剩余时域优化目标为

$$\min \sum_{t \in \tau_r} [c_t p_t + 1.5c_t \delta_t^+ - 0.5c_t \delta_t^-] - \lambda_r (S_{145} - S_{\min}), \quad (25)$$

式中 c_t 为 t 时段购电单价, τ_r 为第 r 次更新时刻尚未执行的时段集合, λ_r 为终端储能价值罚项权重。最终调整购电量始终相对 0:00 计划购电量结算一次:

$$C_{3,t} = c_t \min(p_t, q_t) + 1.5c_t (q_t - p_t)^+ + 0.5c_t (p_t - q_t)^+ + 5c_t u_t, \quad (26)$$

式中 u_t 为应急购电量。该公式具有清楚的边际含义: 下调部分按 $0.5c_t$ 计费, 上调部分按 $1.5c_t$ 计费, 电量缺口执行 5 倍应急购电价格。每个时段的最终购电量只相对 0:00 原计划结算一次, 中间多次滚动优化的目标值不直接相加, 因此不会重复计费。

5.4.2 约束条件

1. 功率平衡约束

$$q_t - w_t + v_t + y_t + u_t = L_t + x_t, \quad (27)$$

式中各变量均为单个 10 分钟时段电量 (kWh); v_t 为光伏消纳量, w_t 为闲置合同电量, x_t, y_t 为充、放电量。

2. 储能系统运行约束

$$\begin{cases} S_{t+1} = S_t + 0.9x_t - \frac{y_t}{0.9} \\ S_{\min} \leq S_t \leq S_{\max} \\ 0 \leq x_t, y_t \leq 833.33 \end{cases} \quad (28)$$

式中 S_t 为储能 t 时刻储能量, x_t, y_t 分别为充、放电功率, $S_{\min}, S_{\max}$ 为 SOC 上下限。

3. 合同调整仅作用于尚未执行时段, 且 $q_t \geq 0$; 已执行时段合同量保持不变。

4. 非负约束

$$u_t \geq 0, \quad P_{\text{curt},t} \geq 0. \quad (29)$$

5.4.3 滚动求解配置

在相邻预报发布时点之间, 调度方法以 $H = 6$ 即 1 小时为滚动时域。每次 LP 同时考虑未来 6 个时段的价格、修正后的净负荷预测和 SOC 可行域, 只执行当前时段的调度决策, 随后利用新的实际信息进入下一次优化, 形成具有 MPC 基本形式的滚动执行。较短时域能够降低尚未更新的远期预测对当前调度决策的影响, 也与 6 小时一次的官方预报更新形成互补。由表7可得, $H = 1$ 时样本费用最低, 但 $H = 6$ 仅高 4273.26 元 (0.03%), 却能保留一个小时内连续时段的储能协同安排, 并与 6 小时一次的官方预报发布时间形成清晰的滚动周期。因此本文将 $H = 6$ 作为样本费用、多时段规划能力和求解速度之间的折中参数, 而非把它表述为费用最优。

表 7: 问题三滚动时域折中

H /时段	考虑的未来时长	年度费用/元
1	10 分钟	14660072.14
3	30 分钟	14662818.12
6	1 小时	14664345.40
12	2 小时	14671133.74

5.5 仿真结果与分析

5.5.1 主策略与基线方案对比

将两组方案按日聚合统计，得到 334 天仿真周期内的年度关键指标，如表8所示。主

表 8: 问题三主策略与基线方案年度指标对比

指标	主策略 (启用全部日内更新)	基线方案 $\mathcal{S} = \emptyset$	变化量
年度总费用/元	14664345.40	15230376.12	-566030.72
应急购电量/kWh	210761.48	334601.06	-123839.58

策略年度费用较基线下降 566030.72 元，降幅约 3.72%，应急购电量同时下降 37%。此外，0:00 计划购电量为 21702724.37 kWh，最终调整购电量为 21702916.77 kWh；累计上调量 513849.94 kWh、下调量 513657.53 kWh，两者基本相抵但发生时段不同，说明合同调整将风险在各时段之间重新分配，整日合同没有单向变化；弃购量为 1228166.90 kWh。

5.5.2 预报信息消融实验

为隔离日内预报信息本身的价值，在保持同一滚动调度方法、同一物理约束与同一结算口径的前提下，仅屏蔽不同时刻的官方预报信息，开展 8 组信息消融实验，结果如表9所示。由表9可得，不使用任何日内新预报的方案费用为 15230376.12 元，全部启用

表 9: 问题三不同日内预报组合的结果比较

预报组合 $\mathcal{S}$	年度费用/元	应急购电/kWh	节省费用/元
$\emptyset$	15230376.12	334601.06	0
{6}	15038617.00	266410.12	191759.11
{12}	14746909.96	224631.56	483466.16
{18}	14805047.54	228882.13	425328.58
{6, 12}	14679898.82	217968.28	550477.30
{6, 18}	14864586.68	218423.74	365789.44
{12, 18}	14710994.09	212056.29	519382.03
{6, 12, 18}	14664345.40	210761.48	566030.72

三个日内预报后降至 14664345.40 元，节省 56.60 万元 (3.72%)。6:00、12:00、18:00 更

新的平均边际收益分别约为 6.15、28.41、16.27 万元，表明预报信息价值存在明显时点差异，其中 12:00 时点临近负荷晚高峰，信息经济价值最高。只采用任一单一时点费用均低于不使用日内预报，说明在题目提供的预报范围内三个日内时点均有实际作用。

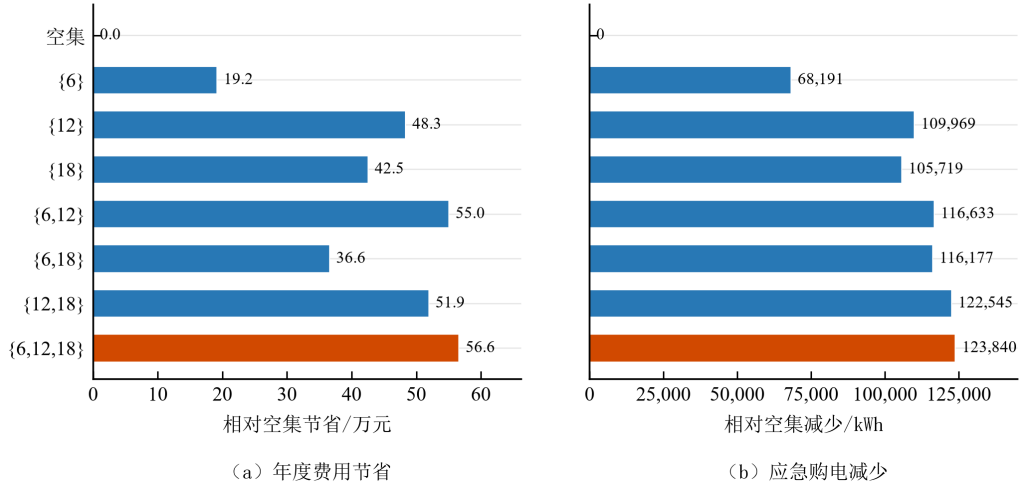


图 9: 不同日内预报组合的信息消融结果

5.5.3 盈亏平衡更新成本

将消融实验的费用差折算为单次合同更新的盈亏平衡成本：

$$\kappa^* = \frac{C_{\emptyset} - C_{\{6,12,18\}}}{\Delta n} = \frac{566030.72}{1002} \approx 564.90 \text{ 元/次}, \quad (30)$$

式中 $\Delta n = 1002$ 为全年合同更新次数。其经济含义为：若一次预报获取、传输、计算、合同重签与执行的综合成本低于 564.90 元，则全年启用全部日内预报在当前样本内具有正净收益。若只能选择一个时点，则 12:00 优先级最高。

5.5.4 其他预报时点的近似测试

题目仅在 0:00、6:00、12:00、18:00 提供独立官方预报；对其他时点的检验仅作为额外重优化机会的敏感性分析，不将其解释为新增独立预报的信息价值。在近似情景下，3:00 和 9:00 虽降低了应急购电量，但合同重算带来的现金费用上升，不满足引入判据；15:00 正收益天数 171 天，仅略超半数，可作为谨慎候选。21:00 虽然在 302 天产生正收益，但附件 2 全年 21:00 后的时段中实际光伏功率均为 0 kW，没有可供修正的日照信息，该收益实质上来自多一次基于当前 SOC 的合同重优化机会，而非 21:00 光伏预报的信息价值。专门设置 21:00 光伏预报更新缺乏物理意义，因此不予推荐。

表 10: 问题三其他预报时点的单次边际近似测试

候选时点	年度费用/元	年度节省/元	κ_t^* /元每次	正收益天数	结论
3:00	15540003.28	-875657.88	-2621.73	39/334	不引入
9:00	14958681.94	-294336.54	-881.25	71/334	不引入
15:00	14653971.85	10373.54	31.06	171/334	可引入
21:00	14613131.18	51214.22	153.34	302/334	不建议 (夜间)

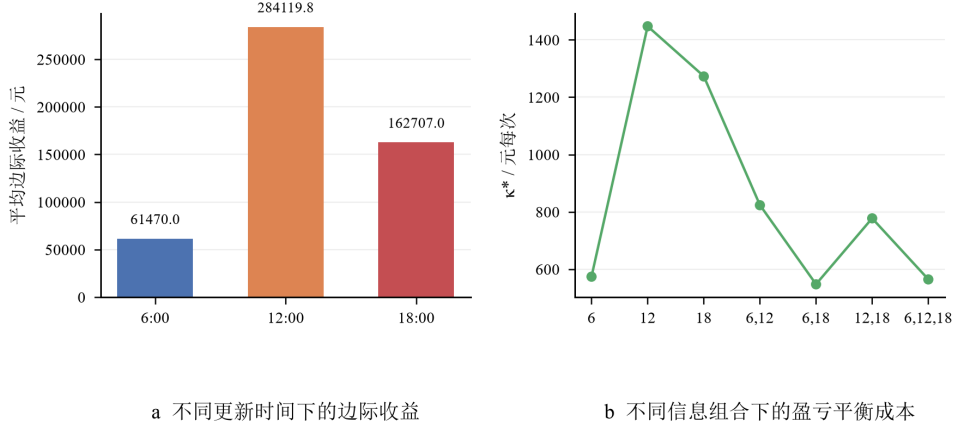


图 10: 预报更新时间的边际价值与物理可行性

5.5.5 代表日分析

选取春、夏、秋、冬 4 个典型代表日，绘制日前计划购电 p_t 与最终合同 q_t 的轨迹，标注 6:00、12:00、18:00 更新事件，如表11所示。不同季节光伏出力特征下，优化模型均在三个更新时刻根据新预报对购电合同进行向上或向下修正，匹配日内真实功率供需关系。

5.6 本章小结

本章建立支持日内购电合同修正的微网滚动 MPC 优化模型，设置不使用日内新预报的基线方案作为对照，完成 334 天全周期仿真。模型在每日 6:00、12:00、18:00 触发合同重优化，每次调整基于当前储能 SOC，最终购电量始终相对 0:00 原计划结算一次，杜绝重复计费；按提前期六层分层的风险裕量贯穿日前规划、合同调整与滚动 MPC 全流程。结果表明：

1. 日内合同调账策略能够利用更新的光伏预报修正购电合同，降低系统功率不平衡程度；相较基线方案，年度总费用下降 3.72%，应急购电量下降 37%；

表 11: 问题三四个代表日指定时段原计划与最终合同

日期	时段	原计划 p_t/kWh	最终合同 q_t/kWh
2025-03-20	10:00-10:10	0.00	0.00
2025-03-20	12:00-12:10	549.71	549.71
2025-03-20	16:00-16:10	652.59	605.82
2025-03-20	18:00-18:10	718.21	718.21
2025-06-21	16:00-16:10	203.53	163.73
2025-09-23	16:00-16:10	600.39	565.35
2025-12-21	12:00-12:10	979.30	945.70
2025-12-21	16:00-16:10	853.69	853.30
2025-12-21	18:00-18:10	750.11	750.11

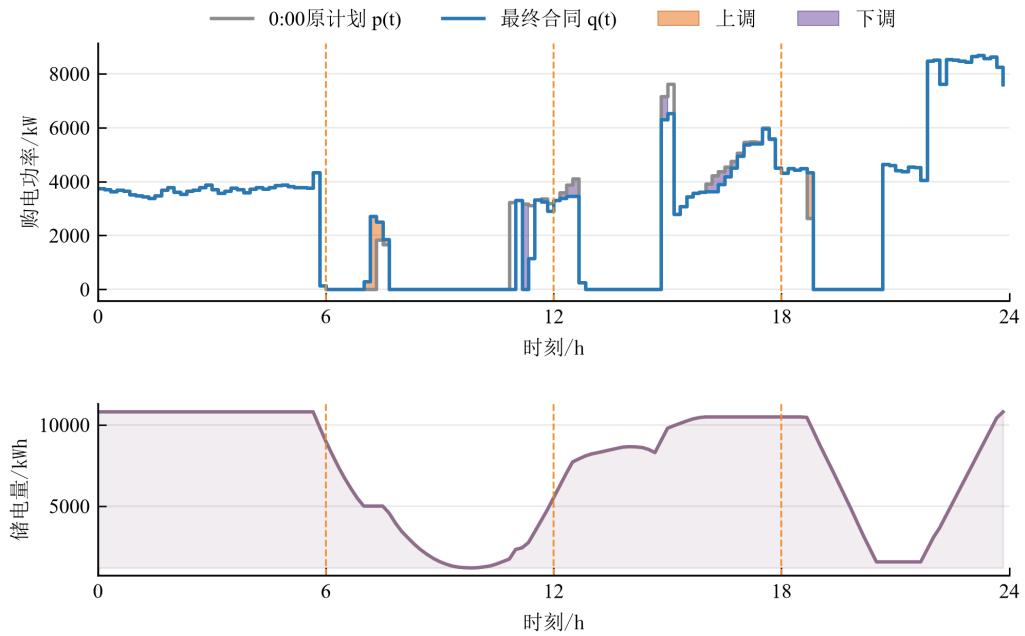


图 11: 问题三事件驱动的合同调整过程

2. 8 组预报信息消融实验表明，全部启用三个官方预报版本相对无日内预报节省 566030.72 元，12:00 时点预报边际价值最高；单次合同更新盈亏平衡成本约 564.90 元，量化了日内预报信息的经济价值；
3. 近似测试表明 15:00 可作为谨慎候选，3:00、9:00 因费用上升不引入，21:00 虽数值改善但缺乏物理意义不推荐；
4. 多季节代表日轨迹表明，调账策略在春、夏、秋、冬不同光伏工况下均可发挥调节效果。

六 问题四：考虑电价预测不确定性的微网多时间尺度调度优化

6.1 问题描述：波动电价场景下的购电策略

在问题三日内合同修正调度框架基础上，问题四将固定购电电价替换为分时波动市场电价。0 时刻无法预知全天真实电价，仅能依托历史时序数据构建电价预测；电价预测偏差会造成日前购电合同与储能调度方案偏离最优工况，带来应急购电、闲置合同等额外经济损失。本节对比两种合同模式：Q4-2 采用固定日前合同，0 时刻签约后全天不可修改；Q4-3 支持日内合同调整，在 6:00、12:00、18:00 基于更新信息修正剩余时段购电计划。两种方案均采用同一套预测基线，保证对比口径统一，用于量化日内合同调整机制在电价不确定性场景下的经济效益。所有决策严格遵守非前视约束：仅能使用决策时刻之前可观测数据，未来真实电价、源荷信息不参与优化求解，仅用于事后费用结算。

6.2 基于历史信息的价格预测

采用四种历史预测曲线构建 0 时刻电价预测基线：昨日同时段、近 7 日均值、近 30 日均值、近 30 日中位数。对每个目标时段，选取对应时段历史电价进行统计估计。表12汇总四种预测方案的 MAE、RMSE、尖峰 MAE 与最大绝对误差。

本文选取近 30 日中位数作为统一预测基线。该选择为 Q4-2、Q4-3 提供完全一致的价格输入，隔离合同调整机制本身带来的收益，同时在四种基准方法中综合误差相对较优。

设 0 时刻的近 30 日中位数价格基线为 $\hat{c}_{d,t}^{(0)}$ ；日内 k 时刻之前已完成时段的价格误差为 $e_{d,j}^c = c_{d,j} - \hat{c}_{d,j}^{(0)}$ 。利用已观测历史误差加权平均，随目标时段与当前时刻距离指数衰减，构造日内修正后的电价预测：

$$\hat{c}_{d,t|k} = \max \left\{ 10^{-4}, \hat{c}_{d,t}^{(0)} + \bar{e}_{d,k}^c \exp[-(t-k)/18] \right\}, \quad t \geq k. \quad (31)$$

表 12: 四种 0:00 基准价格预测方法误差

基准方法	MAE	RMSE	尖峰 MAE	最大绝对误差
昨日同时段	0.08	0.12	0.09	0.53
近 7 日均值	0.08	0.11	0.10	0.50
近 30 日均值	0.09	0.11	0.11	0.54
近 30 日中位数	0.08	0.12	0.10	0.62

单位: 元/kWh。

式中下限 10^{-4} 用于防止数值求解出现非负价格。该公式仅使用 k 时刻前已经观测到的价格数据，未来真实电价不参与决策，仅在结算阶段使用。

四种基准预测方法的误差排序与年度总费用排序并不完全一致。MAE 对所有时段等权重，但运行费用还受负荷、应急购电和储能状态影响。两种合同方案统一采用近 30 日中位数价格预测，并沿用问题二的负荷预测与官方光伏预报方案，以隔离合同调整机制的影响。

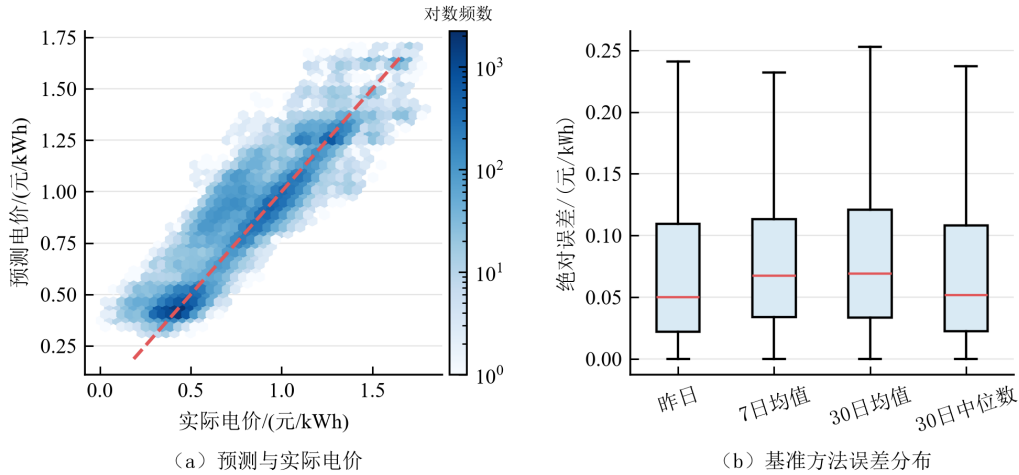


图 12: 波动电价预测的拟合与误差分布

6.3 调度模型与两种合同模式

Q4-2 与 Q4-3 的核心差异在于购电合同是否允许日内调整。

- **Q4-2 (固定合同模式):** 在 0 时刻一次性生成全天购电合同，合同一经确定全天保持不变；日内仅依靠储能充放电与应急购电平抑功率缺口，不支持合同修改。
- **Q4-3 (可调合同模式):** 0 时刻生成初始购电合同，在 6:00、12:00、18:00 三个时

点，利用更新的电价、光伏与负荷信息，对尚未执行的时段重新优化购电计划，已执行时段不可回溯修改。

两种模式在滚动 LP 优化中，均使用预测电价、预测负荷与光伏安排调度；合同调整沿用问题三的结算规则，仅相对 0 时刻原始计划进行一次结算，最终运行费用使用附件 4 的真实电价计算。

6.4 仿真结果与分析

6.4.1 固定合同与可调合同对比

两种模式全年仿真汇总结果如表13。

表 13: 问题四两种模式的年度结果（电量单位：kWh，费用单位：元）

模式	总费用/元	应急购电量/kWh	更新次数
Q4-2 固定合同	17378687.69	570732.66	0
Q4-3 可调合同	15465962.60	210394.02	1002

由表可见，Q4-3 相比 Q4-2 年度总费用减少 1912725.09 元，降幅 11.01%；应急购电量下降 360338.64 kWh，降幅 63.14%。结果表明，在分时电价波动场景下，日内合同调整的价值体现在两方面：一是直接削减功率缺口对应的高价应急购电，二是通过储能跨时段优化，合理安排各时段购电量，降低电价波动带来的成本。

6.4.2 不同价格基准与理想信息下界分析

为验证结论鲁棒性，在四种历史电价预测基线、以及 0 时刻预知全部真实电价的理想信息场景下，分别求解 Q4-2 与 Q4-3，结果如表14。

四种历史基准方法下，Q4-2 费用极差为 23619.64 元，Q4-3 费用极差为 31199.58 元，说明合同调整机制的费用优势在不同价格预测器下均能观察到。将全知理想电价场景作为下界，因无法提前获取真实电价带来的额外费用损失：Q4-2 损失 78602.31 元，Q4-3 损失 82912.80 元，仅占各自总费用的 0.45% 与 0.54%。这说明波动电价带来的成本变化，不能全部归因于电价预测误差，同时包含电价水平、尖峰位置、光伏与负荷缺口的耦合作用。

在四种基准方案下，Q4-3 费用均低于 Q4-2，节约幅度稳定在约 190 万元。对比不同预测方法之间的费用差异，合同更新机制带来的收益，远大于更换简单电价预测器带来的收益。

表 14: 问题四价格基准方法与理想信息结果

价格基准方法	Q4-2 费用/元	Q4-3 费用/元
昨日同时段	17402307.33	15485837.98
近 7 日均值	17390589.20	15454638.40
近 30 日均值	17400942.40	15455190.24
近 30 日中位数	17378687.69	15465962.60
0:00 已知实际电价	17300085.39	15383049.81

6.5 本章小结

本章针对分时波动电价场景，构建基于历史时序的电价预测方法，搭建固定合同、日内可调合同两类微网购电调度方案，开展全周期仿真对比。得到结论如下：

1. 在近 30 日中位数电价预测基线下，相比固定合同方案，支持日内调整的 Q4-3 年度总费用降低 11.01%，应急购电量下降 63.14%，日内合同调整可以显著对冲电价波动带来的经济损失；
2. 在多种电价预测基准下，可调合同策略均稳定优于固定合同，合同调整机制的收益不依赖特定的电价预测模型；
3. 与预知全部真实电价的理想场景对比，电价预测误差带来的额外损失占总成本比例很低，电价波动场景的运行成本，是电价时序、源荷缺口、储能调度、合同规则多因素耦合的结果。

本章从电价不确定性角度验证了日内合同调整机制的价值，与问题三光伏预测不确定性的结论形成完整互补，共同论证了多信息更新下滚动合同修正策略在微网经济调度中的有效性。

七 模型检验与灵敏度分析

7.1 数值与物理检验、规划—执行偏差诊断

为验证模型求解结果满足物理约束与数值可靠性，对 Q1–Q4 四个问题的仿真输出逐项校验。检验维度包含：母线功率平衡残差、蓄电池 SOC 时序递推校验、变量上下界约束、储能充放电互斥约束、储能跨日状态连续传递、全周期逐时段现金流水与汇总

核算一致性。同时开展**规划-执行偏差诊断**：对比 MPC 规划输出的储能状态与真实工况执行后的 SOC，计算一步预测误差。该指标用于佐证仿真流程的因果时序约束：若存在未来信息泄露，规划值将高度贴近真实执行值，一步误差会异常偏小。

对全量求解结果校验可得：全部方案的储能 SOC 时序递推、母线功率平衡残差均处于 10^{-13} 量级，属于求解器浮点计算误差范围；全部方案满足储能充放电互斥约束 $\min(x_t, y_t) = 0$ ，不存在同一时隙同时充放电的数值伪解。仿真全程储能 SOC 始终处于 $1200 \leq S_t \leq 10800$ kWh 的硬件边界之内。各方案规划-执行一步 SOC 误差如表15所示，一步 MAE 为 5-6 kWh 量级，RMSE 为 17-20 kWh 量级，存在合理的执行偏差，符合“决策时刻仅使用历史与已发布预测、真实值仅事后用于执行修正”的设定，未出现利用未来真值的异常现象。全年度逐时段现金流累加结果与年度汇总总费用的最大绝对偏差

表 15: 各方案规划-执行一步 SOC 偏差诊断

方案	一步 MAE/kWh	一步 RMSE/kWh	最大绝对误差/kWh
Q2 主策略	5.33	19.67	284.58
Q3 $\mathcal{S} = \{6, 12, 18\}$	5.06	17.63	271.98
Q4-2 固定合同	4.80	17.30	271.98
Q4-3 可调合同	5.16	18.07	271.98

为 0 元，费用分项核算完全一致，不存在口径冲突。综合以上校验，各问题调度输出满足全部物理约束，数值求解误差很小，规划与执行之间存在合理偏差，仿真结果具备可信性。

7.2 尾部风险与跨月稳定性检验

仅依靠年度平均总费用不足以完整评价调度策略的经济性能，进一步对日运行费用分布做尾部风险度量，采用 95% 分位 VaR、CVaR 以及最坏单日费用评估极端工况下的风险水平，结果如表16所示。对比 Q4-2 与 Q4-3 可看出，启用日内合同调整后， $CVaR_{0.95}$ 由 101705.29 元降至 83469.29 元，下降 18235.99 元；最坏单日费用由 134220.63 元降至 92033.53 元。合同调整机制不仅降低平均费用，也显著压缩极端高费用日风险，策略具备良好抗风险能力。

进一步开展跨月稳定性检验，考察各月份启用全部日内预报相对不使用新预报（Q3 空集）的费用改善幅度。2-12 月各月日均费用差均为负值，即每月均实现成本节约，收益覆盖全年所有月份，并非由个别异常月份单独驱动，策略改善效果具备时间维度的稳

表 16: 各方案尾部风险指标 (95% 置信水平)

方案	VaR _{0.95} /元	CVaR _{0.95} /元	最坏单日费用/元
Q2 主策略	66075.35	70048.64	88726.19
Q3 $\mathcal{S} = \{6, 12, 18\}$	65177.30	70474.34	77111.50
Q4-2 固定合同	87294.40	101705.29	134220.63
Q4-3 可调合同	76237.79	83469.29	92033.53

健性。

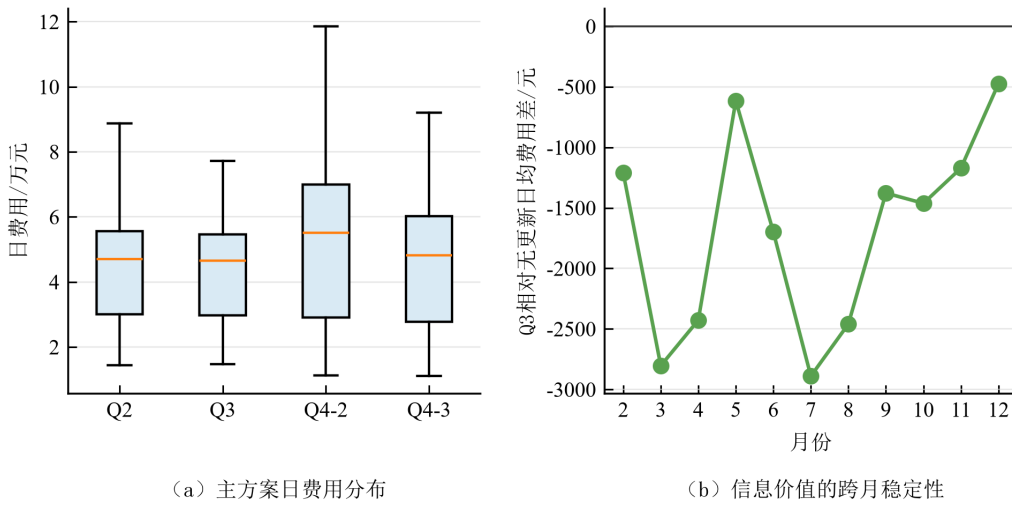


图 13: 固定合同与可调合同的尾部风险及跨月稳定性。

7.3 结构敏感性分析

为评估模型输出结果对关键参数的鲁棒性，开展多维度敏感性压力测试。**重要测试原则：**全部敏感性实验均在相同负荷序列、电价信息集、结算规则下完整重跑全周期仿真，不采用”直接替换参数代入原有调度轨迹”的简化估算，保证各组对比公平。测试变量包含：充放电效率、仿真初始 SOC、储能容量/功率上限、负荷预测误差倍率，结果如图14所示。

7.3.1 充放电效率敏感性

对比两组充放电效率配置 $(\eta_{ch}, \eta_{dis}) = (0.9, 0.9)$ 与 $(\sqrt{0.9}, \sqrt{0.9})$ 。效率取 $\sqrt{0.9}$ 时，年度总费用由基准的 14786101.35 元降至 14322473.62 元，下降 463627.73 元 (-3.14%)。

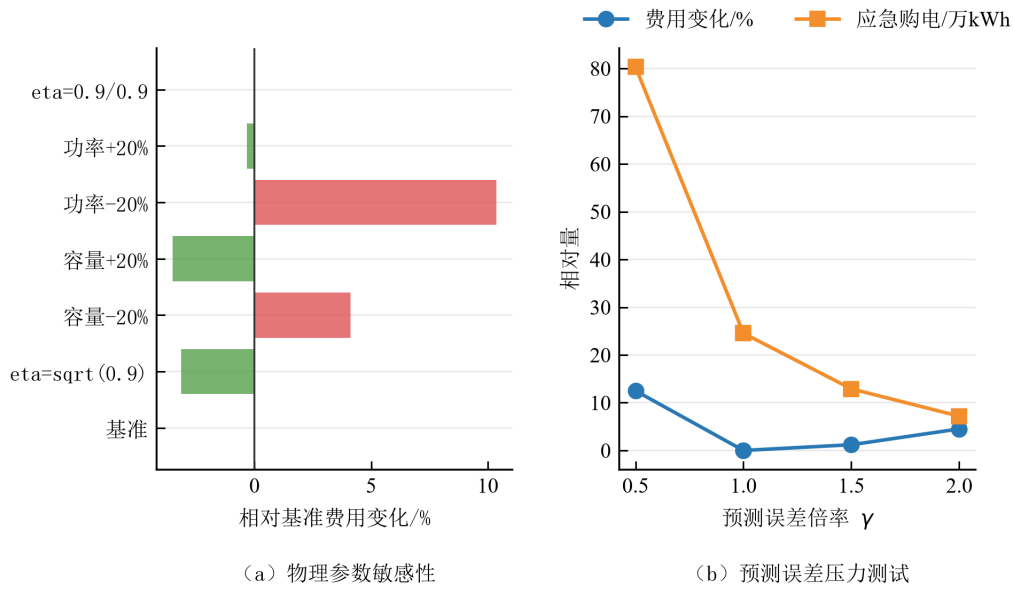


图 14: 多维度参数敏感性测试结果汇总

效率提高减少储能充放电过程损耗，系统运行费用随之下降，充放电效率属于中等敏感参数。

7.3.2 初始 SOC 敏感性

分别取储能初始 SOC 为额定容量的 20%、50%、80%。三种初始状态下年度总费用均为 14786101.35 元，应急购电量也完全一致，说明经过 1 月预热周期后，初始 SOC 差异对正式统计窗口的影响被完全消除，全年调度结果对初始荷电状态不敏感。

7.3.3 储能容量与功率上限敏感性

将储能容量、充放电功率上限分别取基准值的 80%、100%、120%。容量降低 20% 使总费用升至 15392912.44 元 (+4.10%)，容量增加 20% 降至 14267282.70 元 (-3.51%)。相比之下，**功率上限降低 20%** 使总费用大幅升至 16318749.90 元 (+10.37%)，应急购电量由 245483.29 增至 563286.32 kWh；功率增加 20% 仅带来 -0.33% 的微幅改善。储能功率上限属于高敏感参数，功率不足会显著限制储能移峰能力，推高应急购电成本；容量参数敏感度相对更低。

7.3.4 负荷预测误差倍率压力测试

设置负荷预测误差放大倍率 $\gamma \in \{0.5, 1.0, 1.5, 2.0\}$ 。随误差倍率增大，年度总费用呈现先升后波动： $\gamma = 0.5$ 时费用为 16632993.38 元 (+12.49%)， $\gamma = 1.0$ 为基准值， $\gamma = 1.5$

时 14963927.62 元 (+1.20%)， $\gamma = 2.0$ 时 15453352.16 元 (+4.51%)。预测误差的放大或缩小都会改变风险裕度的匹配度，费用在基准倍率附近取得较低水平。在 $\gamma \leq 2.0$ 范围内，本滚动风险修正框架仍能维持良好的经济性能，未出现求解失效。

综合全部敏感性实验结果：

1. 储能充放电功率上限、负荷预测误差倍率属于高敏感参数；充放电效率为中等敏感参数；全年仿真结果对初始 SOC 敏感度最低，对储能额定容量敏感度相对较弱。
2. 1 月预热机制能有效消除仿真初始 SOC 带来的短期扰动，长期年度结论不受初始储能状态干扰。
3. 在合理预测扰动范围内，整套滚动风险修正与合同调整框架可稳定发挥能量转移与风险对冲作用，同时上述结果明确了模型策略的适用边界。

八 模型评价与推广

8.1 模型特点

本文针对赛题给定的信息时序约束、风险费用结构与结算规则，完成如下建模处理：

1. 四个子问题采用统一物理约束与决策可用信息集 $\mathcal{I}_{d,t}$ ，仅根据题目要求修改购电量调整权限与电价时序；实测真值在当前时段决策生成之后，方可用于功率修正与事后费用结算。
2. 5 倍应急购电溢价使得风险裕量具备经济必要性；计划购电与应急购电之间的最优权衡由优化命题给出，理论分位取 0.8，实际依托历史净负荷误差的经验分位数实现，并基于该参数完成全年逐日仿真。通过逐时段供需平衡、SOC 递推和费用分项汇总进行独立核验，各时段最终购电方案仅相对 0:00 原始计划执行一次结算。
3. 设计八组预报信息组合开展费用对比，借助边际收益与 κ^* 衡量各预报时刻对应的经济价值；设置理想电价信息基准场景，用于量化电价未知带来的额外购电成本。
4. 问题三、四采用同一套滚动调度框架，对比不同预报方案与合同调整策略；通过预报组合对照，定量刻画不同预报时刻下总费用、应急购电量的变化规律。
5. 问题三总费用为 14664345.40 元；问题四固定合同与可调合同的总费用分别为 17378687.69 元和 15465962.60 元。

6. 问题三按预报提前期划分六层风险裕量，保留了“提前期越长、误差通常越大”的时序结构；贪心与 MPC 在共同信息集下对照，结果显示理论上更强的跨时段优化未必转化为显著的实际收益。
7. 对其他时刻的检验表明，21:00 时段光伏实际值恒为零，相关收益主要来自额外重优化机会而非新增光伏信息，说明预报价值必须结合物理意义审慎解释。
8. 同时报告费用、应急购电量和尾部风险，避免只用单一平均费用评价策略；统一物理模型、非前视信息集、合同调账规则和分层风险裕度，形成从预测、计划、执行到结算的闭环框架。

8.2 局限与推广

题目未给出并网容量上限以及售电相关机制，本文假设外购电无上限，且微网不可向主网反向送电。若工程场景存在并网容量约束，则需增加约束 $g_t + u_t \leq G_{\max}$ 。模型采用经验分位数刻画历史残差尾部特征，未刻画连续多时段预测误差间的相关性，因此逐时段覆盖率不能等价为整日可靠概率。仿真采用 10 分钟聚合时间粒度，忽略时隙内部测量延迟与功率爬坡过程；实际工程应用时，可缩短仿真步长或增加延迟状态变量予以修正。本文全部结论基于 2025 年历史数据得到，更换其他年份数据集时，需要重新校验模型参数与费用表现。

按预报提前时间分组设置风险裕量时，采用同一种风险裕量的全局方案，在样本上的总费用相比分组方案低 8860.56 元。本文仍然选用分组方案，原因在于该方案保留了“预报提前期越长，预测误差通常越大”的数据固有特征。问题三中 $H = 1$ 对应的样本费用略低于 $H = 6$ ，说明滚动时域长度需要在样本经济性与多时段规划能力之间做折中取舍。

模型可扩展到含风电、可控柔性负荷与多组储能的园区微网场景。新增分布式风电资源，只需在供需平衡方程内增加对应出力供给变量；引入需求响应资源，则增加负荷平移变量，并配套用户用电舒适度约束。若放开售电权限或存在负电价场景，应当采用带有显式充放电互斥约束的混合整数规划模型，规避充放电循环套利问题。微网分层控制的一般性理论背景可参见参考文献。

若能够获取多年实测数据，可将当前 35 日滑动分位替换为季节条件分位或场景树方法，并采用独立年份样本完成参数 W, α, H 的优选；如果预报供应商按次收取预报费用，可将 κ 纳入预报获取方案优化，把盈亏平衡分析拓展为 0-1 整数选择模型；当园区配备多块储能电池时，可以对每一块电池单独建立 SOC 状态方程，额外增加共享并网

容量约束与电池寿命折损成本。上述所有拓展场景，均可沿用本文的购电计划生成、日内实际调度以及费用结算完整流程。

九 结论

本文在统一储能与供需平衡模型下完成四个子问题的仿真。问题一的典型日购电量为 59482.70 kWh，费用为 35126.95 元。问题二采用 F1 预测与风险裕量，334 天总费用为 14786101.35 元，应急购电量为 245483.29 kWh。问题三在 0:00、6:00、12:00 和 18:00 更新预报并调整合同，总费用为 14664345.40 元，相比不使用日内新预报的方案节省 566030.72 元，应急购电量为 210761.48 kWh。问题四中，固定合同和可调合同的总费用分别为 17378687.69 元和 15465962.60 元，可调合同节省 1912725.09 元，并使应急购电量下降 63.14%。

上述结论仅适用于不可售电、无并网功率上限、充放电效率均为 0.9、10 分钟离散以及 2025 年历史样本的设定。数值检验显示供需平衡和 SOC 递推误差处于求解器浮点误差范围；敏感性分析表明，储能功率上限和预测误差倍率是影响费用的主要因素。模型还可扩展到含风电、需求响应和多储能的园区微网，但扩展后需要重新核验参数和费用口径。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于建模方案比较、参数分析、代码调试、图示设计和语言润色等，详细使用情况见支撑材料。

参考文献

- [1] 姜启源, 谢金星, 叶俊. 数学模型 (第五版)[M]. 北京: 高等教育出版社, 2018.
- [2] 李航. 统计学习方法 [M]. 北京: 清华大学出版社, 2019.
- [3] Ropke S, Pisinger D. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows[J]. Transportation Science, 2006, 40(4): 455-472.
- [4] Bouzid A M, Guerrero J M, Chériti A, et al. A survey on control of electric power distributed generation systems for microgrid applications[J]. Renewable and Sustainable Energy Reviews, 2015, 44: 751-766.
- [5] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python[J]. Nature Methods, 2020, 17(3): 261-272.

A 题目要求表格与指定日期结果

题目要求在论文中呈现的结果表格统一列于本附录；正文仅保留模型、分析和关键结论。

1.1 问题一：表 1 与表 2

表 17: 问题一指定时间段购电量及全天汇总

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	486.40	14:00–14:10	0.00
16:00–16:10	394.93	18:00–18:10	636.98	20:00–20:10	0.00
全天购电量	59482.70			全天购电费/元	35126.95

表 18: 问题一分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	4500.00	0.00	4:00–8:00	833.33	6365.84
8:00–12:00	4787.96	1703.00	12:00–16:00	5286.04	91.10
16:00–20:00	0.00	5780.13	20:00–24:00	5333.33	2859.87
0:00 储电量	6000.00 kWh		24:00 储电量	6000.00 kWh	

1.2 问题 2：表 1、表 2 与表 3

表 19: 问题 2：2025-03-20 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	479.31	14:00–14:10	0.00
16:00–16:10	562.26	18:00–18:10	698.00	20:00–20:10	5.40

表 20: 问题 2：2025-03-20 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	0.00	0.00	4:00–8:00	0.00	5499.86
8:00–12:00	5900.91	3122.72	12:00–16:00	5283.31	436.64
16:00–20:00	0.00	5388.59	20:00–24:00	10252.91	2920.55
日初储电量	10800.00		日末储电量	10795.24	

表 21: 问题 2: 2025-06-21 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	78.57	18:00-18:10	0.00	20:00-20:10	0.00

表 22: 问题 2: 2025-06-21 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	2482.63	4:00-8:00	0.00	5366.66
8:00-12:00	6224.02	0.00	12:00-16:00	3482.98	13.38
16:00-20:00	0.00	5234.59	20:00-24:00	9531.47	2485.90
日初储电量	10800.00		日末储电量	10800.00	

表 23: 问题 2: 2025-09-23 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	3.39	12:00-12:10	354.19	14:00-14:10	0.00
16:00-16:10	589.43	18:00-18:10	764.33	20:00-20:10	11.74

表 24: 问题 2: 2025-09-23 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	43.52	5804.35
8:00-12:00	5817.31	2823.94	12:00-16:00	5004.29	178.42
16:00-20:00	0.00	5602.91	20:00-24:00	10426.13	2806.64
日初储电量	10760.83		日末储电量	10793.77	

表 25: 问题 2: 2025-12-21 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	932.83	14:00-14:10	0.00
16:00-16:10	801.38	18:00-18:10	715.41	20:00-20:10	0.00

表 26: 问题 2: 2025-12-21 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	881.63	1508.33
8:00-12:00	5655.40	7806.67	12:00-16:00	8333.33	2690.88
16:00-20:00	0.00	5512.67	20:00-24:00	10315.06	2842.53
日初储电量	10756.53		日末储电量	10800.00	

表 27: 问题 2: 指定日期紧急购电量

日期	紧急购电时间段	购电量/kWh
2025-03-20	0:20-0:30	1.94
2025-03-20	1:00-1:40	116.54
2025-03-20	3:10-3:30	36.11
2025-06-21	1:20-1:30	1.70
2025-06-21	1:50-2:00	1.24
2025-06-21	4:00-4:10	0.61
2025-09-23	0:40-0:50	13.55
2025-09-23	2:10-2:20	14.23
2025-09-23	4:50-5:10	38.37
2025-12-21	3:00-3:10	7.54
2025-12-21	3:30-3:50	65.05
2025-12-21	10:30-10:50	28.95

1.3 问题 3: 表 1、表 2 与表 3

表 28: 问题 3: 2025-03-20 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	549.71	14:00-14:10	0.00
16:00-16:10	605.82	18:00-18:10	718.21	20:00-20:10	0.00

表 29: 问题 3: 2025-03-20 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	0.00	6154.48
8:00-12:00	3970.58	2485.52	12:00-16:00	6708.75	328.09
16:00-20:00	43.56	5094.62	20:00-24:00	10253.90	2928.49
日初储电量	10800.00		日末储电量	10800.00	

表 30: 问题 3: 2025-06-21 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	163.73	18:00-18:10	0.00	20:00-20:10	0.00

表 31: 问题 3: 2025-06-21 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	2944.22	4:00-8:00	191.07	5150.11
8:00-12:00	8517.34	0.00	12:00-16:00	1293.95	7.58
16:00-20:00	0.00	5541.12	20:00-24:00	9909.90	2485.90
日初储电量	10800.00		日末储电量	10800.00	

表 32: 问题 3: 2025-09-23 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	362.95	14:00-14:10	0.00
16:00-16:10	565.35	18:00-18:10	783.44	20:00-20:10	0.00

表 33: 问题 3: 2025-09-23 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	0.00	6591.76
8:00-12:00	5624.09	2048.24	12:00-16:00	5445.49	388.79
16:00-20:00	0.00	5602.91	20:00-24:00	10391.99	2752.18
日初储电量	10800.00		日末储电量	10800.00	

表 34: 问题 3: 2025-12-21 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	945.70	14:00-14:10	0.00
16:00-16:10	858.30	18:00-18:10	750.11	20:00-20:10	0.00

表 35: 问题 3: 2025-12-21 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	833.33	1508.33
8:00-12:00	5218.33	7703.98	12:00-16:00	7534.09	1792.14
16:00-20:00	0.00	5434.51	20:00-24:00	10218.56	2842.53
日初储电量	10800.00		日末储电量	10800.00	

表 36: 问题 3: 指定日期紧急购电量

日期	紧急购电时间段	购电量/kWh
2025-03-20	0:20-0:30	7.34
2025-03-20	1:00-1:40	99.45
2025-03-20	3:10-3:30	36.04
2025-06-21	0:20-0:30	5.36
2025-06-21	1:20-1:30	11.72
2025-06-21	1:50-2:00	12.69
2025-09-23	0:40-0:50	14.46
2025-09-23	2:10-2:20	14.35
2025-09-23	4:50-5:10	21.85
2025-12-21	1:30-1:40	7.73
2025-12-21	3:00-3:10	27.03
2025-12-21	3:30-3:50	104.82

B 完整程序代码

本附录嵌入支撑材料中用于结果复现的全部 Python 源代码。

Listing 1: run_lite.py

```
from __future__ import annotations

import argparse
import subprocess
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent
SCRIPT_DIR = Path(__file__).resolve().parent

def commands() -> list[list[str]]:
    py = sys.executable
    return [
        [py, str(SCRIPT_DIR / "问题1_求解.py"), "--no-figures"],
        [py, str(SCRIPT_DIR / "问题2_求解.py"), "--main-only", "--days",
         "365", "--executor", "mpc"],
        [py, str(SCRIPT_DIR / "问题3_求解.py"), "--days", "365", "--",
         "workers", "1", "--main-only"],
        [py, str(SCRIPT_DIR / "问题4_求解.py"), "--days", "365", "--",
         "workers", "2", "--main-only"],
    ]

def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--list", action="store_true", help="只列出复现步骤，不执行")
    args = ap.parse_args()
    jobs = commands()
    if args.list:
        for job in jobs:
            print(subprocess.list2cmdline(job))
        return 0
    started = time.perf_counter()
    for i, job in enumerate(jobs, 1):
        print(f"[Lite {i}/{len(jobs)}] {subprocess.list2cmdline(job)}",
              flush=True)
        subprocess.run(job, cwd=ROOT, check=True)
    print(f"[Lite] all done, elapsed_s={time.perf_counter() - started:.1f}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

```
import pandas as pd
from scipy.optimize import linprog

ROOT = Path(__file__).resolve().parent
DATA_DIR = ROOT / "data" / "附件"
RESULT_DIR = ROOT / "results"
FIGURE_DIR = ROOT / "figures"

T = 144
DT = 1.0 / 6.0
ETA_C = 0.9
ETA_D = 0.9
E_MAX = 12000.0
S_MIN = 1200.0
S_MAX = 10800.0
P_MAX = 5000.0
B_MAX = P_MAX * DT
SO = 6000.0
EMERGENCY_FACTOR = 5.0
OVER_FACTOR = 1.5
UNDER_FACTOR = 0.5

DAY0 = _dt.date(2025, 1, 1)
START_DATE = _dt.date(2025, 2, 1)
END_DATE = _dt.date(2025, 12, 31)
N_OFFICIAL = (END_DATE - START_DATE).days + 1

EPS_THROUGHPUT = 1e-4

_TIME_COLS = None

def _read_power_matrix(path: Path, sheet) -> np.ndarray:
    df = pd.read_excel(path, sheet_name=sheet, header=0)
    if df.shape[1] != 145:
        raise ValueError(f"{path.name}/{sheet} 列数异常: {df.shape[1]}")
    vals = df.iloc[:, 1:].to_numpy(dtype=float)
    if vals.shape != (365, T):
        raise ValueError(f"{path.name}/{sheet} 形状异常: {vals.shape}")
    if not np.isfinite(vals).all():
        raise ValueError(f"{path.name}/{sheet} 含缺失值")
    return vals

def load_inputs() -> dict:
    global _TIME_COLS
    a1 = pd.read_excel(DATA_DIR / "附件1.xlsx", sheet_name=0, header=0)
    a1.columns = [str(c).strip() for c in a1.columns]
    price1 = a1.iloc[:, 1].to_numpy(dtype=float)
    load1 = a1.iloc[:, 2].to_numpy(dtype=float)
```

Listing 2: microgrid_core.py

```
from __future__ import annotations

import datetime as _dt
from pathlib import Path

import numpy as np
```

```

pv1 = a1.iloc[:, 3].to_numpy(dtype=float)
if not (len(pricel) == len(load1) == len(pv1) == T):
    raise ValueError("附件1 长度异常")

load_act = _read_power_matrix(DATA_DIR / "附件2.xlsx", "小区负载")
pv_act = _read_power_matrix(DATA_DIR / "附件2.xlsx", "光伏发电实际功率")
price_act = _read_power_matrix(DATA_DIR / "附件4.xlsx", 0)

a3 = pd.read_excel(DATA_DIR / "附件3.xlsx", sheet_name=0, header=0)
if a3.shape != (1460, 26):
    raise ValueError(f"附件3 形状异常: {a3.shape}")
raw_date = a3.iloc[:, 0].ffill()
dates3 = pd.to_datetime(raw_date, format="%Y-%m-%d", errors="coerce")
if dates3.isna().any():

    parsed = []
    cur = None
    for v in a3.iloc[:, 0]:
        if isinstance(v, str) and v.strip():
            cur = pd.Timestamp(_dt.datetime.strptime(v.strip(), "%Y-%m-%d").date())
            parsed.append(cur)
    dates3 = pd.Series(parsed)
issue = a3.iloc[:, 1].astype(str).str.strip().to_numpy()
fc_raw = a3.iloc[:, 2:26].to_numpy(dtype=float)

issue_hours = np.array([int(s.split(":")[0]) for s in issue])

day_idx = np.array([(d.date() - DAY0).days for d in dates3])
slot_idx = issue_hours // 6
fc = np.full((365, 4, 24), np.nan)
for k in range(len(day_idx)):
    fc[day_idx[k], slot_idx[k]] = fc_raw[k]
if not np.isfinite(fc).all():
    raise ValueError("附件3 预报张量存在空洞")

_TIME_COLS = [str(c) for c in pd.read_excel(DATA_DIR / "附件2.xlsx", sheet_name="小区负载", header=0).columns[1:]]

return {
    "pricel": pricel,
    "load1": load1,
    "pv1": pv1,
    "load_act": load_act,
    "pv_act": pv_act,
    "price_act": price_act,
    "fc_raw": fc,
}

def date_of(day_index: int) -> _dt.date:
    return DAY0 + _dt.timedelta(days=int(day_index))

_DAY_LP_CACHE: dict = {}

def _day_lp_structure(n: int):
    if n in _DAY_LP_CACHE:

        return _DAY_LP_CACHE[n]
    N = 5 * n
    A_eq = np.zeros((2 * n, N))
    b_eq = np.zeros(2 * n)
    for t in range(n):

        A_eq[t, 2 * n + t] = 1.0
        A_eq[t, 3 * n + t] = 1.0
        A_eq[t, n + t] = 1.0
        A_eq[t, t] = -1.0

        r = n + t
        A_eq[r, 4 * n + t] = 1.0
        if t > 0:
            A_eq[r, 4 * n + t - 1] = -1.0
            A_eq[r, t] = -ETA_C
            A_eq[r, n + t] = 1.0 / ETA_D
        _DAY_LP_CACHE[n] = (A_eq, b_eq)
    return A_eq, b_eq

def solve_day_lp(c, load_kwh, pv_kwh, s_init, s_end=None, lam=None, charge_cap=None, extra_throughput_penalty=EPS_THROUGHPUT):
    c = np.asarray(c, dtype=float)
    n = len(c)
    A_eq, _ = _day_lp_structure(n)
    N = 5 * n

    b_eq = np.empty(2 * n)
    b_eq[:n] = load_kwh
    b_eq[n] = s_init
    b_eq[n + 1:] = 0.0

    obj = np.zeros(N)
    obj[2 * n:3 * n] = c
    obj[:n] += extra_throughput_penalty
    obj[n:2 * n] += extra_throughput_penalty
    if lam is not None:
        obj[4 * n + n - 1] = -lam

    lb = np.zeros(N)
    ub = np.full(N, np.inf)
    ub[:n] = B_MAX
    ub[n:2 * n] = B_MAX
    ub[2 * n:3 * n] = np.inf
    ub[3 * n:4 * n] = pv_kwh
    lb[4 * n:5 * n] = S_MIN
    ub[4 * n:5 * n] = S_MAX
    if charge_cap is not None:
        ub[:n] = np.minimum(ub[:n], charge_cap)
    if s_end is not None:
        lb[4 * n + n - 1] = s_end
        ub[4 * n + n - 1] = s_end

    bounds = list(zip(lb, ub))
    res = linprog(obj, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError(f"日前 LP 求解失败: {res.message}")
    z = res.x
    return {
        "x": z[:n], "y": z[n:2 * n], "g": z[2 * n:3 * n],
        "v": z[3 * n:4 * n], "S": z[4 * n:5 * n],
        "curtail": pv_kwh - z[3 * n:4 * n],
        "objective": float(res.fun), "status": res.status,
    }

```

```

}

def causal_dispatch(p, load_kwh, pv_kwh, s_init):
    n = len(p)
    x = np.zeros(n); y = np.zeros(n); u = np.zeros(n)
    w = np.zeros(n); v = np.zeros(n); S = np.zeros(n)
    s = float(s_init)
    for t in range(n):
        A = p[t] + pv_kwh[t] - load_kwh[t]
        if A >= 0.0:
            xe = min(A, B_MAX, (S_MAX - s) / ETA_C)
            xe = max(xe, 0.0)
            ye = 0.0
            ue = 0.0
            r = A - xe
            we = min(p[t], r)
            ve = pv_kwh[t] - max(0.0, r - we)
        else:
            xe = 0.0
            ye = min(-A, B_MAX, (s - S_MIN) * ETA_D)
            ue = -A - ye
            we = 0.0
            ve = pv_kwh[t]
        x[t], y[t], u[t], w[t], v[t] = xe, ye, ue, we, ve
        s = s + ETA_C * xe - ye / ETA_D
        S[t] = s
    return {"x": x, "y": y, "u": u, "w": w, "v": v, "S": S}

_ROLL_LP_CACHE: dict = {}

def _rolling_lp_structure(n: int):
    cached = _ROLL_LP_CACHE.get(n)
    if cached is not None:
        return cached

    N = 5 * n + (n + 1)
    o = {k: i * n for i, k in enumerate(("x", "y", "v", "u", "w"))}
    s0 = 5 * n
    A_eq = np.zeros((2 * n, N))
    for t in range(n):
        A_eq[t, o["x"] + t] = -1.0
        A_eq[t, o["y"] + t] = 1.0
        A_eq[t, o["v"] + t] = 1.0
        A_eq[t, o["u"] + t] = 1.0
        A_eq[t, o["w"] + t] = -1.0
        A_eq[n + t, o["x"] + t] = -ETA_C
        A_eq[n + t, o["y"] + t] = 1.0 / ETA_D
        A_eq[n + t, s0 + t] = -1.0
        A_eq[n + t, s0 + t + 1] = 1.0
    result = (A_eq, o, s0, N)
    _ROLL_LP_CACHE[n] = result
    return result

def rolling_lp_action(s_init, g_seg, load_seg, pv_seg, c_seg, lam,
                    time_limit=None):
    n = len(g_seg)

    A_eq, o, s0, N = _rolling_lp_structure(n)
    b_eq = np.concatenate([load_seg - g_seg, np.zeros(n)])

    obj = np.zeros(N)
    obj[o["u"] + n] = EMERGENCY_FACTOR * c_seg
    obj[o["x"] + n] += EPS_THROUGHPUT
    obj[o["y"] + n] += EPS_THROUGHPUT
    obj[s0 + n] = -lam

    surplus = np.maximum(g_seg + pv_seg - load_seg, 0.0)
    deficit = np.maximum(load_seg - g_seg - pv_seg, 0.0)

    lb = np.zeros(N); ub = np.full(N, np.inf)
    ub[o["x"] + n] = np.minimum(B_MAX, surplus)
    ub[o["y"] + n] = B_MAX
    ub[o["v"] + n] = pv_seg
    ub[o["u"] + n] = deficit
    ub[o["w"] + n] = g_seg
    lb[s0] = ub[s0] = s_init
    lb[s0 + 1:s0 + n + 1] = S_MIN
    ub[s0 + 1:s0 + n + 1] = S_MAX

    kwargs = {} if time_limit is None else {"options": {"time_limit":
                                                    float(time_limit)}}
    res = linprog(obj, A_eq=A_eq, b_eq=b_eq, bounds=list(zip(lb, ub)),
                  method="highs", **kwargs)
    if not res.success or res.x is None:
        return None
    return float(res.x[o["x"]]), float(res.x[o["y"]])

def clip_slot(xm, ym, g, load_kwh, pv_kwh):
    xe = min(xm, max(g + pv_kwh - load_kwh, 0.0))
    xe = max(xe, 0.0)
    ye = min(ym, load_kwh + xe)
    ye = max(ye, 0.0)
    b = load_kwh + xe - ye - g - pv_kwh
    if b >= 0.0:
        u, w, v = b, 0.0, pv_kwh
    else:
        u = 0.0
        w = min(g, -b)
        v = pv_kwh - (-b - w)
        v = max(v, 0.0)
    return xe, ye, u, w, v, b

def rolling_execute(g_contract, load_kwh, pv_kwh, price, s_init,
                  horizon=36, lam_price=None):
    lam_price = None
    n = len(g_contract)
    x = np.zeros(n); y = np.zeros(n); u = np.zeros(n)
    w = np.zeros(n); v = np.zeros(n); S = np.zeros(n)
    s = float(s_init)
    for t in range(n):
        K = min(t + horizon - 1, n - 1)
        seg = slice(t, K + 1)
        c_seg = price[seg]
        if lam_price is None:
            lam = ETA_D * float(np.median(c_seg))
        else:
            lam = lam_price[t]
        act = rolling_lp_action(s, g_contract[seg], load_kwh[seg], pv_kwh[seg],
                               c_seg, lam)
        if act is None:

```

```

        xm = ym = 0.0
    else:
        xm, ym = act
        xe, ye, ue, we, ve, _ = clip_slot(xm, ym, g_contract[t], load_
            kwh[t], pv_kwh[t])
        x[t], y[t], u[t], w[t], v[t] = xe, ye, ue, we, ve
        s = s + ETA_C * xe - ye / ETA_D
        if s < S_MIN - 1e-6 or s > S_MAX + 1e-6:
            raise RuntimeError(f"滚动执行 SOC 越界: t={t}, S={s}")
        s = min(max(s, S_MIN), S_MAX)
        S[t] = s
    return {"x": x, "y": y, "u": u, "w": w, "v": v, "S": S}

_CONTRACT_UPDATE_CACHE: dict = {}

def _contract_update_structure(n: int):
    cached = _CONTRACT_UPDATE_CACHE.get(n)
    if cached is not None:
        return cached
    N = 7 * n
    o = {"x": 0, "y": n, "v": 2 * n, "d": 3 * n, "e": 4 * n,
        "u": 5 * n, "S": 6 * n}
    A_eq = np.zeros((2 * n, N))
    for t in range(n):
        A_eq[t, o["x"] + t] = -1.0
        A_eq[t, o["y"] + t] = 1.0
        A_eq[t, o["v"] + t] = 1.0
        A_eq[t, o["d"] + t] = 1.0
        A_eq[t, o["e"] + t] = -1.0
        A_eq[t, o["u"] + t] = 1.0
        r = n + t
        A_eq[r, o["S"] + t] = 1.0
        if t > 0:
            A_eq[r, o["S"] + t - 1] = -1.0
            A_eq[r, o["x"] + t] = -ETA_C
            A_eq[r, o["y"] + t] = 1.0 / ETA_D
    result = (A_eq, o, N)
    _CONTRACT_UPDATE_CACHE[n] = result
    return result

def contract_update_lp(p_rem, c_rem, load_kwh, pv_kwh, s_init, s_end=
    None, lam=None):
    n = len(p_rem)
    A_eq, o, N = _contract_update_structure(n)
    bx, by, bv, bd, be, bu, bS = (o[k] for k in ("x", "y", "v", "d", "
        e", "u", "S"))
    b_eq = np.zeros(2 * n)
    for t in range(n):
        b_eq[t] = load_kwh[t] - p_rem[t]
    b_eq[n] = s_init

    obj = np.zeros(N)
    obj[bd:bd + n] = OVER_FACTOR * c_rem
    obj[be:be + n] = -UNDER_FACTOR * c_rem
    obj[bu:bu + n] = EMERGENCY_FACTOR * c_rem
    obj[bx:bx + n] += EPS_THROUGHPUT
    obj[by:by + n] += EPS_THROUGHPUT
    if lam is not None:
        obj[bS + n - 1] = -lam

    lb = np.zeros(N); ub = np.full(N, np.inf)

    ub[bx:bx + n] = np.minimum(B_MAX, np.maximum(p_rem + pv_kwh - load
        _kwh, 0.0))
    ub[by:by + n] = B_MAX
    ub[bv:bv + n] = pv_kwh
    ub[bu:bu + n] = np.maximum(load_kwh - pv_kwh, 0.0)
    ub[be:be + n] = p_rem
    lb[bS:bS + n] = S_MIN
    ub[bS:bS + n] = S_MAX
    if s_end is not None:
        lb[bS + n - 1] = ub[bS + n - 1] = s_end

    res = linprog(obj, A_eq=A_eq, b_eq=b_eq, bounds=list(zip(lb, ub)),
        method="highs")
    if not res.success:
        return None
    z = res.x
    q = p_rem + z[bd:bd + n] - z[be:be + n]
    return {
        "q": q, "x": z[bx:bx + n], "y": z[by:by + n],
        "v": z[bv:bv + n], "u": z[bu:bu + n],
        "S": z[bS:bS + n], "objective": float(res.fun),
    }

def settle_q12(p, u, price):
    return float(np.sum(price * p) + EMERGENCY_FACTOR * np.sum(price *
        u))

def settle_q3(p, q, u, price):
    over = np.maximum(q - p, 0.0)
    under = np.maximum(p - q, 0.0)
    return float(np.sum(price * np.minimum(p, q) + OVER_FACTOR * price
        * over
        + UNDER_FACTOR * price * under
        + EMERGENCY_FACTOR * price * u))

def forecast_f1(day_index, load_act, pv_act, load1, pv1):
    if day_index >= 7:
        lhat = load_act[day_index - 7].copy()
        vhat = pv_act[day_index - 7:day_index].mean(axis=0)
    elif day_index >= 1:
        lhat = load_act[:day_index].mean(axis=0)
        vhat = pv_act[:day_index].mean(axis=0)
    else:
        lhat = load1.copy()
        vhat = pv1.copy()
    return lhat, vhat

def empirical_quantile(samples, alpha):
    s = np.sort(np.asarray(samples, dtype=float))
    if s.size == 0:
        return 0.0
    k = int(np.ceil(alpha * s.size))
    k = min(max(k, 1), s.size)
    return float(s[k - 1])

```

```

def residual_quantile_safe(day_index, nhath_kw, vhat_kw, resid_hist,
    alpha, W):
    lo = max(1, day_index - W)
    n = len(nhath_kw)
    q = np.zeros(n)
    for t in range(n):
        samples = [resid_hist[(j, t)] for j in range(lo, day_index)
            if (j, t) in resid_hist]
        q[t] = empirical_quantile(samples, alpha) if samples else 0.0
    ntilde = nhath_kw + q
    vtilde = np.maximum(np.maximum(vhat_kw, -ntilde), 0.0)
    ltilde = vtilde + ntilde
    return ltilde, vtilde, ntilde

def bias_corrected_official_pv(day_index, issue_slot, pv_act, fc_raw,
    err_hist, W,
    anchor_prev):
    raw = fc_raw[day_index, issue_slot, :].copy()
    corr = np.zeros(24)
    for h in range(1, 25):
        key_h = h - 1
        samples = [err_hist[(j, issue_slot, key_h)] for j in range(
            max(0, day_index - W), day_index) if (j, issue_slot, key_h)
            in err_hist]
        corr[key_h] = float(np.mean(samples)) if samples else 0.0
    return np.maximum(raw + corr, 0.0)

def interp_official_profile(corrected_hourly, issue_hour, anchor_kw):
    pos0 = issue_hour * 6
    knots_x = [pos0]
    knots_y = [anchor_kw]
    for h in range(1, 25):
        pos = pos0 + 6 * h
        if pos > 144:
            break
        knots_x.append(pos)
        knots_y.append(corrected_hourly[h - 1])
    grid = np.arange(knots_x[0], knots_x[-1] + 1)
    prof = np.interp(grid, knots_x, knots_y)
    out = np.zeros(T + 1)
    out[knots_x[0]:knots_x[-1] + 1] = prof
    if knots_x[0] > 0:
        out[:knots_x[0]] = knots_y[0]
    return out

def price_forecast_0h(day_index, price_act, price1, base="median30"):
    if day_index <= 0:
        return price1.copy()
    lo = max(0, day_index - 30)
    hist = price_act[lo:day_index]
    if hist.shape[0] == 0:
        return price1.copy()
    if base == "median30":
        return np.median(hist, axis=0)
    if base == "mean30":
        return hist.mean(axis=0)
    if base == "mean7":
        return price_act[max(0, day_index - 7):day_index].mean(axis=0)
    if base == "prev_day":
        return price_act[day_index - 1].copy()
    raise ValueError(base)

def price_intraday_correct(base_pred, price_act_day, t, n_recent=18,
    decay=3.0, floor=1e-4):
    lo = max(0, t - n_recent)
    if t <= 0 or lo >= t:
        m = 0.0
    else:
        m = float(np.mean(price_act_day[lo:t] - base_pred[lo:t]))
    lead = (np.arange(len(base_pred)) - t) * DT
    corr = np.where(lead > 0, m * np.exp(-lead / decay), 0.0)
    return np.maximum(base_pred + corr, floor)

def four_hour_blocks(arr):
    a = np.asarray(arr, dtype=float).reshape(6, 24)
    return a.sum(axis=1)

def merge_intervals(mask, threshold=1e-8):
    spans = []
    t = 0
    n = len(mask)
    while t < n:
        if mask[t]:
            start = t
            while t + 1 < n and mask[t + 1]:
                t += 1
            spans.append((start, t))
            t += 1
    return spans

def slot_label_start(i):
    m = i * 10
    return f"{m // 60}:{m % 60:02d}"

def slot_label_end(i):
    m = (i + 1) * 10
    if m == 1440:
        return "0:00+1"
    return f"{m // 60}:{m % 60:02d}"

def interval_label(i0, i1):
    return f"{slot_label_start(i0)}-{slot_label_end(i1)}"

from __future__ import annotations

import datetime as _dt
import time

import numpy as np
import openpyxl
import pandas as pd

try:
    import microgrid_core as mc
except ModuleNotFoundError:

```

Listing 3: solution_lib.py

```

from scripts import microgrid_core as mc

ROOT = mc.ROOT
TPL_DIR = ROOT / "data" / "附件" / "附件5"
RES = ROOT / "results"
CSV = RES / "csv"
FIG = ROOT / "figures"

REP_DATES = [_dt.date(2025, 3, 20), _dt.date(2025, 6, 21),
              _dt.date(2025, 9, 23), _dt.date(2025, 12, 21)]
REP_SLOTS = [60, 72, 84, 96, 108, 120]
DAY_INDEX = {d: (d - mc.DAY0).days for d in REP_DATES}
BLOCK_LABELS = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
                 "12:00-16:00", "16:00-20:00", "20:00-24:00"]

def predict_center(kind: str, d: int, data: dict):
    load_act, pv_act = data["load_act"], data["pv_act"]
    if kind == "F1":
        return mc.forecast_f1(d, load_act, pv_act, data["load1"], data[
            "pv1"])
    if kind == "F2":
        cand = [load_act[d - k] for k in (7, 14, 21) if d - k >= 0]
        if not cand:
            lhat = load_act[:d].mean(axis=0) if d >= 1 else data["load1"]
        elif len(cand) == 1:
            lhat = cand[0].copy()
        else:
            lhat = np.median(np.vstack(cand), axis=0)
        if d >= 1:
            vhat = pv_act[max(0, d - 7):d].mean(axis=0)
        else:
            vhat = data["pv1"].copy()
        return lhat, vhat
    if kind == "F0":
        if d == 0:
            return data["load1"].copy(), data["pv1"].copy()
        j = np.arange(max(0, d - 90), d)
        w = np.exp(-(d - j) / 60.0)
        dow = np.array([(mc.DAY0 + _dt.timedelta(days=int(x))).weekday()
                        for x in j])
        w1 = w * np.where(dow == (mc.DAY0 + _dt.timedelta(days=d)).
                          weekday(), 2.0, 1.0)
        lhat = (load_act[j] * w1[:, None]).sum(axis=0) / w1.sum()
        vhat = (pv_act[j] * w1[:, None]).sum(axis=0) / w1.sum()
        rho = min(1.0, len(j) / 21.0)
        lhat = rho * lhat + (1 - rho) * data["load1"]
        vhat = rho * vhat + (1 - rho) * data["pv1"]
        return lhat, vhat
    raise ValueError(kind)

def net_residual_matrix(kind: str, data: dict) -> np.ndarray:
    load_act, pv_act = data["load_act"], data["pv_act"]
    out = np.zeros((365, mc.T))
    for d in range(365):
        lhat, vhat = predict_center(kind, d, data)
        out[d] = (load_act[d] - pv_act[d]) - (lhat - vhat)
    return out

def safe_trajectory(d: int, lhat_kw, vhat_kw, resid: np.ndarray,
                    alpha: float, W: int):
    lo = max(1, d - W)
    if d <= 0 or lo >= d:
        q = np.zeros(mc.T)
    else:
        srt = np.sort(resid[lo:d], axis=0)
        k = int(np.ceil(alpha * srt.shape[0]))
        k = min(max(k, 1), srt.shape[0])
        q = srt[k - 1]
    ntilde = lhat_kw - vhat_kw + q
    vtilde = np.maximum(np.maximum(vhat_kw, -ntilde), 0.0)
    ltilde = vtilde + ntilde
    return ltilde, vtilde, ntilde

def point_error_metrics(kind: str, data: dict) -> dict:
    eL, eV, eN = [], [], []
    for d in range(31, 365):
        lhat, vhat = predict_center(kind, d, data)
        eL.append(data["load_act"][d] - lhat)
        eV.append(data["pv_act"][d] - vhat)
        eN.append((data["load_act"][d] - data["pv_act"][d]) - (lhat -
            vhat))
    eL, eV, eN = np.asarray(eL), np.asarray(eV), np.asarray(eN)
    return {
        "load_MAE": float(np.abs(eL).mean()),
        "load_Bias": float(eL.mean()),
        "pv_MAE": float(np.abs(eV).mean()),
        "pv_Bias": float(eV.mean()),
        "net_MAE": float(np.abs(eN).mean()),
        "net_RMSE": float(np.sqrt((eN ** 2).mean())),
        "net_Bias": float(eN.mean()),
    }

def run_year_q2(data: dict, kind: str = "F1", alpha: float = 0.805, W
: int = 35,
                price_mode: str = "fixed", resid: np.ndarray | None =
                None,
                no_battery: bool = False, horizon: int = 36,
                n_days: int = 365, executor: str = "mpc",
                terminal_value_factor: float = 1.0) -> dict:
    if not no_battery:
        if price_mode == "fixed":
            mpc_price_mode, settle_price = "fixed", "fixed"
        elif price_mode == "actual":
            mpc_price_mode, settle_price = "oracle", "actual"
        else:
            raise ValueError(f"未知价格模式: {price_mode}")
    run = run_year_q34(
        data, prof=None, kind=kind, alpha=alpha, W=W, mode="uniform",
        H=horizon, allowed=(0,), updates=(), pv_source="center",
        price_mode=mpc_price_mode, settle_price=settle_price,
        n_days=n_days, resid_override=resid, executor=executor,
        terminal_value_factor=terminal_value_factor,
    )
    return run

load_act, pv_act = data["load_act"], data["pv_act"]
s = float(mc.S0)
n = n_days
P = np.zeros((n, mc.T))

```

```

X = np.zeros((n, mc.T))
Y = np.zeros((n, mc.T))
U = np.zeros((n, mc.T))
Wd = np.zeros((n, mc.T))
Vv = np.zeros((n, mc.T))
S = np.zeros((n, mc.T))
cash = np.zeros(n)
pv_used_ratio = np.zeros(n)
load_kwh = load_act * mc.DT
pv_kwh = pv_act * mc.DT
for d in range(n):
    c = data["price1"] if price_mode == "fixed" else data["price_act"][d]
    L = load_kwh[d]
    V = pv_kwh[d]
    p = np.maximum(L - V, 0.0)
    S[d] = s
    Vv[d] = np.minimum(V, L)
    U[d] = np.maximum(L - V - p, 0.0)
    Wd[d] = np.maximum(p - (L - Vv[d]), 0.0)
    cash[d] = float(np.sum(c * p) + mc.EMERGENCY_FACTOR * np.sum(c * U[d]))
    P[d] = p
return dict(P=P, X=X, Y=Y, U=U, W=Wd, V=Vv, S=S, cash=cash,
            kind=kind, alpha=alpha, resid_W=W, price_mode=price_mode,
            S_init=mc.S0, S_end=float(s), pv_curtail_ratio=pv_used_ratio,
            resid=resid, load_kwh=load_kwh, pv_kwh=pv_kwh,
            load_kw=load_act, pv_kw=pv_act)

def perfect_foresight_q2(data: dict, horizon: int = 36) -> dict:
    load_act, pv_act = data["load_act"], data["pv_act"]
    s = float(mc.S0)
    n = 365
    out = {k: np.zeros((n, mc.T)) for k in ("P", "X", "Y", "U", "W", "V", "S")}
    cash = np.zeros(n)
    for d in range(n):
        c = data["price1"]
        L = load_act[d] * mc.DT
        V = pv_act[d] * mc.DT
        lp = mc.solve_day_lp(c, L, V, s, lam=mc.ETA_D * float(np.median(c)))
        p = np.maximum(lp["g"], 0.0)
        pv_versions = np.tile(V, (N_ISSUE, 1))
        price_seg_fn = lambda t, K, c=c: c[t:K + 1]
        ex = rolling_execute_q34(
            p, L, pv_versions, L, V, price_seg_fn, s,
            horizon=horizon, allowed=(0,), time_limit=5.0,
        )
        out["P"][d] = p
        out["X"][d], out["Y"][d] = ex["x"], ex["y"]
        out["U"][d], out["W"][d], out["V"][d], out["S"][d] = ex["u"],
            ex["w"], ex["v"], ex["S"]
        cash[d] = mc.settle_q12(out["P"][d], out["U"][d], c)
        s = float(min(max(ex["S"][-1], mc.S_MIN), mc.S_MAX))
    out["cash"] = cash
    out["S_end"] = s
    return out

def official_slice(arr, offset: int = 31):
    return arr[offset:365]

def year_metrics(run: dict, offset: int = 31) -> dict:
    cash = official_slice(run["cash"], offset)
    U = official_slice(run["U"], offset)
    P = official_slice(run["P"], offset)
    Y = official_slice(run["Y"], offset)
    X = official_slice(run["X"], offset)
    Wd = official_slice(run["W"], offset)
    S = official_slice(run["S"], offset)
    price = None
    return {
        "daily_cash": cash,
        "cash_total": float(cash.sum()),
        "contract_kwh": float(P.sum()),
        "charge_kwh": float(X.sum()),
        "discharge_kwh": float(Y.sum()),
        "unused_contract_kwh": float(Wd.sum()),
        "emergency_kwh": float(U.sum()),
        "emergency_slots": int((U > 1e-8).sum()),
        "S_min": float(S.min()), "S_max": float(S.max()),
        "S_end": float(run["S_end"])
    }

def energy_block_table(run: dict, day_indices, offset: int = 31) -> pd.DataFrame:
    rows = []
    for d in day_indices:
        ch = mc.four_hour_blocks(run["X"][d])
        dis = mc.four_hour_blocks(run["Y"][d])
        s0 = run["S"][d - 1][-1] if d >= 1 else mc.S0
        for k in range(6):
            rows.append({
                "日期": str(mc.date_of(d)), "时段": BLOCK_LABELS[k],
                "充电量_kWh": ch[k], "放电量_kWh": dis[k],
                "日初储电量_kWh": s0, "日末储电量_kWh": run["S"][d][-1]
            })
    return pd.DataFrame(rows)

def slot_table(run: dict, price_day, day_indices, slots=None) -> pd.DataFrame:
    slots = REP_SLOTS if slots is None else slots
    rows = []
    for d in day_indices:
        c = price_day if np.ndim(price_day) == 1 else price_day[d]
        for t in slots:
            rows.append({
                "日期": str(mc.date_of(d)),
                "时段": f"{mc.slot_label_start(t)}-{mc.slot_label_end(t)}",
                "电价_元每kWh": c[t],
                "负荷_kWh": float(run["load_kw"][d][t]),
                "光伏实际_kWh": float(run["pv_kw"][d][t]),
                "合同购电_p_kWh": run["P"][d][t],
                "实际购电_kWh": run["P"][d][t] - run["W"][d][t] + run["U"][d][t],
                "应急购电_u_kWh": run["U"][d][t],
                "充电量_kWh": run["X"][d][t],
                "放电量_kWh": run["Y"][d][t],
                "槽末储电量_kWh": run["S"][d][t],
            })
    return pd.DataFrame(rows)

```

```

    "official_days": int(max(0, n - offset)),
    **bounds,
}

def emergency_intervals(U_day: np.ndarray, date, threshold: float = 1
                        e-8,
                        max_intervals: int = 3) -> list:
    spans = mc.merge_intervals(np.asarray(U_day) > threshold)
    out = []
    for i0, i1 in spans[:max_intervals]:
        out.append({"日期": str(date), "购电时间段": mc.interval_label(
            i0, i1),
            "购电量_kWh": float(U_day[i0:i1 + 1].sum())})
    return out

def emergency_table(run: dict, day_indices, threshold: float = 1e-8)
    -> pd.DataFrame:
    rows = []
    for d in day_indices:
        ivs = emergency_intervals(run["U"][d], mc.date_of(d), threshold
            )
        if not ivs:
            rows.append({"日期": str(mc.date_of(d)), "购电时间段": "无",
                "购电量_kWh": 0.0})
        rows.extend(ivs)
    return pd.DataFrame(rows)

def check_run(run: dict, offset: int = 31) -> dict:
    P, X, Y, U, Wd, V, S = (np.asarray(run[k])
        for k in ("P", "X", "Y", "U", "W", "V", "S"))

    n = P.shape[0]
    L = np.asarray(run["load_kwh"])[0:n]
    pv = np.asarray(run["pv_kwh"])[0:n]
    bal = np.abs((P - Wd) + V + Y + U - X - L)
    soc_chain = 0.0
    for d in range(1, n):
        expected = S[d - 1, -1] + mc.ETA_C * X[d, 0] - Y[d, 0] / mc.ETA
            _D
        soc_chain = max(soc_chain, abs(S[d, 0] - expected))
    price = run.get("price_settle")
    settle = float("nan") if price is None else float(np.abs(
        np.asarray(run["cash"])[0:n]
        - (np.asarray(run["cash_contract"])[0:n]
            + mc.EMERGENCY_FACTOR * (np.asarray(price)[0:n] * U).sum(axis
                =1))
    ).max())
    bounds = {
        "bound_p": float(-min(P.min(), 0.0)),
        "bound_x": float(max(-min(X.min(), 0.0), (X - mc.B_MAX).max())),
        ,
        "bound_y": float(max(-min(Y.min(), 0.0), (Y - mc.B_MAX).max())),
        ,
        "bound_v": float(max(-min(V.min(), 0.0), (V - pv).max())),
        "bound_u": float(-min(U.min(), 0.0)),
        "bound_w": float(max(-min(Wd.min(), 0.0), (Wd - P).max())),
        "bound_S": float(max(mc.S_MIN - S.min(), S.max() - mc.S_MAX)),
    }
    return {
        "balance_max": float(bal.max()),
        "overlap_max": float(np.minimum(X, Y).max()),
        "charge_emergency_max": float(np.minimum(X, U).max()),
        "S_min": float(S.min()), "S_max": float(S.max()),
        "soc_chain_max_kWh": float(soc_chain),
        "soc_end_mismatch_kWh": float(abs(S[-1, -1] - run["S_end"])),
        "settle_flow_max_yuan": settle,
    }

def _date_blocks(ws, n_rows: int) -> list:
    blocks = []
    for row in range(2, n_rows + 1):
        value = ws.cell(row=row, column=1).value
        if value is None:
            continue
        text = str(value).strip()
        if text[:4].isdigit() and "-" in text:
            blocks.append((row, text[:10]))
    return blocks

def _day_index_of(date_text: str, offset: int = 31):
    try:
        d = _dt.date.fromisoformat(date_text[:10])
    except ValueError:
        return None
    idx = (d - mc.DAY0).days
    return idx if offset <= idx < 365 else None

def _fill_plan_sheet(ws, run: dict, offset: int = 31, key: str = "P",
    fee_key: str = "cash") -> None:
    arr = run[key]
    fee = run.get(fee_key, run["cash"])
    for r, d in enumerate(range(offset, arr.shape[0])):
        row = 2 + r
        a = arr[d]
        for t in range(mc.T):
            ws.cell(row=row, column=2 + t).value = float(a[t])
            ws.cell(row=row, column=146).value = float(a.sum())
            ws.cell(row=row, column=147).value = float(fee[d])

def _fill_cycle_sheet(ws, run: dict, offset: int = 31) -> None:
    for base, date_text in _date_blocks(ws, ws.max_row):
        d = _day_index_of(date_text, offset)
        if d is None or d >= run["X"].shape[0]:
            continue
        ch = mc.four_hour_blocks(run["X"][d])
        dis = mc.four_hour_blocks(run["Y"][d])
        for k in range(6):
            ws.cell(row=base + k, column=3).value = float(ch[k])
            ws.cell(row=base + k, column=4).value = float(dis[k])
            ws.cell(row=base, column=6).value = float(
                run["S"][d - 1][-1] if d >= 1 else mc.S0)
            ws.cell(row=base + 1, column=6).value = float(run["S"][d][-1])

def _fill_emergency_sheet(ws, run: dict, offset: int = 31) -> None:
    for base, date_text in _date_blocks(ws, ws.max_row):
        d = _day_index_of(date_text, offset)
        if d is None or d >= run["U"].shape[0]:
            continue
        ivs = emergency_intervals(run["U"][d], mc.date_of(d))
        if not ivs:
            ws.cell(row=base, column=2).value = "无"
            ws.cell(row=base, column=3).value = 0.0

```

```

        continue
    for k, iv in enumerate(ivs):
        ws.cell(row=base + k, column=2).value = iv["购电时间段"]
        ws.cell(row=base + k, column=3).value = iv["购电量_kWh"]

def write_workbook(template: str, out_name: str, sheets: dict) -> None:
    RES.mkdir(parents=True, exist_ok=True)
    wb = openpyxl.load_workbook(TPL_DIR / template)
    for name, filler in sheets.items():
        filler(wb[name])
    wb.save(RES / out_name)

def monthly_summary(run: dict, offset: int = 31) -> pd.DataFrame:
    rows = []
    for d in range(offset, run["P"].shape[0]):
        dt = mc.date_of(d)
        rows.append({"日期": dt, "月份": dt.strftime("%Y-%m"),
            "现金费_元": run["cash"][d],
            "应急购电量_kWh": run["U"][d].sum(),
            "合同购电量_kWh": run["P"][d].sum(),
            "充电量_kWh": run["X"][d].sum(),
            "放电量_kWh": run["Y"][d].sum(),
            "弃购量_kWh": run["W"][d].sum(),
            "弃光量_kWh": float((run["pv_kwh"][d] - run["V"][d]).sum())
            if "pv_kwh" in run else np.nan,
            "日末储电量_kWh": run["S"][d][-1]})
    return pd.DataFrame(rows)

ISSUE_SLOTS = (0, 36, 72, 108)
ISSUE_HOURS = (0, 6, 12, 18)
N_ISSUE = 4
N_STRATA = 6

def build_pv_archive(data: dict, W: int = 30) -> np.ndarray:
    pv_act = data["pv_act"]
    fc = data["fc_raw"]
    prof = np.zeros((365, N_ISSUE, mc.T + 1))
    for d in range(365):
        lo = max(0, d - W)
        for r in range(N_ISSUE):
            corr = np.zeros(24)
            if d > lo:
                for h in range(1, 25):
                    pos = ISSUE_SLOTS[r] + 6 * h
                    if pos > mc.T:
                        break
                    corr[h - 1] = float(np.mean(
                        [pv_act[j, pos - 1] - fc[j, r, h - 1] for j in
                        range(lo, d)]))
            corrected = np.maximum(fc[d, r, :] + corr, 0.0)
            if r == 0:
                anchor = float(pv_act[d - 1, -1]) if d >= 1 else 0.0
            else:
                anchor = float(pv_act[d, ISSUE_SLOTS[r] - 1])
            prof[d, r] = mc.interp_official_profile(corrected, ISSUE_
                HOURS[r], anchor)
    return prof

def issue_index_of_slot(t: int, allowed=(0, 1, 2, 3)) -> int:
    best = 0
    for r in allowed:
        if ISSUE_SLOTS[r] <= t:
            best = r
    return best

def version_of_slot_array(allowed=(0, 1, 2, 3)) -> np.ndarray:
    return np.array([issue_index_of_slot(t, allowed) for t in range(mc
        .T)], dtype=int)

def pv_version_curve(prof_day: np.ndarray, allowed=(0, 1, 2, 3)) -> np.ndarray:
    return np.array([prof_day[issue_index_of_slot(t, allowed), t + 1]
        for t in range(mc.T)])

def horizon_stratum(allowed=(0, 1, 2, 3)) -> np.ndarray:
    out = np.empty(mc.T, dtype=int)
    for t in range(mc.T):
        r = issue_index_of_slot(t, allowed)
        out[t] = min((t - ISSUE_SLOTS[r]) // 6, N_STRATA - 1)
    return out

STRATUM = horizon_stratum()

def q3_net_residual(data: dict, prof: np.ndarray, allowed=(0, 1, 2,
    3),
        kind: str = "F1", pv_source: str = "archive"):
    pv_ver = np.zeros((365, mc.T))
    resid = np.zeros((365, mc.T))
    for d in range(365):
        lhat, vhat = predict_center(kind, d, data)
        if pv_source == "archive":
            pv_ver[d] = pv_version_curve(prof[d], allowed)
        else:
            pv_ver[d] = vhat
            resid[d] = (data["load_act"][d] - data["pv_act"][d]) - (lhat -
                pv_ver[d])
    return resid, pv_ver

def q3_reserve(d: int, resid: np.ndarray, alpha: float, W: int,
    mode: str = "stratified", allowed=(0, 1, 2, 3)):
    lo = max(1, d - W)
    if d <= 0 or lo >= d:
        return np.zeros(mc.T), np.ones(N_STRATA)
    hist = resid[lo:d]
    if mode == "uniform":
        return (np.array([mc.empirical_quantile(hist[:, t], alpha)
            for t in range(mc.T)]), np.ones(N_STRATA))
    stratum = horizon_stratum(allowed)
    sig = np.ones(N_STRATA)
    for h in range(N_STRATA):
        cols = np.where(stratum == h)[0]
        if cols.size:
            sig[h] = float(np.sqrt((hist[:, cols] ** 2).mean()))
    sig = np.maximum.accumulate(np.maximum(sig, 1e-9))
    z = hist / sig[stratum][None, :]
    qs = mc.empirical_quantile(z.ravel(), alpha)

```

```

return sig[stratum] * qs, sig

def safe_from_reserve(lhat_kwh, vhat_kwh, reserve_kwh):
    ntilde = lhat_kwh - vhat_kwh + reserve_kwh
    vtild = np.maximum(np.maximum(vhat_kwh, -ntilde), 0.0)
    return vtild + ntilde, vtild, ntilde

def rolling_execute_q34(g_contract, load_pred_kwh, pv_ver_kwh, load_
    kwh, pv_kwh,
        price_seg_fn, s_init, horizon: int = 36,
        allowed=(0, 1, 2, 3), time_limit: float = 5.0,
        contract_update_fn=None, executor: str = "mpc",
        terminal_value_factor: float = 1.0):
    n = mc.T
    contract = np.asarray(g_contract, dtype=float).copy()
    x = np.zeros(n); y = np.zeros(n); u = np.zeros(n)
    w = np.zeros(n); v = np.zeros(n); S = np.zeros(n)
    plan_x = np.zeros(n); plan_y = np.zeros(n); plan_s_next = np.zeros
        (n)
    s = float(s_init)
    stat = {"clip_charge": 0, "clip_discharge": 0, "fallback": 0,
        "updates": 0, "lp_time": []}
    if executor not in ("mpc", "greedy"):
        raise ValueError(f"未知执行器: {executor}")
    prev_net = None
    for t in range(n):
        if contract_update_fn is not None:
            q_rem = contract_update_fn(t, s, contract)
            if q_rem is not None:
                q_rem = np.asarray(q_rem, dtype=float)
                if q_rem.shape != (n - t,):
                    raise ValueError(f"合同更新长度异常: t={t}, shape={q_
                        rem.shape}")
                contract[t:] = np.maximum(q_rem, 0.0)
                stat["updates"] += 1
        r = issue_index_of_slot(t, allowed)
        load_curve = (load_pred_kwh[r] if np.ndim(load_pred_kwh) == 2
            else load_pred_kwh)
        if executor == "mpc":
            K = min(t + horizon - 1, n - 1)
            seg = slice(t, K + 1)
            c_seg = price_seg_fn(t, K)
            lam = terminal_value_factor * mc.ETA_D * float(np.median(c_
                seg))
            tic = time.perf_counter()
            act = mc.rolling_lp_action(s, contract[seg], load_curve[seg
                ],
                    pv_ver_kwh[r][seg], c_seg, lam,
                    time_limit=time_limit)
            stat["lp_time"].append(time.perf_counter() - tic)
        if act is None:
            net_est = float(prev_net) if prev_net is not None else
                float(
                    load_curve[t] - pv_ver_kwh[r][t])
            if net_est < 0.0:
                xm = min(mc.B_MAX, -net_est,
                    max((mc.S_MAX - s) / mc.ETA_C, 0.0))
                ym = 0.0
            else:
                xm = 0.0
                ym = min(mc.B_MAX, net_est,
                    max((s - mc.S_MIN) * mc.ETA_D, 0.0))
            stat["fallback"] += 1
    else:

```

```

        xm, ym = act
    else:
        surplus = max(contract[t] + pv_ver_kwh[r][t] - load_curve[t
            ], 0.0)
        deficit = max(load_curve[t] - contract[t] - pv_ver_kwh[r][t
            ], 0.0)
        xm = min(mc.B_MAX, surplus, max((mc.S_MAX - s) / mc.ETA_C,
            0.0))
        ym = min(mc.B_MAX, deficit, max((s - mc.S_MIN) * mc.ETA_D,
            0.0))
    xe, ye, ue, we, ve, _ = mc.clip_slot(xm, ym, contract[t], load_
        kwh[t],
            pv_kwh[t])
    plan_x[t], plan_y[t] = xm, ym
    plan_s_next[t] = min(max(s + mc.ETA_C * xm - ym / mc.ETA_D,
        mc.S_MIN), mc.S_MAX)
    if xm - xe > 1e-9:
        stat["clip_charge"] += 1
    if ym - ye > 1e-9:
        stat["clip_discharge"] += 1
    x[t], y[t], u[t], w[t], v[t] = xe, ye, ue, we, ve
    s = s + mc.ETA_C * xe - ye / mc.ETA_D
    s = min(max(s, mc.S_MIN), mc.S_MAX)
    S[t] = s
    prev_net = float(load_kwh[t] - pv_kwh[t])
    return {"x": x, "y": y, "u": u, "w": w, "v": v, "S": S,
        "plan_x": plan_x, "plan_y": plan_y,
        "plan_s_next": plan_s_next,
        "contract": contract, "stats": stat}

def run_year_q34(data: dict, prof: np.ndarray | None = None, kind:
    str = "F1",
        alpha: float = 0.7725, W: int = 35, mode: str = "
            stratified",
            H: int = 6, allowed=(0, 1, 2, 3), updates=(1, 2, 3),
            pv_source: str = "archive", price_mode: str = "fixed",
            price_base: str = "median30", settle_price: str = "
                fixed",
            time_limit: float = 5.0, n_days: int = 365,
            resid_override: np.ndarray | None = None,
            executor: str = "mpc", terminal_value_factor: float = 8
                1.0) -> dict:
    if prof is None and pv_source == "archive":
        prof = build_pv_archive(data, W)
    resid, pv_ver = q3_net_residual(data, prof, allowed, kind, pv_
        source)
    if resid_override is not None:
        resid = np.asarray(resid_override, dtype=float)
    load_kw = data["load_act"]
    pv_kw = data["pv_act"]
    load_kwh = load_kw * mc.DT
    pv_kwh = pv_kw * mc.DT
    keys = ("P", "Q", "X", "Y", "U", "W", "V", "S",
        "plan_x", "plan_y", "plan_s_next")
    arr = {k: np.zeros((n_days, mc.T)) for k in keys}
    cash = np.zeros(n_days)
    cash_plan = np.zeros(n_days)
    cash_contract = np.zeros(n_days)
    price_settle = np.zeros((n_days, mc.T))
    stats = {"clip_charge": 0, "clip_discharge": 0, "fallback": 0,
        "updates": 0, "lp_time": []}
    daily_cnt = {k: np.zeros(n_days, dtype=int)
        for k in ("clip_charge", "clip_discharge", "fallback",
            "updates")}

```

```

lp_by_day = []
s = float(mc.S0)
for d in range(n_days):
    lhat_kw, vhat_kw = predict_center(kind, d, data)
    load_pred = lhat_kw * mc.DT
    if pv_source == "archive":

        pv_curves = np.vstack([prof[d, r, 1:mc.T + 1] for r in
                                range(N_ISSUE)]) * mc.DT
    else:
        pv_curves = np.tile(vhat_kw * mc.DT, (N_ISSUE, 1))
    if price_mode == "fixed":
        forecast_price = data["price1"]
        price_seg_fn = (lambda t, K, bp=forecast_price: bp[t:K +
1])
    elif price_mode == "oracle":

        forecast_price = data["price_act"][d]
        price_seg_fn = (lambda t, K, bp=forecast_price: bp[t:K +
1])
    else:
        forecast_price = mc.price_forecast_0h(d, data["price_act"],
data["price1"],
                                price_base)

    def price_seg_fn(t, K, bp=forecast_price, d=d):
        return mc.price_intraday_correct(bp, data["price_act"][d
], t)[t:K + 1]

    settle_day_price = data["price1"] if settle_price == "fixed"
    else data["price_act"][d]
    price_settle[d] = settle_day_price

    reserve_kw, _ = q3_reserve(d, resid, alpha, W, mode, allowed)
    reserve = reserve_kw * mc.DT

    safe_load_curves = np.zeros_like(pv_curves)
    safe_pv_curves = np.zeros_like(pv_curves)
    for r in range(N_ISSUE):
        safe_load_curves[r], safe_pv_curves[r], _ = safe_from_
reserve(
        load_pred, pv_curves[r], reserve
    )

    ltil, vtil = safe_load_curves[0], safe_pv_curves[0]
    lp = mc.solve_day_lp(forecast_price, ltil, vtil, s,
        lam=terminal_value_factor * mc.ETA_D * float(
            np.median(forecast_price)))
    p = np.maximum(lp["g"], 0.0)
    q = p.copy()
    cash_plan[d] = float(np.sum(forecast_price * p))
    update_at = {ISSUE_SLOTS[r]: r for r in updates}

    def update_contract(t, current_s, current_q):
        r_up = update_at.get(t)
        if r_up is None:
            return None
        c_up = price_seg_fn(t, mc.T - 1)
        ltil_up = safe_load_curves[r_up][t:]
        vtil_up = safe_pv_curves[r_up][t:]

        up = mc.contract_update_lp(
            p[t:], c_up, ltil_up, vtil_up, current_s,
            lam=terminal_value_factor * mc.ETA_D * float(np.median(c
_up)),
        )

```

```

        return None if up is None else up["q"]

    ex = rolling_execute_q34(q, safe_load_curves, safe_pv_curves,
        load_kwh[d], pv_kwh[d],
        price_seg_fn, s, horizon=H, allowed=
        allowed,
        time_limit=time_limit,
        contract_update_fn=update_contract if
        updates else None,
        executor=executor,
        terminal_value_factor=terminal_value_
        factor)

    q = ex["contract"]
    cash_contract[d] = mc.settle_q3(p, q, np.zeros(mc.T), settle_
        day_price)
    for k, key in (("x", "X"), ("y", "Y"), ("u", "U"), ("w", "W"),
        ("v", "V"), ("S", "S"), ("plan_x", "plan_x"),
        ("plan_y", "plan_y"), ("plan_s_next", "plan_s_next
        ")):
        arr[key][d] = ex[k]
    arr["P"][d] = p
    arr["Q"][d] = q
    cash[d] = mc.settle_q3(p, q, ex["u"], settle_day_price)
    for k in ("clip_charge", "clip_discharge", "fallback", "updates
        "):
        stats[k] += ex["stats"][k]
    stats["lp_time"].extend(ex["stats"]["lp_time"])
    for k in ("clip_charge", "clip_discharge", "fallback", "updates
        "):
        daily_cnt[k][d] += ex["stats"][k]
    lp_by_day.append(list(ex["stats"]["lp_time"]))
    s = float(min(max(ex["S"][-1], mc.S_MIN), mc.S_MAX))
    stats["daily_cnt"] = daily_cnt
    stats["lp_by_day"] = lp_by_day
    out = dict(arr)
    out.update(cash=cash, cash_plan=cash_plan, cash_contract=cash_
        contract,
        price_settle=price_settle,
        stats=stats, kind=kind, alpha=alpha, resid_W=W, mode=mode
        , H=H,
        allowed=tuple(allowed), updates=tuple(updates), pv_source
        =pv_source,
        price_mode=price_mode, price_base=price_base,
        settle_price=settle_price, executor=executor,
        terminal_value_factor=terminal_value_factor,
        S_init=mc.S0, S_end=float(s),
        resid=resid, pv_ver=pv_ver, load_kwh=load_kwh, pv_kwh=pv_
        kwh,
        load_kw=load_kw, pv_kw=pv_kw)
    return out

def _sliced_count(stats: dict, key: str, offset: int) -> int:
    dc = stats.get("daily_cnt")
    if dc is None:
        return int(stats[key])
    return int(np.asarray(dc[key])[offset:].sum())

def year_metrics_q34(run: dict, offset: int = 31) -> dict:
    P = official_slice(run["P"], offset)
    Q = official_slice(run["Q"], offset)
    U = official_slice(run["U"], offset)
    X = official_slice(run["X"], offset)
    Y = official_slice(run["Y"], offset)
    Wd = official_slice(run["W"], offset)

```

```

S = official_slice(run["S"], offset)
lp = np.asarray(run["stats"]["lp_time"], dtype=float)
by_day = run["stats"].get("lp_by_day")
if by_day is not None:

    lp = np.asarray([v for day in by_day[offset:] for v in day],
                     dtype=float)
return {
    "daily_cash": official_slice(run["cash"], offset),
    "cash_total": float(official_slice(run["cash"], offset).sum()),
    "plan_kwh": float(P.sum()),
    "contract_kwh": float(Q.sum()),
    "adjust_up_kwh": float(np.maximum(Q - P, 0.0).sum()),
    "adjust_down_kwh": float(np.maximum(P - Q, 0.0).sum()),
    "charge_kwh": float(X.sum()),
    "discharge_kwh": float(Y.sum()),
    "unused_contract_kwh": float(Wd.sum()),
    "emergency_kwh": float(U.sum()),
    "emergency_slots": int((U > 1e-8).sum()),
    "S_min": float(S.min()), "S_max": float(S.max()), "S_end":
        float(run["S_end"]),
    "updates": _sliced_count(run["stats"], "updates", offset),
    "clip_charge": _sliced_count(run["stats"], "clip_charge",
                                offset),
    "clip_discharge": _sliced_count(run["stats"], "clip_discharge",
                                    offset),
    "fallback": _sliced_count(run["stats"], "fallback", offset),
    "lp_ms_median": float(np.median(lp) * 1e3) if lp.size else
        float("nan"),
    "lp_ms_p95": float(np.percentile(lp, 95) * 1e3) if lp.size else
        float("nan"),
    "lp_ms_max": float(lp.max() * 1e3) if lp.size else float("nan")
    ,
}

def check_run_q34(run: dict, offset: int = 31) -> dict:
    P, Q, X, Y, U, Wd, V, S = (np.asarray(run[k])
                                for k in ("P", "Q", "X", "Y", "U", "W", "V",
                                           "S"))

    n = P.shape[0]

    L, pv = np.asarray(run["load_kwh"][:n], np.asarray(run["pv_kwh"][:n])

    bal = np.abs((Q - Wd) + V + Y + U - X - L)

    b_p = float(-min(P.min(), 0.0))
    b_q = float(-min(Q.min(), 0.0))
    b_x = float(max(-min(X.min(), 0.0), (X - mc.B_MAX).max()))
    b_y = float(max(-min(Y.min(), 0.0), (Y - mc.B_MAX).max()))
    b_v = float(max(-min(V.min(), 0.0), (V - pv).max()))
    b_u = float(-min(U.min(), 0.0))
    b_w = float(max(-min(Wd.min(), 0.0), (Wd - Q).max()))
    b_s = float(max(mc.S_MIN - S.min(), S.max() - mc.S_MAX))
    chain = 0.0
    for d in range(1, n):
        chain = max(chain, abs((S[d][0] - S[d - 1][-1])
                               - (mc.ETA_C * X[d][0] - Y[d][0] / mc.ETA_D)))

    price = run.get("price_settle")
    if price is None:
        flow = float("nan")
    else:
        flow = float(np.abs(run["cash"] - (run["cash_contract"]
                                           + mc.EMERGENCY_FACTOR * (price * U

```

```

        ).sum(axis=1))).max())

    return {
        "balance_max_kWh": float(bal.max()),
        "overlap_max_kWh": float(np.minimum(X, Y).max()),
        "charge_emergency_max_kWh": float(np.minimum(X, U).max()),
        "bound_p": b_p, "bound_q": b_q, "bound_x": b_x, "bound_y": b_y,
        "bound_v": b_v, "bound_u": b_u, "bound_w": b_w, "bound_s": b_s,
        "soc_chain_max_kWh": float(chain),
        "soc_end_mismatch_kWh": float(abs(S[-1][-1] - run["S_end"])),
        "settle_flow_max_yuan": flow,
        "official_days": int(uniform_slice_count(n, offset)),
    }

def uniform_slice_count(n: int, offset: int) -> int:
    return max(0, n - offset)

```

Listing 4: 问题 1_ 求解.py

```

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))

import matplotlib.pyplot as plt
import numpy as np
import openpyxl
import pandas as pd

import microgrid_core as mc
import utils.plot_style as ps

ROOT = mc.ROOT
TPL_DIR = ROOT / "data" / "附件" / "附件5"
RES = ROOT / "results"
CSV = RES / "csv"
FIG = ROOT / "figures" / "问题1"

def solve() -> dict:
    data = mc.load_inputs()
    c = data["price1"]
    load_kwh = data["load1"] * mc.DT
    pv_kwh = data["pv1"] * mc.DT
    res = mc.solve_day_lp(c, load_kwh, pv_kwh, mc.S0, s_end=mc.S0)

    S = np.concatenate([[mc.S0], res["S"]])
    g = np.maximum(res["g"], 0.0)
    x = np.maximum(res["x"], 0.0)
    y = np.maximum(res["y"], 0.0)
    v = np.maximum(res["v"], 0.0)
    out = dict(
        c=c, load_kwh=load_kwh, pv_kwh=pv_kwh,
        g=g, x=x, y=y, v=v, S=S,
        curtail=np.maximum(res["curtail"], 0.0),
        cash_cost=float(np.sum(c * g)),
        solver_objective=res["objective"],
    )
    return out

```

```
def verify(sol: dict) -> dict:
    g, x, y, v, S = sol["g"], sol["x"], sol["y"], sol["v"], sol["S"]
    L, V = sol["load_kwh"], sol["pv_kwh"]
    balance_res = float(np.abs(g + v + y - x - L).max())
    soc_res = float(np.abs(np.diff(S) - (mc.ETA_C * x - y / mc.ETA_D))
        .max())
    bound_res = float(max(
        max(np.maximum(S - mc.S_MAX, 0).max(), np.maximum(mc.S_MIN - S,
            0).max()),
        max(np.maximum(x - mc.B_MAX, 0).max(), np.maximum(y - mc.B_MAX,
            0).max()),
        max(np.maximum(v - V, 0).max(), -min(v.min(), 0.0)),
        -min(g.min(), 0.0),
    ))
    overlap = float(np.minimum(x, y).max())
    X, Y, G = float(x.sum()), float(y.sum()), float(g.sum())
    ident = {
        "Y_minus_0.81X": Y - mc.ETA_C * mc.ETA_D * X,
        "G_identity": G - (float(L.sum()) - float(v.sum()) + (1 - mc.
            ETA_C * mc.ETA_D) * X),
        "no_battery_purchase": float((L - V).clip(min=0).sum()),
    }
    return dict(
        balance_max=balance_res, soc_res_max=soc_res, bound_max=bound_
            res,
        overlap_max=overlap, X=X, Y=Y, G=G,
        S_min=float(S.min()), S_max=float(S.max()),
        curtail_total=float(sol["curtail"].sum()),
        cash_cost=float(sol["cash_cost"]),
        solver_objective=float(sol["solver_objective"]),
        **ident,
    )
```

```
def write_template(sol: dict) -> None:
    RES.mkdir(parents=True, exist_ok=True)
    wb = openpyxl.load_workbook(TPL_DIR / "result1.xlsx")
    ws = wb["计划购电量"]
    for i in range(mc.T):
        ws.cell(row=2 + i, column=2).value = float(sol["g"][i])
    ws2 = wb["充放电量"]
    ch = mc.four_hour_blocks(sol["x"])
    dis = mc.four_hour_blocks(sol["y"])
    for k in range(6):
        ws2.cell(row=2 + k, column=2).value = float(ch[k])
        ws2.cell(row=2 + k, column=3).value = float(dis[k])
    ws2.cell(row=2, column=5).value = float(sol["S"][0])
    ws2.cell(row=3, column=5).value = float(sol["S"][-1])
    wb.save(RES / "result1.xlsx")
```

```
def write_csv(sol: dict, met: dict) -> None:
    CSV.mkdir(parents=True, exist_ok=True)
    idx = np.arange(mc.T)
    df = pd.DataFrame({
        "槽序号": idx + 1,
        "自然区间": [f"{mc.slot_label_start(i)}-{mc.slot_label_end(i)}"
            for i in idx],
        "电价_元每kWh": sol["c"],
        "负荷_kWh": sol["load_kwh"],
        "光伏可用_kWh": sol["pv_kwh"],
        "光伏利用_kWh": sol["v"],
        "弃光_kWh": sol["curtail"],
        "计划购电量_kWh": sol["g"],
        "充电量_kWh": sol["x"],
        "放电量_kWh": sol["y"],
```

```
        "槽末储电量_kWh": sol["S"][1:],
    })
    df.to_csv(CSV / "问题1_逐槽计划.csv", index=False, encoding="utf-8-
        sig")
```

```
blocks = pd.DataFrame({
    "时间段": ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00",
        "16:00-20:00", "20:00-24:00"],
    "充电量_kWh": mc.four_hour_blocks(sol["x"]),
    "放电量_kWh": mc.four_hour_blocks(sol["y"]),
})
blocks.loc[len(blocks)] = ["全天", blocks["充电量_kWh"].sum(),
    blocks["放电量_kWh"].sum()]
blocks.to_csv(CSV / "问题1_四小时聚合.csv", index=False, encoding="
    utf-8-sig")
```

```
summary = pd.DataFrame([
    "项目": "数值", "全天计划购电量_kWh": met["G"],
    "全天购电费_元": met["cash_cost"],
    "含吞吐正则的求解目标值": met["solver_objective"],
    "全天充电量_kWh": met["X"], "全天放电量_kWh": met["Y"],
    "0时储电量_kWh": sol["S"][0], "24时储电量_kWh": sol["S"][-1],
    "弃光量_kWh": met["curtail_total"],
    "无储能购电量_kWh": met["no_battery_purchase"],
])
summary.to_csv(CSV / "问题1_汇总.csv", index=False, encoding="utf
    -8-sig")
with open(CSV / "问题1_校验.json", "w", encoding="utf-8") as fh:
    json.dump(met, fh, ensure_ascii=False, indent=2)
```

```
def figures(sol: dict, met: dict) -> None:
```

```
    FIG.mkdir(parents=True, exist_ok=True)
    ps.apply_publication_style("zh", "report")
    t = (np.arange(mc.T) + 0.5) * mc.DT
    L = sol["load_kwh"] / mc.DT
    V = sol["pv_kwh"] / mc.DT
    g = sol["g"] / mc.DT
```

```
fig, axes = ps.publication_subplots(3, 1, aspect=1.55, height_
    ratios=[1, 1.35, 1.25])
ax = axes[0]
ax.plot(t, sol["c"], color=ps.PALETTE["secondary"])
ax.set_ylabel("电价/(元/kWh)")
ax.set_title("典型日电价")
ax.set_xlim(0, 24)
```

```
ax = axes[1]
ax.plot(t, L, color=ps.PALETTE["dark"], label="负荷")
ax.plot(t, V, color=ps.PALETTE["positive"], label="可用光伏")
ax.plot(t, g, color=ps.PALETTE["primary"], label="计划购电")
ax.plot(t, sol["v"] / mc.DT, color=ps.PALETTE["sky"], lw=0.9,
    label="光伏利用")
ax.set_ylabel("功率/kW")
ax.set_title("负荷、光伏与计划购电")
ax.legend(ncols=2)
ax.set_xlim(0, 24)
```

```
ax = axes[2]
ax.plot(t, sol["S"][1:], color=ps.PALETTE["contrast"], label="储电
    量")
ax.axhline(mc.S_MIN, color=ps.PALETTE["neutral"], lw=0.7, ls="--",
    label="SOC 上下限")
ax.axhline(mc.S_MAX, color=ps.PALETTE["neutral"], lw=0.7, ls="--")
```

```

ax.set_ylabel("储电量/kWh")
ax.set_title("储能状态轨迹")
ax.set_xlabel("时刻/h")
ax.legend()
ax.set_xlim(0, 24)
ps.add_panel_labels(axes)
ps.export_figure(fig, FIG / "问题1_典型日最优计划")
plt.close(fig)

fig, ax = ps.publication_subplots(1, 1, aspect=0.55)
labels = ["0-4", "4-8", "8-12", "12-16", "16-20", "20-24"]
pos = np.arange(6)
ch = mc.four_hour_blocks(sol["x"])
dis = mc.four_hour_blocks(sol["y"])
ax.bar(pos - 0.18, ch, 0.36, color=ps.PALETTE["primary"], label="
    充电量")
ax.bar(pos + 0.18, dis, 0.36, color=ps.PALETTE["contrast"], label="
    放电量")
ax.set_xticks(pos, labels)
ax.set_xlabel("时段/h")
ax.set_ylabel("电量/kWh")
ax.set_title("四小时聚合充放电电量")
ax.legend()
ps.export_figure(fig, FIG / "问题1_四小时聚合充放电")
plt.close(fig)

fig, axes = ps.publication_subplots(1, 2, aspect=0.72)
g0 = (sol["load_kwh"] - sol["pv_kwh"]).clip(min=0)
labels = ["有储能", "无储能"]
axes[0].bar(labels, [met["G"], met["no_battery_purchase"]],
    color=[ps.PALETTE["primary"], ps.PALETTE["neutral"]],
    width=0.5)
axes[0].set_ylabel("全天购电量/kWh")
axes[0].set_title("全天购电量")
axes[1].bar(labels, [met["objective"], float((sol["c"] * g0).sum()
    )],
    color=[ps.PALETTE["primary"], ps.PALETTE["neutral"]],
    width=0.5)
axes[1].set_ylabel("全天购电费/元")
axes[1].set_title("全天购电费")
ps.export_figure(fig, FIG / "问题1_储能效益对比")
plt.close(fig)

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--no-figures", action="store_true",
        help="只生成数值结果和模板, 不生成或修改图片")
    args = ap.parse_args()
    sol = solve()
    met = verify(sol)
    write_template(sol)
    write_csv(sol, met)
    if not args.no_figures:
        figures(sol, met)
    print(json.dumps(met, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

from __future__ import annotations

import argparse
import json
import os
import sys
import time
from multiprocessing import Pool
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl
import utils.plot_style as ps

FIG = sl.FIG / "问题2"

_SENS_G = {}

def _sens_init(data: dict) -> None:
    _SENS_G["data"] = data

def _sens_worker(cfg: tuple) -> dict:
    kind, W, alpha, tag, horizon = cfg
    data = _SENS_G["data"]
    resid = sl.net_residual_matrix(kind, data)
    run = sl.run_year_q2(data, kind, alpha, W, resid=resid, horizon=
        horizon)
    out = {k: v for k, v in sl.year_metrics(run).items() if k != "
        daily_cash"}
    out.update({"方案": tag, "预测器": kind, "W": W, "alpha": alpha, "H
        ": horizon})
    if tag.startswith("主配置"):
        out["_run"] = run
    return out

def sensitivity(data: dict, resid_cache: dict, workers: int = 1) ->
    pd.DataFrame:
    configs = [
        ("F1", 35, 0.805, "主配置 F1-W35-a0.805-H49", 49),
        ("F1", 35, 0.800, "分位 a=0.800", 49),
        ("F1", 35, 0.810, "分位 a=0.810", 49),
        ("F1", 35, 0.805, "时段 H=48", 48),
        ("F1", 35, 0.805, "时段 H=50", 50),
        ("F1", 30, 0.805, "窗口 W=30", 49),
        ("F1", 45, 0.805, "窗口 W=45", 49),
        ("F2", 35, 0.805, "预测器 F2", 49),
        ("F0", 35, 0.805, "预测器 F0", 49),
    ]
    if workers > 1:
        with Pool(workers, initializer=_sens_init, initargs=(data,)) as
            pool:
                packed = pool.map(_sens_worker, configs)
        base = next(row.pop("_run") for row in packed if "_run" in row)
        return pd.DataFrame(packed, base)
    rows = []
    base = None
    for kind, W, alpha, tag, horizon in configs:

```

Listing 5: 问题 2_ 求解.py

```

        if kind not in resid_cache:
            resid_cache[kind] = sl.net_residual_matrix(kind, data)
        run = sl.run_year_q2(data, kind, alpha, W, resid=resid_cache[
            kind], horizon=horizon)
        m = {k: v for k, v in sl.year_metrics(run).items() if k != "
            daily_cash"}
        m.update({"方案": tag, "预测器": kind, "W": W, "alpha": alpha,
            "H": horizon})
        rows.append(m)
        if tag.startswith("主配置"):
            base = run
    return pd.DataFrame(rows), base

def figures(data: dict, run: dict, base: dict, sens: pd.DataFrame,
    day: int = 265) -> None:
    FIG.mkdir(parents=True, exist_ok=True)
    ps.apply_publication_style("zh", "report")
    t = (np.arange(mc.T) + 0.5) * mc.DT
    ticks = list(range(0, 25, 4))

    lhat, vhat = sl.predict_center("F1", day, data)
    fig, axes = ps.publication_subplots(1, 3, aspect=0.95)
    axes[0].plot(t, data["load_act"][day], color=ps.PALETTE["dark"],
        label="实测")
    axes[0].plot(t, lhat, color=ps.PALETTE["primary"], label="F1 预测")
    axes[0].set_title("负荷")
    axes[0].set_ylabel("功率/kW")
    axes[1].plot(t, data["pv_act"][day], color=ps.PALETTE["dark"],
        label="实测")
    axes[1].plot(t, vhat, color=ps.PALETTE["positive"], label="F1 预测")
    axes[1].set_title("光伏")
    axes[2].plot(t, data["load_act"][day] - data["pv_act"][day],
        color=ps.PALETTE["dark"], label="实测")
    axes[2].plot(t, lhat - vhat, color=ps.PALETTE["contrast"], label="
        F1 预测")
    axes[2].set_title("净负荷")
    for ax in axes:
        ax.set_xlabel("时刻/h")
        ax.set_xticks(ticks)
        ax.legend()
    ps.add_panel_labels(axes)
    fig.suptitle(f"问题2_中心预测与实测 ({mc.date_of(day)}) ", fontsize
        =8.5)
    ps.export_figure(fig, FIG / "问题2_中心预测对比")
    plt.close(fig)

    ltilde, vtilde, ntilde = sl.safe_trajectory(day, lhat, vhat, run["
        resid"], 0.9, 30)
    nbar = lhat - vhat
    nact = data["load_act"][day] - data["pv_act"][day]
    ncontract = run["P"][day] / mc.DT
    fig, ax = ps.publication_subplots(1, 1, aspect=0.6)
    ax.fill_between(t, nbar, ntilde, color=ps.PALETTE["sky"], alpha
        =0.35,
        label="分位风险余量")
    ax.plot(t, nbar, color=ps.PALETTE["primary"], label="中心净负荷")
    ax.plot(t, nact, color=ps.PALETTE["dark"], label="实测净负荷")
    ax.plot(t, ncontract, color=ps.PALETTE["contrast"], lw=0.9, label=
        "合同购电")
    ax.set_xlabel("时刻/h")
    ax.set_ylabel("功率/kW")

    ax.set_xticks(ticks)
    ax.legend(ncols=2)
    ax.set_title("安全轨迹、合同与实测净负荷")
    fig.suptitle(f"{mc.date_of(day)}", fontsize=8.5)
    ps.export_figure(fig, FIG / "问题2_安全轨迹与合同")
    plt.close(fig)

    days = np.arange(31, 365)
    dates = [mc.date_of(int(d)) for d in days]
    fig, axes = ps.publication_subplots(2, 1, aspect=1.15, height_
        ratios=[1.3, 1])
    axes[0].plot(dates, run["cash"][31:365], color=ps.PALETTE["primary"]
        , lw=0.8)
    axes[0].set_ylabel("日现金费/元")
    axes[0].set_title("逐日购电现金费")
    axes[1].bar(dates, run["U"][31:365].sum(axis=1), color=ps.PALETTE[
        "contrast"],
        width=0.9)
    axes[1].set_ylabel("应急购电量/kWh")
    axes[1].set_title("逐日应急购电量")
    ps.add_panel_labels(axes)
    ps.export_figure(fig, FIG / "问题2_年度费用与应急")
    plt.close(fig)

    fig, axes = ps.publication_subplots(2, 1, aspect=1.3, height_
        ratios=[1, 1])
    pos = (np.arange(mc.T) + 0.5) * mc.DT
    axes[0].bar(pos, run["X"][day], 0.15, color=ps.PALETTE["primary"],
        label="充电量")
    axes[0].bar(pos, -run["Y"][day], 0.15, color=ps.PALETTE["contrast"]
        , label="放电量")
    axes[0].axhline(0, color=ps.PALETTE["neutral"], lw=0.6)
    axes[0].set_ylabel("电量/kWh")
    axes[0].set_title("实际充放电")
    axes[0].legend()
    axes[1].plot(pos, run["S"][day], color=ps.PALETTE["secondary"],
        label="实际储电量")
    axes[1].axhline(mc.S_MIN, color=ps.PALETTE["neutral"], ls="--", lw
        =0.7, label="SOC 上下限")
    axes[1].axhline(mc.S_MAX, color=ps.PALETTE["neutral"], ls="--", lw
        =0.7)
    axes[1].set_ylabel("储电量/kWh")
    axes[1].set_xlabel("时刻/h")
    axes[1].set_title("储能状态")
    axes[1].legend()
    for ax in axes:
        ax.set_xlim(0, 24)
        ax.set_xticks(ticks)
    ps.add_panel_labels(axes)
    fig.suptitle(f"{mc.date_of(day)}", fontsize=8.5)
    ps.export_figure(fig, FIG / "问题2_重点日充放电与储能")
    plt.close(fig)

    fig, axes = ps.publication_subplots(1, 3, aspect=0.8)
    cfg = sens.set_index("方案")
    MAIN = "主配置 F1-W30-a0.9"
    groups = [
        ("分位 alpha", ["分位 a=0.8", MAIN, "分位 a=0.95"],
            ["0.8", "0.9(主)", "0.95"]),
        ("残差窗口 W", ["窗口 W=21", MAIN, "窗口 W=45"],
            ["21", "30(主)", "45"]),
    ]
    for ax, (name, keys, labs) in zip(axes[:2], groups):

```

```

ax.bar(labs, [float(cfg.loc[k, "cash_total"]) / 1e4 for k in
    keys],
    color=ps.PALETTE["primary"], width=0.55)
ax.set_title(name)
ax.set_ylabel("年度现金费/万元")
lab = ["F0", "F1", "F2"]

vals = [float(sens.loc[sens["预测器"] == k, "cash_total"].iloc[0])
    / 1e4 for k in lab]
axes[2].bar(lab, vals, color=ps.PALETTE["primary"], width=0.55)
axes[2].set_title("预测器")
axes[2].set_ylabel("年度现金费/万元")
ps.add_panel_labels(axes)
ps.export_figure(fig, FIG / "问题2_配置敏感性")
plt.close(fig)

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--main-only", action="store_true",
        help="只运行 F1-W30-alpha0.9 主配置, 不跑敏感性和绘图")
    ap.add_argument("--executor", choices=["mpc", "greedy"], default="mpc",
        help="执行器; greedy 为不读取当前槽实测的预测驱动贪心")
    ap.add_argument("--days", type=int, default=365,
        help="运行前 N 天; 小于 365 时只作 Lite 烟雾验证")
    ap.add_argument("--no-figures", action="store_true",
        help="完成全部计算与表格输出, 但不生成图件")
    ap.add_argument("--workers", type=int, default=0,
        help="完整敏感性计算的并行进程数")
    args = ap.parse_args()
    t0 = time.perf_counter()
    data = mc.load_inputs()
    sl.CSV.mkdir(parents=True, exist_ok=True)

    if args.main_only:
        resid = sl.net_residual_matrix("F1", data)
        run = sl.run_year_q2(data, "F1", 0.805, 35, resid=resid,
            horizon=49, n_days=args.days, executor=args.executor)
        gates = sl.check_run(run)
        with open(sl.CSV / "问题2_门禁.json", "w", encoding="utf-8") as fh:
            json.dump(gates, fh, ensure_ascii=False, indent=2)
        daily = sl.monthly_summary(run)
        daily.to_csv(sl.CSV / "问题2_逐日汇总.csv", index=False,
            encoding="utf-8-sig")
        if args.days == 365:
            sl.write_workbook("result2.xlsx", "result2.xlsx", {
                "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run),
                "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run),
                "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws,
                    run),
            })
            np.savez_compressed(sl.CSV / "q2_bundle.npz",
                **{k: run[k] for k in
                    ("P", "X", "Y", "U", "W", "V", "S",
                     "cash", "plan_x", "plan_y", "plan_s_
                     next")})
            metrics = {k: v for k, v in sl.year_metrics(run).items()
                if k != "daily_cash"}
            print(json.dumps(dict(gates=gates, **metrics), ensure_ascii=
                False,
                    indent=2))
            print("elapsed_s", round(time.perf_counter() - t0, 1))

    return

err = pd.DataFrame([dict(预测器=k, **sl.point_error_metrics(k,
    data))
        for k in ("F0", "F1", "F2")])
err.to_csv(sl.CSV / "问题2_预测误差对比.csv", index=False, encoding=
    "utf-8-sig")

resid_cache = {k: sl.net_residual_matrix(k, data) for k in ("F0",
    "F1", "F2")}
workers = args.workers or min(4, max(1, (os.cpu_count() or 4) - 2))
sens, run = sensitivity(data, resid_cache, workers=workers)
sens.to_csv(sl.CSV / "问题2_配置敏感性.csv", index=False, encoding=
    "utf-8-sig")

pf = sl.perfect_foresight_q2(data)
nb = sl.run_year_q2(data, no_battery=True)
ref = pd.DataFrame([
    dict(方案="主配置", **{k: v for k, v in sl.year_metrics(run).
        items()
            if k != "daily_cash"}),
    dict(方案="理想信息参照", **{k: v for k, v in sl.year_metrics(pf
        ).items()
            if k != "daily_cash"}),
    dict(方案="无储能", **{k: v for k, v in sl.year_metrics(nb).
        items()
            if k != "daily_cash"}),
])
ref.to_csv(sl.CSV / "问题2_参照方案.csv", index=False, encoding="
    utf-8-sig")

gates = sl.check_run(run)
with open(sl.CSV / "问题2_门禁.json", "w", encoding="utf-8") as fh
    :
        json.dump(gates, fh, ensure_ascii=False, indent=2)

day_idx = [sl.DAY_INDEX[d] for d in sl.REP_DATES]
t1 = sl.slot_table(run, data["price1"], day_idx)
t1.to_csv(sl.CSV / "问题2_表1_指定时段.csv", index=False, encoding=
    "utf-8-sig")
t2 = sl.energy_block_table(run, day_idx)
t2.to_csv(sl.CSV / "问题2_表2_四小时聚合.csv", index=False,
    encoding="utf-8-sig")
t3 = sl.emergency_table(run, day_idx)
t3.to_csv(sl.CSV / "问题2_表3_紧急购电.csv", index=False, encoding=
    "utf-8-sig")

daily = sl.monthly_summary(run)
daily.to_csv(sl.CSV / "问题2_逐日汇总.csv", index=False, encoding="
    utf-8-sig")

sl.write_workbook("result2.xlsx", "result2.xlsx", {
    "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run),
    "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run),
    "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws, run),
})

if not args.no_figures:
    figures(data, run, pf, sens)
print(json.dumps(dict(gates=gates, **{k: v for k, v in sl.year_
    metrics(run).items()

```

```

        if k != "daily_cash")), ensure_
            ascii=False, indent=2))
    print("elapsed_s", round(time.perf_counter() - t0, 1))

if __name__ == "__main__":
    main()

```

Listing 6: 问题 3_ 求解.py

```

from __future__ import annotations

import argparse
import json
import os
import time
from multiprocessing import Pool

import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl

KEYS = ("P", "Q", "X", "Y", "U", "W", "V", "S", "cash", "cash_plan",
        "cash_contract", "plan_x", "plan_y", "plan_s_next")
FULL_SET = (0, 1, 2, 3)

def _cfg(tag, mode, H, allowed, updates, note, keep=False):
    return {"tag": tag, "mode": mode, "H": H, "allowed": tuple(allowed),
            "updates": tuple(updates), "note": note, "keep": keep}

def build_configs() -> list:
    cfigs = [
        _cfg("main", "stratified", 6, FULL_SET, (1, 2, 3),
            "主配置: 分层储备 + H=6 + 三次调整, keep=True),
        _cfg("uniform", "uniform", 6, FULL_SET, (1, 2, 3),
            "对照: 统一逐槽分位储备 + H=6", keep=True),
        _cfg("h72", "stratified", 72, FULL_SET, (1, 2, 3),
            "对照: 分层储备 + H=72", keep=True),
    ]
    for allowed, updates, note in [
        ((0,), ()), "消融 空集: 仅 0:00 预报, 仍共享每 10 min 重算机会",
        ((0, 1), (1,)), "消融 {6}",
        ((0, 2), (2,)), "消融 {12}",
        ((0, 3), (3,)), "消融 {18}",
        ((0, 1, 2), (1, 2)), "消融 {6,12}",
        ((0, 1, 3), (1, 3)), "消融 {6,18}",
        ((0, 2, 3), (2, 3)), "消融 {12,18}",
    ]:
        tag = "abl_" + ("_" + ".".join(str(h) for h in updates) or "none")
        cfigs.append(_cfg(tag, "stratified", 6, allowed, updates, note,
            keep=True))

    return cfigs

_G = {}

```

```

def _init(data, prof, n_days):
    _G["data"] = data
    _G["prof"] = prof
    _G["n_days"] = n_days

def _coverage(data, prof, allowed, mode, alpha, W, d0=31, d1=365):
    resid, pv_ver = sl.q3_net_residual(data, prof, allowed, "F1", "archive")
    L, V = data["load_act"], data["pv_act"]
    R = np.empty((d1 - d0, mc.T))
    for i, d in enumerate(range(d0, d1)):
        lhat, _ = sl.predict_center("F1", d, data)
        res_kw, _ = sl.q3_reserve(d, resid, alpha, W, mode, allowed)
        R[i] = (L[d] - V[d]) - (lhat - pv_ver[d] + res_kw)
    resid_f1 = resid[d0:d1]
    return {
        "Ncov": float((R <= 0.0).mean()),
        "pinball": float((R * (alpha - (R < 0.0))).mean()),
        "resid_MAE_kw": float(np.abs(resid_f1).mean()),
        "resid_RMSE_kw": float(np.sqrt((resid_f1 ** 2).mean())),
    }

def _worker(cfg):
    data, prof, n_days = _G["data"], _G["prof"], _G["n_days"]
    run = sl.run_year_q34(data, prof, "F1", 0.7725, 35, cfg["mode"],
        cfg["H"],
        cfg["allowed"], cfg["updates"], "archive", "fixed",
        "median30", "fixed", 5.0, n_days)
    out = dict(cfg)
    if n_days > 31:
        out["metrics"] = sl.year_metrics_q34(run, 31)
        out["coverage"] = _coverage(data, prof, cfg["allowed"], cfg["mode"],
            0.7725, 35)
        out["gates"] = sl.check_run_q34(run, 31)
    if cfg["keep"]:
        out["arrays"] = {k: run[k] for k in KEYS}
        out["stats"] = run["stats"]
        out["S_end"] = run["S_end"]
        out["resid"] = run["resid"]
        out["pv_ver"] = run["pv_ver"]
    return out

def slot_table_q3(run, price_day, day_indices):
    rows = []
    for d in day_indices:
        c = price_day if np.ndim(price_day) == 1 else price_day[d]
        for t in sl.REP_SLOTS:
            rows.append({
                "日期": str(mc.date_of(d)),
                "时段": "%s-%s" % (mc.slot_label_start(t), mc.slot_label_end(t)),
                "电价_元每kWh": c[t],
                "负荷_kw": float(run["load_kw"][d][t]),
                "光伏实际_kw": float(run["pv_kw"][d][t]),
                "原计划合同_p_kWh": run["P"][d][t],
                "最终合同_q_kWh": run["Q"][d][t],
                "实际购电_kw": run["Q"][d][t] - run["W"][d][t] + run["U"][d][t],
                "应急购电_u_kWh": run["U"][d][t],
            })

```

```

        "充电量_kWh": run["X"][d][t],
        "放电量_kWh": run["Y"][d][t],
        "槽末储电量_kWh": run["S"][d][t],
        "合同调整量_kWh": run["Q"][d][t] - run["P"][d][t],
    })
    return pd.DataFrame(rows)

def ablation_table(records):
    by_tag = {r["tag"]: r for r in records if "metrics" in r}
    base = by_tag["abl_none"]["metrics"]
    rows = []
    for rec in records:
        if "metrics" not in rec or not (rec["tag"].startswith("abl_")
                                         or rec["tag"] == "main"):
            continue
        m = rec["metrics"]
        dn = int(m["updates"] - base["updates"])
        dc = float(base["cash_total"] - m["cash_total"])
        rows.append({
            "获准集合S": ("{6,12,18}" if rec["tag"] == "main" else
                          ("{" if rec["tag"] == "abl_none" else
                           rec["note"].replace("消融 ", ""))),
            "tag": rec["tag"],
            "允许发布时刻": ", ".join(str(sl.ISSUE_HOURS[r]) for r in rec
                                       ["allowed"]),
            "现金总费_元": m["cash_total"],
            "计划购电量_kWh": m["plan_kwh"],
            "最终合同量_kWh": m["contract_kwh"],
            "调整事件数": m["updates"],
            "dn_相对空集": dn,
            "应急购电量_kWh": m["emergency_kwh"],
            "应急槽数": m["emergency_slots"],
            "弃购量_kWh": m["unused_contract_kwh"],
            "期末储电量_kWh": m["S_end"],
            "相对空集节约_元": dc,
            "kappa_star_元每次": (dc / dn if dn > 0 else float("nan")),
        })
    df = pd.DataFrame(rows)
    cash = {r["tag"]: (r["metrics"]["cash_total"], r["allowed"])
             for r in records if "metrics" in r and
             (r["tag"].startswith("abl_") or r["tag"] == "main")}
    for r in (1, 2, 3):
        diffs = []
        for tag, (c_s, allowed) in cash.items():
            if r in allowed:
                continue
            enlarged = tuple(sorted((set(allowed) | {r}) - {0}))
            tag2 = "main" if enlarged == (1, 2, 3) else (
                "abl_" + ("_".join(str(h) for h in enlarged) or "none"))
            if tag2 in cash:
                diffs.append(c_s - cash[tag2][0])
        df["版本%d时点_平均边际收益_元" % sl.ISSUE_HOURS[r]] = (
            float(np.mean(diffs)) if diffs else float("nan"))
    return df

def lead_scale_table(data, prof, offset=31):
    rows = []
    for r in range(sl.N_ISSUE):
        for h in range(1, 25):
            pos = sl.ISSUE_SLOTS[r] + 6 * h
            if pos > mc.T:
                continue
            err = np.array([data["pv_act"][d, pos - 1] - prof[d, r, pos]
                           for d in range(offset, 365)])
            rows.append({
                "发布日期_h": sl.ISSUE_HOURS[r], "提前期_h": h,
                "MAE_kW": float(np.abs(err).mean()),
                "RMSE_kW": float(np.sqrt((err ** 2).mean())),
                "Bias_kW": float(err.mean()),
                "q90_kW": mc.empirical_quantile(err, 0.9),
                "尺度_s_kW": float(np.sqrt((err ** 2).mean())),
            })
    df = pd.DataFrame(rows)
    agg = df.groupby("提前期_h").agg(
        MAE_kW=("MAE_kW", "mean"), RMSE_kW=("RMSE_kW", "mean"),
        Bias_kW=("Bias_kW", "mean"), q90_kW=("q90_kW", "mean")).reset_index()
    return df, agg

def monthly_table(run, offset=31):
    rows = []
    for d in range(offset, run["P"].shape[0]):
        dt = mc.date_of(d)
        rows.append({
            "日期": dt, "月份": dt.strftime("%Y-%m"),
            "现金费_元": run["cash"][d],
            "计划购电量_kWh": run["P"][d].sum(),
            "最终合同量_kWh": run["Q"][d].sum(),
            "调整量_kWh": (run["Q"][d] - run["P"][d]).sum(),
            "应急购电量_kWh": run["U"][d].sum(),
            "充电量_kWh": run["X"][d].sum(),
            "放电量_kWh": run["Y"][d].sum(),
            "弃购量_kWh": run["W"][d].sum(),
            "弃光量_kWh": float((run["pv_kwh"][d] - run["V"][d]).sum()),
            "日末储电量_kWh": run["S"][d][-1],
        })
    return pd.DataFrame(rows)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--days", type=int, default=365)
    ap.add_argument("--workers", type=int, default=0)
    ap.add_argument("--main-only", action="store_true")
    args = ap.parse_args()
    t0 = time.perf_counter()
    mc.RESULT_DIR.mkdir(parents=True, exist_ok=True)
    sl.CSV.mkdir(parents=True, exist_ok=True)

    data = mc.load_inputs()
    prof = sl.build_pv_archive(data, 35)
    cfigs = build_configs()
    if args.main_only:
        cfigs = cfigs[:1]
    n_cpu = os.cpu_count() or 4
    workers = args.workers or min(len(cfigs), max(1, n_cpu - 4))
    print("[Q3] configs=%d workers=%d days=%d" % (len(cfigs), workers,
                                                    args.days),
          flush=True)

    if workers <= 1:
        _init(data, prof, args.days)
        records = [_worker(c) for c in cfigs]
    else:

```

```

        with Pool(workers, initializer=_init, initargs=(data, prof,
            args.days)) as pool:
            records = pool.map(_worker, cfgs)
        print("[Q3] runs done in %.0fs" % (time.perf_counter() - t0),
            flush=True)

specs = []
for rec in records:
    if "metrics" not in rec:
        continue
    row = {"方案": rec["note"], "tag": rec["tag"], "mode": rec["
        mode"],
        "H": rec["H"], "updates": rec["updates"]}
    row.update({k: v for k, v in rec["metrics"].items() if k != "
        daily_cash"})
    row.update({"cov_" + k: v for k, v in rec["coverage"].items()})
    specs.append(row)
pd.DataFrame(specs).to_csv(sl.CSV / "问题3_方案对比.csv", index=
    False,
        encoding="utf-8-sig")

if any(r["tag"].startswith("abl_") for r in records):
    ablation_table(records).to_csv(sl.CSV / "问题3_信息消融.csv",
        index=False,
            encoding="utf-8-sig")

lead_df, lead_agg = lead_scale_table(data, prof)
lead_df.to_csv(sl.CSV / "问题3_预见期误差_分来源.csv", index=False,
    encoding="utf-8-sig")
lead_agg.to_csv(sl.CSV / "问题3_预见期尺度.csv", index=False,
    encoding="utf-8-sig")

main_rec = next(r for r in records if r["tag"] == "main")
run = dict(main_rec["arrays"])
run["stats"] = main_rec["stats"]
run["S_end"] = main_rec["S_end"]
run["resid"] = main_rec["resid"]
run["pv_ver"] = main_rec["pv_ver"]
run["load_kwh"] = data["load_act"] * mc.DT
run["pv_kwh"] = data["pv_act"] * mc.DT
run["load_kw"] = data["load_act"]
run["pv_kw"] = data["pv_act"]
run["price_settle"] = np.tile(data["price1"], (run["P"].shape[0],
    1))

n_run = run["P"].shape[0]
day_idx = [sl.DAY_INDEX[d] for d in sl.REP_DATES if sl.DAY_INDEX[d]
    ] < n_run
if day_idx:
    slot_table_q3(run, data["price1"], day_idx).to_csv(
        sl.CSV / "问题3_表1_指定时段.csv", index=False, encoding="
            utf-8-sig")
    sl.energy_block_table(run, day_idx).to_csv(
        sl.CSV / "问题3_表2_四小时聚合.csv", index=False, encoding="
            utf-8-sig")
    sl.emergency_table(run, day_idx).to_csv(
        sl.CSV / "问题3_表3_紧急购电.csv", index=False, encoding="
            utf-8-sig")
monthly_table(run).to_csv(sl.CSV / "问题3_逐日汇总.csv", index=
    False,
        encoding="utf-8-sig")

gates = dict(main_rec["gates"])
gates["coverage"] = main_rec["coverage"]
gates["all_numeric_ok"] = bool(

gates["balance_max_kWh"] <= 1e-6 and gates["overlap_max_kWh"]
    <= 1e-5
and gates["charge_emergency_max_kWh"] <= 1e-5
and max(gates[k] for k in ("bound_p", "bound_q", "bound_x", "
    bound_y",
        "bound_v", "bound_u", "bound_w", "bound_
            S")) <= 1e-7
and gates["settle_flow_max_yuan"] <= 1e-5)
with open(sl.CSV / "问题3_门禁.json", "w", encoding="utf-8") as fh
    :
        json.dump(gates, fh, ensure_ascii=False, indent=2, default=
            float)

sl.write_workbook("result3.xlsx", "result3.xlsx", {
    "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run, key="P",
        fee_key="cash_plan"),
    "调整购电量": lambda ws: sl._fill_plan_sheet(ws, run, key="Q",
        fee_key="cash_contract"),
    "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run),
    "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws, run),
})

np.savez_compressed(sl.CSV / "q3_bundle.npz",
    **{"%s_%s" % (r["tag"], k): r["arrays"][k]
        for r in records if "arrays" in r for k in
            KEYS})

def _jsonable(rec):
    out = {}
    for k, v in rec.items():
        if k in ("arrays", "resid", "pv_ver", "stats"):
            continue
        if k == "metrics":
            v = {kk: vv for kk, vv in v.items() if kk != "daily_cash"}
        out[k] = v
    return out

meta = {"days": args.days, "configs": [_jsonable(r) for r in
    records]}
with open(sl.CSV / "q3_metrics.json", "w", encoding="utf-8") as fh
    :
        json.dump(meta, fh, ensure_ascii=False, indent=2, default=float
            )

summary = {k: v for k, v in main_rec["metrics"].items() if k != "
    daily_cash"}
print(json.dumps({"summary": summary, "gates": gates}, ensure_
    ascii=False,
        indent=2, default=float))
print("elapsed_s", round(time.perf_counter() - t0, 1))

if __name__ == "__main__":
    main()

```

Listing 7: 问题 4_ 求解.py

```

from __future__ import annotations

import argparse
import json
import os
import time
from multiprocessing import Pool

```

```

import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl

KEYS = ("P", "Q", "X", "Y", "U", "W", "V", "S", "cash", "cash_plan",
        "cash_contract", "plan_x", "plan_y", "plan_s_next")
BASES = ("prev_day", "mean7", "mean30", "median30")
BASE_LABEL = {"prev_day": "昨日同槽", "mean7": "近7日均值",
              "mean30": "近30日均值", "median30": "近30日中位数",
              "oracle": "0时已知真价（理想信息）"}
DEFAULT_BASE = "median30"
FULL_SET = (0, 1, 2, 3)
UPDATES2 = (1, 2, 3)
SPIKE_Q = 0.75
OFFSET = 31
NTOT = 365

def build_configs():
    cfigs = []
    for base in BASES:
        cfigs.append({"tag": "Q4-2_" + base, "mode_note": "模式一（固定合同）",
                     "base": base, "updates": (), "pv": "archive",
                     "price_mode": "forecast", "keep": base == DEFAULT_BASE})
    for base in BASES:
        cfigs.append({"tag": "Q4-3_" + base, "mode_note": "模式二（可调合同）",
                     "base": base, "updates": UPDATES2, "pv": "archive",
                     "price_mode": "forecast", "keep": base == DEFAULT_BASE})
    cfigs.append({"tag": "Q4-2_oracle", "mode_note": "模式一（固定合同）",
                  "base": "oracle", "updates": (), "pv": "archive",
                  "price_mode": "oracle", "keep": True})
    cfigs.append({"tag": "Q4-3_oracle", "mode_note": "模式二（可调合同）",
                  "base": "oracle", "updates": UPDATES2, "pv": "archive",
                  "price_mode": "oracle", "keep": True})
    cfigs.append({"tag": "Q4-2_center_" + DEFAULT_BASE,
                  "mode_note": "模式一（固定合同）", "base": DEFAULT_BASE,
                  "updates": (), "pv": "center", "price_mode": "forecast",
                  "keep": True})
    return cfigs

_G = {}

def _init(data, prof, n_days=NTOT):
    _G["data"] = data
    _G["prof"] = prof
    _G["n_days"] = n_days

def _worker(cfg):
    data, prof = _G["data"], _G["prof"]
    n_days = _G["n_days"]
    run = sl.run_year_q34(data, prof, "F1", 0.7725, 35, "stratified",
                          6, FULL_SET,
                          cfg["updates"], cfg["pv"], cfg["price_mode"],

                                cfg["base"], "actual", 5.0, n_days)
    out = dict(cfg)
    out["metrics"] = sl.year_metrics_q34(run, OFFSET)
    out["gates"] = sl.check_run_q34(run, OFFSET)
    if cfg["keep"]:
        out["arrays"] = {k: run[k] for k in KEYS}
        out["stats"] = run["stats"]
        out["S_end"] = run["S_end"]
    return out

def price_error_table(data, offset=OFFSET, end=NTOT):
    A = data["price_act"]
    days = list(range(offset, end))
    spike = np.vstack([A[d] >= np.quantile(A[d], SPIKE_Q) for d in
                       days])
    rows = []
    for base in BASES:
        e0, e12 = [], []
        for d in days:
            bp = mc.price_forecast_0h(d, A, data["price1"], base)
            e0.append(bp - A[d])
            e12.append(mc.price_intraday_correct(bp, A[d], 72) - A[d])
        E0 = np.vstack(e0)
        E12 = np.vstack(e12)
        rows.append({
            "价格基线": base, "基线说明": BASE_LABEL[base],
            "MAE0_元每kWh": float(np.abs(E0).mean()),
            "RMSE0_元每kWh": float(np.sqrt((E0 ** 2).mean())),
            "Bias0_元每kWh": float(E0.mean()),
            "尖峰MAE0_元每kWh": float(np.abs(E0[spike]).mean()),
            "MAE12_元每kWh": float(np.abs(E12).mean()),
            "尖峰MAE12_元每kWh": float(np.abs(E12[spike]).mean()),
            "最大绝对误差0": float(np.abs(E0).max()),
        })
    return pd.DataFrame(rows)

def coverage_of(data, prof, alpha=0.7725, W=35, mode="stratified", d0
                =OFFSET,
                d1=NTOT):
    resid, pv_ver = sl.q3_net_residual(data, prof, FULL_SET, "F1", "archive")
    L, V = data["load_act"], data["pv_act"]
    R = np.empty((d1 - d0, mc.T))
    for i, d in enumerate(range(d0, d1)):
        lhat, _ = sl.predict_center("F1", d, data)
        res_kw, _ = sl.q3_reserve(d, resid, alpha, W, mode, FULL_SET)
        R[i] = (L[d] - V[d]) - (lhat - pv_ver[d] + res_kw)
    return {"Ncov": float((R <= 0.0).mean()),
            "pinball": float((R * (alpha - (R < 0.0))).mean()),
            "resid_MAE_kW": float(np.abs(resid[d0:d1]).mean())}

def slot_table(run, price_act, day_indices):
    rows = []
    for d in day_indices:
        c = price_act[d]
        for t in sl.REP_SLOTS:
            rows.append({

```

```

        "日期": str(mc.date_of(d)),
        "时段": "%s-%s" % (mc.slot_label_start(t), mc.slot_label_end(t)),
        "实际结算电价_元每kWh": float(c[t]),
        "负荷_kW": float(run["load_kw"][d][t]),
        "光伏实际_kW": float(run["pv_kw"][d][t]),
        "原计划合同_p_kWh": float(run["P"][d][t]),
        "最终合同_q_kWh": float(run["Q"][d][t]),
        "实际购电_kWh": float(run["Q"][d][t] - run["W"][d][t] + run["U"][d][t]),
        "应急购电_u_kWh": float(run["U"][d][t]),
        "充电量_kWh": float(run["X"][d][t]),
        "放电量_kWh": float(run["Y"][d][t]),
        "槽末储电量_kWh": float(run["S"][d][t]),
        "合同调整量_kWh": float(run["Q"][d][t] - run["P"][d][t])
    },
    )
    return pd.DataFrame(rows)

def daily_table(run, offset=OFFSET):
    rows = []
    for d in range(offset, run["P"].shape[0]):
        dt = mc.date_of(d)
        rows.append({
            "日期": dt, "月份": dt.strftime("%Y-%m"),
            "现金费_元": run["cash"][d],
            "计划购电量_kWh": run["P"][d].sum(),
            "最终合同量_kWh": run["Q"][d].sum(),
            "调整量_kWh": (run["Q"][d] - run["P"][d]).sum(),
            "应急购电量_kWh": run["U"][d].sum(),
            "充电量_kWh": run["X"][d].sum(),
            "放电量_kWh": run["Y"][d].sum(),
            "弃购量_kWh": run["W"][d].sum(),
            "弃光量_kWh": (run["pv_kwh"][d] - run["V"][d]).sum(),
            "日末储电量_kWh": run["S"][d][-1])
        })
    return pd.DataFrame(rows)

def _jsonable(rec):
    out = {}
    for k, v in rec.items():
        if k in ("arrays", "stats"):
            continue
        if k == "metrics":
            v = {kk: vv for kk, vv in v.items() if kk != "daily_cash"}
        out[k] = v
    return out

def _numeric_ok(g: dict) -> bool:
    return bool(
        g["balance_max_kWh"] <= 1e-6 and g["overlap_max_kWh"] <= 1e-5
        and g["charge_emergency_max_kWh"] <= 1e-5
        and max(g[k] for k in ("bound_p", "bound_q", "bound_x", "bound_y",
                                "bound_v", "bound_u", "bound_w", "bound_S"))
            <= 1e-7
        and g["soc_chain_max_kWh"] <= 1e-6
        and g["soc_end_mismatch_kWh"] <= 1e-6
        and g["settle_flow_max_yuan"] <= 1e-5
    )

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--workers", type=int, default=0)
    ap.add_argument("--days", type=int, default=NTOT,
                    help="运行前 N 天; 小于 365 时只作 Lite 烟雾验证")
    ap.add_argument("--main-only", action="store_true",
                    help="只运行 Q4-2/Q4-3 的 median30 主配置")
    args = ap.parse_args()
    if not OFFSET < args.days <= NTOT:
        ap.error("--days 必须在 32..365 之间")
    t0 = time.perf_counter()
    sl.CSV.mkdir(parents=True, exist_ok=True)

    data = mc.load_inputs()
    prof = sl.build_pv_archive(data, 35)
    cfigs = build_configs()
    if args.main_only:
        keep_tags = {"Q4-2_" + DEFAULT_BASE, "Q4-3_" + DEFAULT_BASE}
        cfigs = [c for c in cfigs if c["tag"] in keep_tags]
    workers = args.workers or min(len(cfigs), max(1, (os.cpu_count() or 4) - 4))
    print("[Q4] configs=%d workers=%d" % (len(cfigs), workers), flush=True)

    if workers <= 1:
        _init(data, prof, args.days)
        records = [_worker(c) for c in cfigs]
    else:
        with Pool(workers, initializer=_init, initargs=(data, prof, args.days)) as pool:
            records = pool.map(_worker, cfigs)
    print("[Q4] runs done in %.0fs" % (time.perf_counter() - t0), flush=True)

    err = price_error_table(data, end=args.days)
    err.to_csv(sl.CSV / "问题4_价格基线误差.csv", index=False, encoding="utf-8-sig")
    err_map = dict(zip(err["价格基线"], err["MAEO_元每kWh"]))
    cov = coverage_of(data, prof, d1=args.days)

    rows = []
    for rec in records:
        m = rec["metrics"]
        row = {"方案": rec["mode_note"], "tag": rec["tag"],
              "价格基线": BASE_LABEL.get(rec["base"], rec["base"]),
              "光伏预报源": rec["pv"], "定价信息": rec["price_mode"]}
        row.update({k: v for k, v in m.items() if k != "daily_cash"})
        row["价格MAEO_元每kWh"] = float(
            err_map.get(rec["base"], 0.0 if rec["base"] == "oracle" else np.nan))
        row["安全轨迹覆盖率"] = cov["Ncov"]
        rows.append(row)
    cmp_df = pd.DataFrame(rows)
    cmp_df.to_csv(sl.CSV / "问题4_方案对比.csv", index=False, encoding="utf-8-sig")

    gate_report = {r["tag"]: dict(r["gates"], coverage_Ncov=cov["Ncov"])
                   for r in records}
    gate_report["_all_numeric_ok"] = bool(
        all(v["all_numeric_ok"] for k, v in gate_report.items()
            if not k.startswith("_")))
    with open(sl.CSV / "问题4_门禁.json", "w", encoding="utf-8") as fh:
        json.dump(gate_report, fh, ensure_ascii=False, indent=2,

```

```

        default=float)

by_tag = {r["tag"]: r for r in records}
prim2 = by_tag["Q4-2_" + DEFAULT_BASE]
prim3 = by_tag["Q4-3_" + DEFAULT_BASE]
common = {"load_kwh": data["load_act"] * mc.DT,
          "pv_kwh": data["pv_act"] * mc.DT,
          "load_kw": data["load_act"], "pv_kw": data["pv_act"],
          "cash_plan": None, "cash_contract": None}
day_idx = [sl.DAY_INDEX[d] for d in sl.REP_DATES
            if sl.DAY_INDEX[d] < args.days]
price_act = data["price_act"]
for prim, key in ((prim2, "问题4-2"), (prim3, "问题4-3")):
    run = dict(prim["arrays"])
    for k, v in common.items():
        if v is not None:
            run[k] = v
    slot_table(run, price_act, day_idx).to_csv(
        sl.CSV / (key + "_表1_指定时段.csv"), index=False,
        encoding="utf-8-sig")
    sl.energy_block_table(run, day_idx).to_csv(
        sl.CSV / (key + "_表2_四小时聚合.csv"), index=False,
        encoding="utf-8-sig")
    sl.emergency_table(run, day_idx).to_csv(
        sl.CSV / (key + "_表3_紧急购电.csv"), index=False,
        encoding="utf-8-sig")
    daily_table(run).to_csv(sl.CSV / (key + "_逐日汇总.csv"), index
                           =False,
                           encoding="utf-8-sig")

run2 = dict(prim2["arrays"])
run2.update({k: v for k, v in common.items() if v is not None})
run3 = dict(prim3["arrays"])
run3.update({k: v for k, v in common.items() if v is not None})
if args.days == NTOT:
    sl.write_workbook("result4-2.xlsx", "result4-2.xlsx", {
        "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run2, key="
            P",
            fee_key="cash_plan"),
        "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run2),
        "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws, run2),
    })
    sl.write_workbook("result4-3.xlsx", "result4-3.xlsx", {
        "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run3, key="
            P",
            fee_key="cash_plan"),
        "调整购电量": lambda ws: sl._fill_plan_sheet(ws, run3, key="
            Q",
            fee_key="cash_contract"),
        "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run3),
        "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws, run3),
    })

np.savez_compressed(sl.CSV / "q4_bundle.npz",
                    **{("%s_%s" % (r["tag"], k)): r["arrays"][k]
                       for r in records if "arrays" in r for k in
                       KEYS})
with open(sl.CSV / "q4_metrics.json", "w", encoding="utf-8") as fh:
    :
    json.dump({"configs": [_jsonable(r) for r in records],
              "price_error": err.to_dict("records"), "coverage":
              cov},
              fh, ensure_ascii=False, indent=2, default=float)

print(cmp_dif[["tag", "cash_total", "emergency_kwh", "updates"

```

```

        ]].to_string())
summary = {"main_base": DEFAULT_BASE,
          "Q4-2_cash": float(prim2["metrics"]["cash_total"]),
          "Q4-3_cash": float(prim3["metrics"]["cash_total"]),
          "Q4-2_emergency_kWh": float(prim2["metrics"]["emergency_
            kwh"]),
          "Q4-3_emergency_kWh": float(prim3["metrics"]["emergency_
            kwh"]),
          "all_numeric_ok": gate_report["all_numeric_ok"]}
print(json.dumps(summary, ensure_ascii=False, indent=2, default=
float))
print("elapsed_s", round(time.perf_counter() - t0, 1))

if __name__ == "__main__":
    main()

```

Listing 8: 问题 3_ 其他时刻增量.py

```

from __future__ import annotations

import argparse
import json
import os
from pathlib import Path
from multiprocessing import Pool

os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")
import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "results" / "csv"
KEYS = ("P", "Q", "X", "Y", "U", "W", "V", "S", "cash", "cash_plan",
        "cash_contract", "plan_x", "plan_y", "plan_s_next")

def make_proxy_profiles(data, W=35):
    old_slots, old_hours, old_n = sl.ISSUE_SLOTS, sl.ISSUE_HOURS, sl.N
    _ISSUE
    base = sl.build_pv_archive(data, W)
    candidate_hours = (3, 9, 15, 21)
    candidate_slots = tuple(6 * h for h in candidate_hours)

    source = (0, 1, 2, 3)
    proxy = np.stack([base[:, r, :] for r in source], axis=1)
    prof = np.concatenate([base, proxy], axis=1)
    sl.ISSUE_SLOTS = old_slots + candidate_slots
    sl.ISSUE_HOURS = old_hours + candidate_hours
    sl.N_ISSUE = old_n + 4
    return prof, old_slots, old_hours, old_n

def restore(state):
    sl.ISSUE_SLOTS, sl.ISSUE_HOURS, sl.N_ISSUE = state

def run_case(data, prof, allowed, updates, n_days):

```

```

run = sl.run_year_q34(data, prof, "F1", 0.7725, 35, "stratified",
6,
allowed, updates, "archive", "fixed", "median30",
"fixed", 5.0, n_days)
return run, sl.year_metrics_q34(run, 31)

def load_existing_baseline(n_days):
    path = OUT / "q3_bundle.npz"
    if not path.exists():
        raise FileNotFoundError("缺少已有问题三结果: %s" % path)
    with np.load(path) as z:
        cash = np.asarray(z["main_cash"][:n_days], dtype=float)
        emergency = np.asarray(z["main_U"][:n_days], dtype=float)
    if n_days <= 31:
        raise ValueError("--days 必须大于 31")
    return {
        "daily_cash": cash[31:],
        "cash_total": float(cash[31:].sum()),
        "emergency_kwh": float(emergency[31:].sum()),
    }

def candidate_worker(job):
    hour, r, n_days = job
    data = mc.load_inputs()
    prof, *state = make_proxy_profiles(data, 35)
    try:
        _, metrics = run_case(
            data, prof, (0, 1, 2, 3, r), (1, 2, 3, r), n_days
        )
        return hour, metrics
    finally:
        restore(state)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--days", type=int, default=365)
    ap.add_argument("--workers", type=int, default=4)
    args = ap.parse_args()
    OUT.mkdir(parents=True, exist_ok=True)
    base_m = load_existing_baseline(args.days)
    jobs = [(hour, r, args.days)
            for hour, r in zip((3, 9, 15, 21), (4, 5, 6, 7))]
    workers = max(1, min(args.workers, len(jobs)))
    print("[Q3-extra] C++ HiGHS scenarios=%d workers=%d days=%d"
          % (len(jobs), workers, args.days), flush=True)
    if workers == 1:
        results = [candidate_worker(job) for job in jobs]
    else:
        with Pool(workers) as pool:
            results = pool.map(candidate_worker, jobs)

    rows = []
    for hour, m in results:
        daily_base = np.asarray(base_m["daily_cash"])
        daily_new = np.asarray(m["daily_cash"])
        saving = daily_base - daily_new
        rows.append({
            "候选时刻_h": hour,
            "对照方案": "{0,6,12,18}",
            "增强方案": "{0,6,12,18,%d}" % hour,
            "代理来源": "此前最近官方版本",
            "delta_n": 1,
            "现金总费_对照_元": base_m["cash_total"],
            "现金总费_增强_元": m["cash_total"],
            "全年节省_元": float(base_m["cash_total"] - m["cash_total"]),
            "kappa_star_元每次": float(saving.mean()),
            "日均节省_元": float(saving.mean()),
            "节省为正天数": int((saving > 1e-8).sum()),
            "验证天数": int(saving.size),
            "多数验证日节省为正": bool((saving > 1e-8).sum() > saving.size / 2),
            "应急购电_对照_kWh": base_m["emergency_kwh"],
            "应急购电_增强_kWh": m["emergency_kwh"],
            "调整事件数_增强": m["updates"],
            "结论": (
                "数值正收益, 但夜间无光伏, 不建议实际引入" if hour ==
                21 else
                "代理情景下可谨慎考虑" if (
                    base_m["cash_total"] > m["cash_total"] and
                    (saving > 1e-8).sum() > saving.size / 2
                ) else "代理情景下不建议引入"
            ),
        })
    df = pd.DataFrame(rows).sort_values("候选时刻_h")
    df.to_csv(OUT / "问题3_其他时刻单次边际测试.csv", index=False,
             encoding="utf-8-sig")
    meta = {
        "status": "proxy_scenario_only",
        "engine": "SciPy linprog + compiled C++ HiGHS, scenarios
        parallelized",
        "workers": workers,
        "candidate_hours": [3, 9, 15, 21],
        "proxy_rule": "each candidate uses the corrected profile of the
        previous official issue time",
        "base_metrics": {k: v for k, v in base_m.items() if k != "daily_cash"},
        "results": rows,
    }
    with open(OUT / "问题3_其他时刻单次边际测试.json", "w", encoding="utf-8") as f:
        json.dump(meta, f, ensure_ascii=False, indent=2, default=float)
    print(df.to_string(index=False))
    print("output:", OUT / "问题3_其他时刻单次边际测试.csv")

if __name__ == "__main__":
    main()

from __future__ import annotations

import argparse
import json
import math
import os
import time
from multiprocessing import Pool

import pandas as pd

import microgrid_core as mc
import solution_lib as sl

```

Listing 9: 补充敏感性计算.py

```

BASE = {
    "ETA_C": mc.ETA_C, "ETA_D": mc.ETA_D, "E_MAX": mc.E_MAX,
    "S_MIN": mc.S_MIN, "S_MAX": mc.S_MAX, "P_MAX": mc.P_MAX,
    "B_MAX": mc.B_MAX, "SO": mc.SO,
}

def _set_parameters(cfg: dict) -> None:
    for key, value in BASE.items():
        setattr(mc, key, value)
    if "eta" in cfg:
        mc.ETA_C = mc.ETA_D = float(cfg["eta"])
    if "capacity_factor" in cfg:
        factor = float(cfg["capacity_factor"])
        mc.E_MAX = BASE["E_MAX"] * factor
        mc.S_MIN = 0.1 * mc.E_MAX
        mc.S_MAX = 0.9 * mc.E_MAX
    if "power_factor" in cfg:
        factor = float(cfg["power_factor"])
        mc.P_MAX = BASE["P_MAX"] * factor
        mc.B_MAX = mc.P_MAX * mc.DT
    if "soc_ratio" in cfg:
        mc.SO = float(cfg["soc_ratio"]) * mc.E_MAX

def _worker(cfg: dict) -> dict:
    _set_parameters(cfg)
    data = mc.load_inputs()
    resid = sl.net_residual_matrix("F1", data)
    gamma = float(cfg.get("gamma", 1.0))
    run = sl.run_year_q2(data, "F1", 0.805, 35, resid=resid * gamma,
                        horizon=49, n_days=365, executor="mpc")
    metrics = sl.year_metrics(run)
    gates = sl.check_run(run)
    return {
        **cfg,
        **{k: v for k, v in metrics.items() if k != "daily_cash"},
        "eta_c": mc.ETA_C, "eta_d": mc.ETA_D,
        "capacity_kWh": mc.E_MAX, "power_kW": mc.P_MAX,
        "S_min_kWh": mc.S_MIN, "S_max_kWh": mc.S_MAX,
        "SO_kWh": mc.SO,
        "numeric_ok": bool(
            gates["balance_max"] <= 1e-6
            and gates["overlap_max"] <= 1e-5
            and gates["bound_S"] <= 1e-7
        ),
    }

def configs() -> list[dict]:
    return [
        {"group": "基准", "scenario": "基准"},
        {"group": "效率", "scenario": "eta=sqrt(0.9)", "eta": math.sqrt(0.9)},
        {"group": "初始SOC", "scenario": "初始SOC=20%", "soc_ratio": 0.2},
        {"group": "初始SOC", "scenario": "初始SOC=80%", "soc_ratio": 0.8},
        {"group": "容量", "scenario": "容量-20%", "capacity_factor": 0.8},
        {"group": "容量", "scenario": "容量+20%", "capacity_factor": 1.2},
        {"group": "功率", "scenario": "功率-20%", "power_factor": 0.8},
        {"group": "功率", "scenario": "功率+20%", "power_factor": 1.2},
        {"group": "误差倍率", "scenario": "gamma=0.5", "gamma": 0.5},
        {"group": "误差倍率", "scenario": "gamma=1.5", "gamma": 1.5},
        {"group": "误差倍率", "scenario": "gamma=2.0", "gamma": 2.0},
    ]

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--workers", type=int, default=0)
    args = ap.parse_args()
    cfgs = configs()
    workers = args.workers or min(4, max(1, (os.cpu_count() or 4) - 2))
    t0 = time.perf_counter()
    print(f"[SENS] scenarios={len(cfgs)} workers={workers}", flush=True)
    if workers <= 1:
        rows = [_worker(c) for c in cfgs]
    else:
        with Pool(workers) as pool:
            rows = pool.map(_worker, cfgs)
    base = next(r for r in rows if r["scenario"] == "基准")
    for group, scenario, extra in [
        ("效率", "eta=0.9/0.9", {"eta": 0.9}),
        ("初始SOC", "初始SOC=50%", {"soc_ratio": 0.5}),
        ("误差倍率", "gamma=1.0", {"gamma": 1.0}),
    ]:
        alias = dict(base)
        alias.update(group=group, scenario=scenario, **extra)
        rows.append(alias)
    df = pd.DataFrame(rows)
    base_cost = float(df.loc[df["scenario"] == "基准", "cash_total"].iloc[0])
    df["相对基准费用变化_元"] = df["cash_total"] - base_cost
    df["相对基准费用变化_pct"] = (df["cash_total"] / base_cost - 1.0) * 100.0
    sl.CSV.mkdir(parents=True, exist_ok=True)
    common = ["group", "scenario", "eta_c", "eta_d", "capacity_kWh", "power_kW", "SO_kWh", "cash_total", "相对基准费用变化_元", "相对基准费用变化_pct", "contract_kwh", "emergency_kwh", "emergency_slots", "charge_kwh", "discharge_kwh", "unused_contract_kwh", "S_min", "S_max", "S_end", "numeric_ok"]
    structural = df[df["group"] != "误差倍率"][common].copy()
    pressure = df[df["group"] == "误差倍率"][["gamma"] + common].copy()
    structural.to_csv(sl.CSV / "模型评价_结构敏感性.csv", index=False, encoding="utf-8-sig")
    pressure.to_csv(sl.CSV / "模型评价_误差倍率压力测试.csv", index=False, encoding="utf-8-sig")
    with pd.ExcelWriter(mc.RESULT_DIR / "补充分析结果.xlsx", engine="openpyxl") as writer:
        structural.to_excel(writer, sheet_name="结构敏感性", index=False)
        pressure.to_excel(writer, sheet_name="误差倍率压力测试", index=False)
    summary = {
        "scenario_count": len(df),
        "all_numeric_ok": bool(df["numeric_ok"].all()),
        "elapsed_s": round(time.perf_counter() - t0, 1),
    }
    with open(sl.CSV / "模型评价_补充计算门禁.json", "w", encoding="utf-8") as fh:
        json.dump(summary, fh, ensure_ascii=False, indent=2)
    print(json.dumps(summary, ensure_ascii=False, indent=2), flush=True)

```

```
if __name__ == "__main__":
    main()
```

Listing 10: 终端价值敏感性.py

```
from __future__ import annotations

import json
import time

import pandas as pd

import microgrid_core as mc
import solution_lib as sl

def main() -> None:
    t0 = time.perf_counter()
    data = mc.load_inputs()
    resid = sl.net_residual_matrix("F1", data)
    rows = []
    for factor in (0.0, 0.5, 1.0, 1.5):
        run = sl.run_year_q2(data, "F1", 0.805, 35, resid=resid,
                             horizon=49, n_days=365, executor="mpc",
                             terminal_value_factor=factor)
        m = sl.year_metrics(run)
        g = sl.check_run(run)
        rows.append({
            "终端价值倍率": factor,
            "现金费用_元": m["cash_total"],
            "合同购电量_kWh": m["contract_kwh"],
            "应急购电量_kWh": m["emergency_kwh"],
            "充电量_kWh": m["charge_kwh"],
            "放电量_kWh": m["discharge_kwh"],
            "弃购量_kWh": m["unused_contract_kwh"],
            "期末SOC_kWh": m["S_end"],
            "SOC下限触及比例": float((run["S"] <= mc.S_MIN + 1e-7).mean()),
            "SOC上限触及比例": float((run["S"] >= mc.S_MAX - 1e-7).mean()),
            "数值门禁": bool(g["balance_max"] <= 1e-6 and
                             g["overlap_max"] <= 1e-5 and
                             g["bound_S"] <= 1e-7 and
                             g["soc_chain_max_kWh"] <= 1e-6),
        })
    df = pd.DataFrame(rows)
    base = float(df.loc[df["终端价值倍率"] == 1.0, "现金费用_元"].iloc[0])
    df["相对倍率1费用变化_元"] = df["现金费用_元"] - base
    df["相对倍率1费用变化_pct"] = (df["现金费用_元"] / base - 1.0) * 100.0
    sl.CSV.mkdir(parents=True, exist_ok=True)
    df.to_csv(sl.CSV / "终端价值敏感性.csv", index=False, encoding="utf-8-sig")
    report = {"scenario_count": len(df), "all_numeric_ok": bool(df["数值门禁"].all()),
              "elapsed_s": round(time.perf_counter() - t0, 1)}
    with open(sl.CSV / "终端价值敏感性_门禁.json", "w", encoding="utf-8") as fh:
        json.dump(report, fh, ensure_ascii=False, indent=2)
    print(json.dumps(report, ensure_ascii=False, indent=2))

if __name__ == "__main__":
```

```
main()
```

Listing 11: 收口计算.py

```
from __future__ import annotations

import json
import math
import time
from pathlib import Path

import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl

ROOT = Path(__file__).resolve().parent
CSV = ROOT / "results" / "csv"
OFF = 31
END = 365

def load_npz(name):
    return {k: v for k, v in np.load(CSV / name).items()}

def fixed_price(data):
    return np.tile(data["price1"], (END, 1))

def actual_price(data):
    return data["price_act"]

def fee_parts(P, Q, U, price, q3):
    P, Q, U, price = map(lambda a: np.asarray(a, dtype=float), (P, Q, U, price))
    if price.ndim == 1:
        price = np.tile(price, (P.shape[0], 1))
    over = np.maximum(Q - P, 0.0)
    under = np.maximum(P - Q, 0.0)
    plan_a = (price * np.minimum(P, Q)).sum(axis=1)
    adj = (1.5 * price * over + 0.5 * price * under).sum(axis=1) if q3 else np.zeros(P.shape[0])
    plan_b = (price * P).sum(axis=1)
    emg = (5.0 * price * U).sum(axis=1)
    return {"plan_A": plan_a, "plan_B": plan_b, "adjust": adj, "emergency": emg,
            "total_A": plan_a + adj + emg, "total_B": plan_b + adj + emg}

def diag_row(label, arrays, price, q3, data):
    parts = fee_parts(arrays["P"], arrays["Q"] if q3 else arrays["P"],
                      arrays["U"], price, q3)
    X, Y, S = arrays["X"], arrays["Y"], arrays["S"]
    plan_x = arrays.get("plan_x", np.full_like(X, np.nan))
    plan_y = arrays.get("plan_y", np.full_like(Y, np.nan))
    plan_s = arrays.get("plan_s_next", np.full_like(S, np.nan))
    slc = slice(OFF, END)
    soc_e = S[slc]
    soc_p = plan_s[slc]
    finite = np.isfinite(soc_p)
    e = soc_e[finite] - soc_p[finite]
```

```

cash = parts["total_A"][slc]
row = {
    "方案": label, "口径A现金总费用_元": float(parts["total_A"][slc].sum()),
    "计划费用A_元": float(parts["plan_A"][slc].sum()),
    "调整费用_元": float(parts["adjust"][slc].sum()),
    "紧急费用_元": float(parts["emergency"][slc].sum()),
    "口径B现金总费用_元": float(parts["total_B"][slc].sum()),
    "口径B相对A_元": float((parts["total_B"] - parts["total_A"])[slc].sum()),
    "计划购电量_kWh": float(arrays["P"][slc].sum()),
    "最终合同量_kWh": float((arrays["Q"] if q3 else arrays["P"])[slc].sum()),
    "调整量_kWh": float(((arrays["Q"] - arrays["P"]) if q3 else np.zeros_like(arrays["P"])[slc].sum()),
    "应急购电量_kWh": float(arrays["U"][slc].sum()),
    "弃购量_kWh": float(arrays["W"][slc].sum()),
    "弃光量_kWh": float((data["pv_act"] * mc.DT - arrays["V"])[slc].sum()),
    "充电量_kWh": float(X[slc].sum()), "放电量_kWh": float(Y[slc].sum()),
    "期末SOC_kWh": float(S[-1, -1]),
    "SOC一步MAE_kWh": float(np.abs(e).mean() if e.size else np.nan),
    "SOC一步RMSE_kWh": float(np.sqrt(np.mean(e * e)) if e.size else np.nan),
    "SOC一步最大绝对误差_kWh": float(np.abs(e).max() if e.size else np.nan),
    "充电一步MAE_kWh": float(np.abs(X[slc][finite] - plan_x[slc][finite]).mean() if e.size else np.nan),
    "放电一步MAE_kWh": float(np.abs(Y[slc][finite] - plan_y[slc][finite]).mean() if e.size else np.nan),
    "费用加总误差_元": float(np.max(np.abs(parts["total_A"] - (parts["plan_A"] + parts["adjust"] + parts["emergency"])[slc])),
}

return row

def settlement_sensitivity(data, q3, q4):
    rows = []
    q3_main = {k[len("main-")]: v for k, v in q3.items() if k.startswith("main-")}
    q4_main = {k[len("Q4-3_median30-")]: v for k, v in q4.items() if k.startswith("Q4-3_median30-")}
    specs = [
        ("Q3主方案", q3_main, fixed_price(data), True),
        ("Q4-3_median30", q4_main, actual_price(data), True),
    ]

    for tag in ("main", "abl_none", "abl_1", "abl_2", "abl_3", "abl_1_2", "abl_1_3", "abl_2_3"):
        a = {k[len(tag)+1:]: v for k, v in q3.items() if k.startswith(tag + "_")}
        p = fee_parts(a["P"], a["Q"], a["U"], fixed_price(data), True)
        rows.append({"问题": "Q3", "方案": tag, "口径A总费用_元": p["total_A"][OFF:END].sum(),
                    "口径B总费用_元": p["total_B"][OFF:END].sum(),
                    "口径B-A_元": (p["total_B"] - p["total_A"])[OFF:END].sum()})

    for label, a, price, q3flag in specs:
        p = fee_parts(a["P"], a["Q"] if q3flag else a["P"], a["U"], price, q3flag)
        rows.append({"问题": label.split("-")[0], "方案": label, "口径A总费用_元": p["total_A"][OFF:END].sum(),
                    "口径B总费用_元": p["total_B"][OFF:END].sum(),
                    "口径B-A_元": (p["total_B"] - p["total_A"])[OFF:END].sum()})

    df = pd.DataFrame(rows)
    df["口径A排序"] = df.groupby("问题")["口径A总费用_元"].rank(method="min")
    df["口径B排序"] = df.groupby("问题")["口径B总费用_元"].rank(method="min")
    return df

def tail_risk(label, daily, dates):
    x = np.asarray(daily, dtype=float)[OFF:END]
    q = float(np.quantile(x, 0.95, method="linear"))
    tail = x[x >= q]
    i = int(np.argmax(x))
    d = pd.Timestamp(str(dates[OFF:END][i])).date()
    return {"方案": label, "日均费用_元": float(x.mean()), "VaR95_元": q,
            "CVaR95_元": float(tail.mean()), "最坏日费用_元": float(x.max()),
            "最坏日": str(d), "最坏5日均费用_元": float(np.sort(x)[-5:].mean())}

def monthly_pair(label_a, a, label_b, b, dates):
    da = pd.DataFrame({"日期": pd.to_datetime(dates[OFF:END]), "a": a[OFF:END], "b": b[OFF:END]})
    da["月份"] = da["日期"].dt.strftime("%Y-%m")
    out = da.groupby("月份").agg(方案A日均费用_元=(a, "mean"), 方案B日均费用_元=(b, "mean"),
                               天数=(a, "size")).reset_index()
    out["A相对B日均差_元"] = out["方案A日均费用_元"] - out["方案B日均费用_元"]
    out.insert(0, "方案A", label_a); out.insert(1, "方案B", label_b)
    return out

def time_audit(data, q2, q3, q4):
    cols = mc._TIME_COLS
    checks = []
    for i, c in enumerate(cols):
        checks.append({"槽序号": i + 1, "原始列标签": c, "自然区间": f"{mc.slot_label_start(i)}-{mc.slot_label_end(i)}",
                      "是否首槽": i == 0, "是否末槽": i == mc.T - 1})
    df = pd.DataFrame(checks)
    boundary = pd.DataFrame([
        {"审计项": "附件2首列", "结果": cols[0], "判定": "第1槽按0:00-0:10解释"},
        {"审计项": "附件2末列", "结果": cols[-1], "判定": "第144槽按23:50-24:00解释"},
        {"审计项": "官方发布槽", "结果": "0,36,72,108", "判定": "对应0/6/12/18时刻"},
        {"审计项": "重点槽", "结果": "60,72,84,96,108,120", "判定": "对应0:00至20:00起始槽"},
        {"审计项": "跨午夜应急合并", "结果": "不跨午夜", "判定": "连续区间按自然日截断"},
    ])

    rows = []
    dates = np.array([str(mc.date_of(d)) for d in range(END)])
    for label, a, price, q3flag in [("Q2", q2, np.tile(data["price1"], (END, 1)), False),
                                    ("Q3", q3, np.tile(data["price1"], (END, 1)), True)]:
        for shift in (-1, 0, 1):
            pp = np.roll(price, shift, axis=1)

```

```

        p = fee_parts(a["P"], a["Q"] if q3flag else a["P"], a["U"],
                      pp, q3flag)
        rows.append({"方案":label, "价格整体平移槽数":shift,
                     "重放费用_元":p["total_A"][OFF:END].sum()})
    audit = pd.DataFrame(rows)
    baseline = audit.loc[audit["价格整体平移槽数"] == 0].set_index("方案")["重放费用_元"]
    audit["相对原映射_元"] = audit["重放费用_元"] - audit["方案"].map(baseline)
    return pd.concat([df, boundary, audit], ignore_index=True)

def main():
    t0 = time.perf_counter()
    data = mc.load_inputs()
    q2 = load_npz("q2_bundle.npz")
    q3 = load_npz("q3_bundle.npz")
    q4 = load_npz("q4_bundle.npz")
    dates = np.array([str(mc.date_of(d)) for d in range(END)])
    q2a = {k: v for k, v in q2.items()}
    q3a = {k[len("main-")]: v for k, v in q3.items() if k.startswith("main-")}
    q4a = {k[len("Q4-3_median30-")]: v for k, v in q4.items() if k.startswith("Q4-3_median30-")}
    q4b = {k[len("Q4-2_median30-")]: v for k, v in q4.items() if k.startswith("Q4-2_median30-")}

    main_rows = [diag_row("Q2-F1-W35-alpha0.805-H49", q2a, np.tile(
        data["price1"], (END,1)), False, data),
                 diag_row("Q3-S={6,12,18}", q3a, np.tile(data["price1"],
                     (END,1)), True, data),
                 diag_row("Q4-2-median30", q4b, data["price_act"], False,
                     data),
                 diag_row("Q4-3-median30", q4a, data["price_act"], True,
                     data)]
    pd.DataFrame(main_rows).to_csv(CSV / "总体主表_费用与执行诊断.csv",
                                   index=False, encoding="utf-8-sig")

    settle = settlement_sensitivity(data, q3, q4)
    settle.to_csv(CSV / "结算歧义敏感性.csv", index=False, encoding="utf-8-sig")
    time_audit(data, q2a, q3a, q4a).to_csv(CSV / "时间标签索引审计.csv",
                                             index=False, encoding="utf-8-sig")

    rows = []
    for label, a, price, q3flag in [("Q2", q2a, np.tile(data["price1"],
        (END,1)), False),
                                    ("Q3", q3a, np.tile(data["price1"], (
        END,1)), True),
                                    ("Q4-2", q4b, data["price_act"], False),
                                    ("Q4-3", q4a, data["price_act"], True)
                                    ]:
        for date in sl.REP_DATES:
            d = (date - mc.DAY0).days
            if d >= END: continue
            c = price[d] if price.ndim == 2 else price
            Q = a["Q"] if q3flag else a["P"]
            rows.append({"方案":label, "日期":str(date), "全天购电费用_元":float(fee_parts(a["P"][d:d+1], Q[d:d+1], a["U"][d:d+1], c, q3flag)["total_A"][0]),
                         "计划购电量_kWh":a["P"][d].sum(), "最终合同量_kWh":Q[d].sum(), "充电量_kWh":a["X"][d].sum(),
                         "放电量_kWh":a["Y"][d].sum(), "应急购电量_kWh":a["U"][d].sum(), "弃购量_kWh":a["W"][d].sum()
                         },
                         {"pv_act":(data["pv_act"][d]*mc.DT-a["V"][d]).sum(), "0时SOC_kWh":(a["S"][d-1,-1] if d
                         else mc.S0), "24时SOC_kWh":a["S"][d,-1]})
    pd.DataFrame(rows).to_csv(CSV / "重点日期_统一诊断表.csv", index=False, encoding="utf-8-sig")

    risk = pd.DataFrame([
        tail_risk("Q2-F1-W35-alpha0.805-H49", q2["cash"], dates),
        tail_risk("Q3-S={6,12,18}", q3["main_cash"], dates),
        tail_risk("Q4-2-median30", q4["Q4-2_median30_cash"], dates),
        tail_risk("Q4-3-median30", q4["Q4-3_median30_cash"], dates),
    ])
    risk.to_csv(CSV / "尾部风险_日费用VaR_CVaR.csv", index=False, encoding="utf-8-sig")
    monthly_pair("Q3-S={6,12,18}", q3["main_cash"], "Q3-空集", q3["abl_none_cash"], dates).to_csv(CSV / "尾部风险_月度配对差.csv",
                                                             index=False, encoding="utf-8-sig")

    checks = {"finite": True, "curtail_nonnegative_max_violation_kWh": 0.0,
              "fee_decomposition_max_error_yuan": 0.0, "required_files": [], "elapsed_s": 0.0}
    for a in (q2a, q3a, q4a, q4b):
        for k, v in a.items():
            if isinstance(v, np.ndarray) and not np.isfinite(v).all():
                checks["finite"] = False
            checks["curtail_nonnegative_max_violation_kWh"] = max(checks["curtail_nonnegative_max_violation_kWh"], float(np.maximum(a["V"] - data["pv_act"] * mc.DT, 0).max()))
            checks["required_files"] = [p.name for p in (CSV / "总体主表_费用与执行诊断.csv", CSV / "结算歧义敏感性.csv", CSV / "时间标签索引审计.csv", CSV / "重点日期_统一诊断表.csv", CSV / "尾部风险_日费用VaR_CVaR.csv", CSV / "尾部风险_月度配对差.csv") if p.exists()]
            checks["elapsed_s"] = round(time.perf_counter() - t0, 1)
    with open(CSV / "结果文件校验.json", "w", encoding="utf-8") as f:
        json.dump(checks, f, ensure_ascii=False, indent=2)
    print(json.dumps(checks, ensure_ascii=False, indent=2))

if __name__ == "__main__": main()

```

Listing 12: 汇总结果.py

```

from __future__ import annotations

import json
import math
from pathlib import Path

import numpy as np
import openpyxl
import pandas as pd

ROOT = Path(__file__).resolve().parent
CSV = ROOT / "results" / "csv"
RES = ROOT / "results"

def read(name: str) -> pd.DataFrame:
    return pd.read_csv(CSV / name)

```

```

def clean_tables() -> dict[str, pd.DataFrame]:
    q2s = read("问题2_配置敏感性.csv").drop(columns=["daily_cash"],
        errors="ignore")
    q2r = read("问题2_参照方案.csv").drop(columns=["daily_cash"],
        errors="ignore")
    q3a = read("问题3_信息消融.csv")
    q3a.loc[q3a["tag"] == "abl_none", "获准集合S"] = "{}"
    q3c = read("问题3_方案对比.csv")
    q4e = read("问题4_价格基线误差.csv")
    q4c = read("问题4_方案对比.csv")
    st = read("模型评价_结构敏感性.csv")
    ps = read("模型评价_误差倍率压力测试.csv")
    tv = read("终端价值敏感性.csv")
    diag = read("总体主表_费用与执行诊断.csv")
    settle = read("结算歧义敏感性.csv")
    risk = read("尾部风险_日费用VaR_CVaR.csv")
    month = read("尾部风险_月度配对差.csv")
    common = ["group", "scenario", "eta_c", "eta_d", "capacity_kWh",
        "power_kW", "S0_kWh", "cash_total", "相对基准费用变化_元",
        "相对基准费用变化_pct", "contract_kwh", "emergency_kwh",
        "emergency_slots", "charge_kwh", "discharge_kwh",
        "unused_contract_kwh", "S_min", "S_max", "S_end", "numeric_
        _ok"]
    st = st[common]
    ps = ps[["gamma"] + common]
    return {"Q2参数敏感性": q2s, "Q2参照方案": q2r,
        "Q3八组信息消融": q3a, "Q3方案对比": q3c,
        "Q4价格预测误差": q4e, "Q4方案与理想基准": q4c,
        "结构敏感性": st, "误差倍率压力测试": ps,
        "终端价值敏感性": tv, "费用执行诊断": diag,
        "结算歧义敏感性": settle, "尾部风险": risk,
        "月度配对差": month}

def key_summary(tables: dict[str, pd.DataFrame]) -> pd.DataFrame:
    q1 = read("问题1_汇总.csv").iloc[0]
    q2 = tables["Q2参数敏感性"].query("方案.str.startswith('主配置')",
        engine="python").iloc[0]
    q3 = tables["Q3八组信息消融"].query("tag == 'main']").iloc[0]
    q4 = tables["Q4方案与理想基准"].set_index("tag")
    return pd.DataFrame([
        {"问题": "Q1", "方案": "确定性LP", "现金费用_元": q1["全天购电费_
        元"],
        "合同购电量_kWh": q1["全天计划购电量_kWh"], "应急购电量_kWh":
        0.0},
        {"问题": "Q2", "方案": "F1-W35-alpha0.805-H49", "现金费用_元":
        q2["cash_total"],
        "合同购电量_kWh": q2["contract_kwh"], "应急购电量_kWh": q2["
        emergency_kwh"]},
        {"问题": "Q3", "方案": "S={6,12,18}", "现金费用_元": q3["现金总
        费_元"],
        "合同购电量_kWh": q3["最终合同量_kWh"], "应急购电量_kWh": q3["应
        急购电量_kWh"]},
        {"问题": "Q4-2", "方案": "median30固定合同", "现金费用_元": q4.
        loc["Q4-2_median30", "cash_total"],
        "合同购电量_kWh": q4.loc["Q4-2_median30", "contract_kwh"],
        "应急购电量_kWh": q4.loc["Q4-2_median30", "emergency_kwh"]},
        {"问题": "Q4-3", "方案": "median30可调合同", "现金费用_元": q4.
        loc["Q4-3_median30", "cash_total"],
        "合同购电量_kWh": q4.loc["Q4-3_median30", "contract_kwh"],
        "应急购电量_kWh": q4.loc["Q4-3_median30", "emergency_kwh"]},
    ])

```

```

def autosize(path: Path) -> None:
    wb = openpyxl.load_workbook(path)
    for ws in wb.worksheets:
        ws.freeze_panes = "A2"
        ws.auto_filter.ref = ws.dimensions
        for col in ws.columns:
            letter = col[0].column_letter
            width = min(28, max(10, max(len(str(c.value or "")) for c
                in col) + 2))
            ws.column_dimensions[letter].width = width
    wb.save(path)

def validate_answer_workbooks() -> dict:
    names = ["result1.xlsx", "result2.xlsx", "result3.xlsx",
        "result4-2.xlsx", "result4-3.xlsx"]
    details = []
    all_ok = True
    for name in names:
        template = openpyxl.load_workbook(ROOT / "data" / "附件" / "附
            件5" / name,
            data_only=False)
        result = openpyxl.load_workbook(RES / name, data_only=False)
        same_sheets = template.sheetnames == result.sheetnames
        same_dimensions = all(template[s].calculate_dimension() ==
            result[s].calculate_dimension()
            for s in template.sheetnames)
        errors, nonfinite, annual_complete = [], [], True
        for ws in result.worksheets:
            for row in ws.iter_rows():
                for cell in row:
                    value = cell.value
                    if isinstance(value, str) and value.startswith("#"):
                        errors.append(f"{ws.title}!{cell.coordinate}:{
                            value}")
                    if isinstance(value, (int, float)) and not math.
                        isfinite(float(value)):
                        nonfinite.append(f"{ws.title}!{cell.coordinate}")
                    if ws.max_row == 335 and ws.max_column == 147:
                        annual_complete = annual_complete and all(
                            ws.cell(r, c).value is not None
                            for r in range(2, 336) for c in range(1, 148))
        ok = same_sheets and same_dimensions and not errors and not
            nonfinite and annual_complete
        all_ok = all_ok and ok
        details.append({"file": name, "same_sheets": same_sheets,
            "same_dimensions": same_dimensions,
            "annual_cells_complete": annual_complete,
            "excel_errors": errors, "nonfinite_cells":
                nonfinite,
            "ok": ok})
    return {"all_ok": all_ok, "details": details}

def main() -> None:
    tables = clean_tables()
    summary = key_summary(tables)
    summary.to_csv(CSV / "全题关键指标.csv", index=False, encoding="utf
        -8-sig")
    tables["Q2参数敏感性"].to_csv(CSV / "问题2_配置敏感性.csv", index=
        False, encoding="utf-8-sig")
    tables["Q2参照方案"].to_csv(CSV / "问题2_参照方案.csv", index=False
        , encoding="utf-8-sig")
    tables["Q3八组信息消融"].to_csv(CSV / "问题3_信息消融.csv", index=
        False, encoding="utf-8-sig")
    tables["结构敏感性"].to_csv(CSV / "模型评价_结构敏感性.csv", index=

```

```

        False, encoding="utf-8-sig")
tables["误差倍率压力测试"].to_csv(CSV / "模型评价_误差倍率压力测试.
    csv", index=False, encoding="utf-8-sig")
out = RES / "补充分析结果.xlsx"
with pd.ExcelWriter(out, engine="openpyxl") as writer:
    summary.to_excel(writer, sheet_name="全题关键指标", index=False
    )
    for name, df in tables.items():
        df.to_excel(writer, sheet_name=name, index=False)
autosize(out)
answer_check = validate_answer_workbooks()
report = {"workbook": str(out), "sheets": 1 + len(tables),
    "rows": {"全题关键指标": len(summary),
        **{k: len(v) for k, v in tables.items()}},
    "finite_numeric_nonmissing": bool(all(
        np.isfinite(a[-np.isnan(a)]).all()

```

```

    for df in [summary, *tables.values()])
    for a in [df.select_dtypes(include=[np.number]).to_
        numpy()])),
    "official_answer_files": ["result1.xlsx", "result2.xlsx",
        "result3.xlsx",
            "result4-2.xlsx", "result4-3.xlsx"
        ],
    "official_answer_check": answer_check}
with open(CSV / "结果文件校验.json", "w", encoding="utf-8") as fh:
    json.dump(report, fh, ensure_ascii=False, indent=2)
print(json.dumps(report, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```